Industrial Agentic Cybernetics

Design Patterns and Formal Architecture
for Autonomous Software Engineering

Bin Wang

Beijing Kailuzhe Technology Co., Ltd.

DOI: https://doi.org/10.5281/zenodo.23636677

License: Code MIT · Text CC BY 4.0

Version v1.1.2

English edition note: This English edition is an AI-assisted translation of the original Chinese text, reviewed and finalized by the author. The Chinese edition remains the authoritative version; in case of any discrepancy, the Chinese text prevails.

Concept DOI resolves to the latest archived version.
Code released under the MIT License; book text under CC BY 4.0.

Contents

Chapter 1: The Paradigm Shift — From Discrete Prompting to Closed-Loop Cybernetics **1**
1.1 The Predicament of Historical Evolution: Toy-Level Assumptions and the Prompt Fallacy . . . 1
1.2 The True Physical Character of the Industrial-Grade Software Substrate . . . 2
1.3 A Cybernetic Perspective: The Leap from First-Order Open Loop to Second-Order Closed Loop . . . 3
1.4 The Architecture of This Book and Reading Guide . . . 4

Chapter 2: A Formal Analysis of Reward Hacking and Substrate Breakthrough **11**
2.1 What Is "Reward Hacking" in Agents? . . . 11
2.2 A Game-Theoretic Formal Analysis of Reward Hacking . . . 12
2.3 The Physical Breakthrough: Three Cybernetic Iron Laws to Eradicate Reward Hacking . . . 13

Chapter 3: Four Cybernetic Axioms and Causal Invariance Formulation **15**
3.1 Introduction: The Invariance of the Software Agent State Space . . . 15
3.2 The Four Foundational Axioms of Industrial Agents . . . 15
3.3 Theorem of Causal Invariance . . . 23
3.4 Open Problems . . . 24

Chapter 4: Pattern 01 — Substrate Penetration Pattern **27**
1. Intent . . . 27
2. Motivation & Real-World Failure Modes . . . 27
3. Applicability . . . 28
4. Participants . . . 28
5. Collaborations . . . 28
6. Consequences . . . 29
7. Anti-Patterns to Avoid . . . 29
8. Algorithmic Formulation . . . 30

Chapter 5: Pattern 02 — Reality Audit VETO Pattern **31**
1. Intent . . . 31
2. Motivation & Real-World Failure Modes . . . 31
3. Applicability . . . 32
4. Participants . . . 32
5. Collaborations . . . 32
6. Consequences . . . 33

7. Anti-Patterns to Avoid . . . 33
8. Algorithmic Formulation . . . 34
9. Formal Theory Support (Lean 4 Formal Proof) . . . 35

Chapter 6: Pattern 03 — Two-Phase Mandate Pattern 37
1. Intent . . . 37
2. Motivation & Real-World Failure Modes . . . 37
3. Applicability . . . 38
4. Participants . . . 38
5. Collaborations . . . 38
6. Consequences . . . 39
7. Anti-Patterns to Avoid . . . 39
8. Algorithmic Formulation . . . 39
9. Epistemic Status . . . 40

Chapter 7: Pattern 04 — Task-Level PoLP Context Slicing Pattern 41
1. Intent . . . 41
2. Motivation & Real-World Failure Modes . . . 41
3. Applicability . . . 42
4. Participants . . . 42
5. Collaborations . . . 42
6. Consequences . . . 43
7. Anti-Patterns to Avoid . . . 43
8. Algorithmic Formulation . . . 44
9. Lean 4 Formal Proof . . . 45

Chapter 8: Pattern 05 — Macro-Micro Elaboration Pattern 47
1. Intent . . . 47
2. Motivation & Real-World Failure Modes . . . 47
3. Applicability . . . 48
4. Participants . . . 49
5. Collaborations . . . 49
6. Consequences . . . 50
7. Anti-Patterns to Avoid . . . 50
8. Algorithmic Formulation & Contract Schema . . . 51
9. Epistemic Status of Quantitative Claims . . . 53

Chapter 9: Pattern 06 — Streaming Decomposition & Selective Resume Pattern 55
1. Intent . . . 55
2. Motivation & Real-World Failure Modes . . . 55
3. Applicability . . . 56
4. Participants . . . 56
5. Collaborations . . . 57
6. Consequences . . . 57
7. Anti-Patterns to Avoid . . . 58
8. Algorithmic Formulation . . . 58
9. Epistemic Status of Quantitative Claims . . . 59

Chapter 10: Pattern 07 — Driver-Navigator Pairing Pattern 61
1. Intent . . . 61

2. Motivation & Real-World Failure Modes 61
3. Applicability 62
4. Participants 62
5. Collaborations 62
6. Consequences 63
7. Anti-Patterns to Avoid 63
8. Algorithmic Formulation 63
9. Epistemic Status of Quantitative Claims 64

Chapter 11: Pattern 08 — Implicit Branch Sniffing & Internal Invariants Pattern 65
1. Intent 65
2. Motivation & Real-World Failure Modes 65
3. Applicability 66
4. Participants 66
5. Collaborations 67
6. Consequences 67
7. Anti-Patterns to Avoid 67
8. Algorithmic Formulation 68
9. Epistemic Status of Quantitative Claims 69

Chapter 12: Pattern 09 — Idempotent Journaling & Mtime Touch Pattern 71
1. Intent 71
2. Motivation & Real-World Failure Modes 71
3. Applicability 72
4. Participants 72
5. Collaborations 72
6. Consequences 73
7. Anti-Patterns to Avoid 73
8. Algorithmic Formulation 74

Chapter 13: Pattern 10 — Substrate Lease Lock & Watchdog Pattern 77
1. Intent 77
2. Motivation & Real-World Failure Modes 77
3. Applicability 78
4. Participants 78
5. Collaborations 78
6. Consequences 79
7. Anti-Patterns to Avoid 79
8. Algorithmic Formulation 80

Chapter 14: Pattern 11 — Primary-Writer Replica-Reader Pattern 83
1. Intent 83
2. Motivation & Real-World Failure Modes 83
3. Applicability 84
4. Participants 84
5. Collaborations 85
6. Consequences 85
7. Anti-Patterns to Avoid 85
8. Algorithmic Formulation 86
9. Epistemic Status and Formal Anchors 86

Chapter 15: Pattern 12 — Peer-Agent Protocol Bridge Pattern **89**
1. Intent . . . 89
2. Motivation & Real-World Failure Modes . . . 89
3. Applicability . . . 90
4. Participants . . . 90
5. Collaborations . . . 90
6. Consequences . . . 91
6.1 Typical Extension Scenario: Asynchronous Result Backflow from Isolated Environments (Stress-Test Feedback Loop) . . . 91
7. Anti-Patterns to Avoid . . . 92
8. Algorithmic Formulation . . . 92

Chapter 16: Pattern 13 — Dual-Mode Technical Crystallization Pattern **95**
1. Intent . . . 95
2. Motivation & Real-World Failure Modes . . . 95
3. Applicability . . . 96
4. Participants . . . 96
5. Collaborations . . . 96
6. Consequences . . . 97
7. Anti-Patterns to Avoid . . . 97
8. Algorithmic Formulation . . . 98
9. Epistemic Status of Quantitative Claims . . . 99

Chapter 17: Pattern 14 — Quarantined Promotion Gate Pattern **101**
1. Intent . . . 101
2. Motivation & Real-World Failure Modes . . . 101
3. Applicability . . . 102
4. Participants . . . 102
5. Collaborations . . . 103
6. Consequences . . . 104
7. Anti-Patterns to Avoid . . . 104
8. Algorithmic Formulation . . . 104

Chapter 18: Pattern 15 — Tainted Pattern Rejection & Anti-Degradation Pattern **107**
1. Intent . . . 107
2. Motivation & Real-World Failure Modes . . . 107
3. Applicability . . . 108
4. Participants . . . 108
5. Collaborations . . . 109
6. Consequences . . . 109
7. Anti-Patterns to Avoid . . . 110
8. Algorithmic Formulation . . . 110
9. Formal-Theoretic Support (Lean 4 Formal Proof) . . . 111

Chapter 19: Pattern 16 — Accordion Multiresolution Projection Pattern **113**
1. Intent . . . 113
2. Motivation & Real-World Failure Modes . . . 113
3. Applicability . . . 114
4. Participants . . . 114

5. Collaborations . . . 114
6. Consequences . . . 115
7. Anti-Patterns to Avoid . . . 115
8. Algorithmic Formulation . . . 116
9. Epistemic Status . . . 116

Chapter 20: Pattern 17 — Asynchronous Flight-Director Interrupt Pattern 119
1. Intent . . . 119
2. Motivation & Real-World Failure Modes . . . 119
3. Applicability . . . 120
4. Participants . . . 120
5. Collaborations . . . 121
6. Consequences . . . 121
7. Anti-Patterns to Avoid . . . 121
8. Algorithmic Formulation . . . 122
9. Epistemic Status . . . 123

Chapter 21: Pattern 18 — Hidden Oracle Sandbox Pattern 125
1. Intent . . . 125
2. Motivation & Real-World Failure Modes . . . 125
3. Applicability . . . 126
4. Participants . . . 126
5. Collaborations . . . 126
6. Consequences . . . 127
7. Anti-Patterns to Avoid . . . 127
8. Algorithmic Formulation . . . 127
9. Isolated-Environment Extension: Stress-Test Hidden Oracle . . . 128

Chapter 22: Pattern 19 — Five-Fold Bypass-Free Evaluator Pattern 133
1. Intent . . . 133
2. Motivation & Real-World Failure Modes . . . 133
3. Applicability . . . 134
4. Participants . . . 134
5. Collaborations . . . 134
6. Consequences . . . 135
7. Anti-Patterns to Avoid . . . 135
8. Algorithmic Formulation . . . 135
9. Epistemic Status . . . 137

Chapter 23: Pattern 20 — Industrial Substrate A/B Benchmark Pattern 139
1. Intent . . . 139
2. Motivation & Real-World Failure Modes . . . 139
3. Applicability . . . 140
4. Participants . . . 140
5. Collaborations . . . 141
6. Consequences . . . 141
7. Anti-Patterns to Avoid . . . 141
8. Algorithmic Formulation . . . 142
9. Epistemic Status of Quantitative Claims . . . 143

Chapter 24: Pattern 21 — Tokenomics Convergence Architecture Pattern 145
1. Intent . . . 145
2. Motivation & Real-World Failure Modes . . . 145
3. Applicability . . . 146
4. Participants . . . 146
5. Collaborations . . . 146
6. Consequences . . . 147
7. Anti-Patterns to Avoid . . . 147
8. Algorithmic Formulation . . . 148
9. Epistemic Status of Quantitative Claims . . . 148

Chapter 25: Pattern 22 — Pure Native Provider Bridge Pattern 149
1. Intent . . . 149
2. Motivation & Real-World Failure Modes . . . 149
3. Applicability . . . 150
4. Participants . . . 150
5. Collaborations . . . 151
6. Consequences . . . 151
7. Anti-Patterns to Avoid . . . 151
8. Algorithmic Formulation . . . 152
9. Epistemic Status of Quantitative Claims . . . 153

Chapter 26: Pattern 23 — Production Runtime Anomaly Sniffing Pattern 155
1. Intent . . . 155
2. Motivation & Real-World Failure Modes . . . 155
3. Applicability . . . 157
4. Participants and Responsibilities . . . 157
5. Collaborations and Causal Constraints . . . 158
6. Consequences . . . 159
7. Anti-Patterns to Avoid . . . 159
8. Algorithmic Formulation . . . 160
Epistemic Status and Formal Anchors . . . 161
Empirical Support (2026-10-22, Production Field Experiments H1/H4) . . . 162

Chapter 27: Pattern 24 — Production Defect Crystallization & Hotfix Loop Pattern 165
1. Intent . . . 165
2. Motivation & Real-World Failure Modes . . . 165
3. Applicability . . . 166
4. Participants and Responsibilities . . . 167
5. Collaborations and Causal Constraints . . . 167
6. Consequences . . . 168
7. Anti-Patterns to Avoid . . . 169
8. Algorithmic Formulation . . . 169
Epistemic Status and Formal Anchors . . . 171
Empirical Support (2026-10-22, Production Field Experiments H2/H3/H5) . . . 171

Chapter 28: Epilogue — Towards Autonomous and Resilient Cybernetics 173
1. Crossing the Canyon of "Random Hallucination" . . . 173
2. The System Exoskeleton Built by 24 Patterns . . . 173

3. The Horizon Ahead: A Self-Sustaining Autonomous Code Ecology 174

Appendix B: Formal Proofs of Convergence and Stability (Lean 4 Mechanized) 175
B.1 Module–Chapter Correspondence Master Table 175
B.2 Axiom I: The Irreplaceability of Grounding (All Proven at the Formalization Layer) 177
B.3 Axiom II: Attention Dilution (All Proven at the Formalization Layer) 178
B.4 Axiom III: Causal Monotonicity (All Proven at the Formalization Layer) 180
B.5 Axiom IV: The Memory-Poisoning Phase Transition (All Proven at the Formalization Layer) 181
B.6 Theorem 3.1: Convergent Causal Invariance (All Proven at the Formalization Layer) 182
B.7 Stochastic Foster–Lyapunov Expected Stopping-Time Bounds (All Proven at the Formalization Layer) 183
B.8 Pattern-Level Formal Anchor: The Lease Lock (All Proven at the Formalization Layer) 184
B.9 Pattern-Level Formal Anchor: Single-Writer Causal Order in Primary-Writer/Replica-Reader (All Proven at the Formalization Layer) 185
B.10 Pattern-Level Formal Anchor: Stability of the Observation–Learning Loop (All Proven at the Formalization Layer) 185
B.11 Pattern-Level Formal Anchor: The Reviewer Reality-Audit Veto Power (All Proven at the Formalization Layer) 186
B.12 Pattern-Level Formal Anchor: Tainted-Contamination Defense and Lineage Tombstoning (All Proven at the Formalization Layer) 186
B.13 Pattern-Level Formal Anchor: Task-Level Least-Privilege Context Projection (All Proven at the Formalization Layer) 187
B.14 Supplement to Theorem 3.1: Limit-Cycle Oscillation and Lyapunov-Contract Residual Stopping-Time Fuse (All Proven at the Formalization Layer) 187
B.15 Empirical Boundary Declarations Relative to the Main Text 188
B.16 Reproduction Instructions 189

Chapter 1: The Paradigm Shift — From Discrete Prompting to Closed-Loop Cybernetics

> **Global terminology convention (reading guide)**: "Cybernetics" is used here as an engineering metaphor of closed-loop feedback (see §1.4.3), not as an inheritance of Wiener's cybernetics discipline; "phase transition / supercritical / near-critical" are Galton-Watson branching-process terms (critical at $\rho = 1$), not thermodynamic phase transitions; "immunity" refers to lineage suppression under the experience reproduction threshold ($q > 1/2$), not biological immunity; the "attention dilution" of Axiom II corresponds to relative signal-to-noise ratio (SNR) loss, not thermodynamic entropy. Throughout this book, the term "empirical asset" uniformly refers to reusable experience entries extracted by agents.

> "In the evolution of any sufficiently complex system, reasoning that lacks physical negative feedback ultimately converges to thermodynamic entropy increase — an agent's self-referential restructuring of code, without the physical verdict of an external compiler, is in essence hallucinatory degradation in the systems-information-theoretic sense."

1.1 The Predicament of Historical Evolution: Toy-Level Assumptions and the Prompt Fallacy

In the early surge of generative large language models (LLMs), academia and early industry practice were broadly immersed in a set of utopian assumptions built on **discrete prompt engineering**. This paradigm assumed:

1. **The Markovian memorylessness assumption**: given sufficiently detailed prompt constraints, an agent's context generation at any single step can produce a globally consistent software system without destructive side effects;
2. **The reflection-based self-healing assumption**: when told "your code has errors," an agent can, relying solely on its own multi-round self-reflection and chain-of-thought, spontaneously escape its cognitive blind spots and correct itself — this assumption has been systematically falsified by empirical evidence: in the absence of external feedback, self-correction by LLMs is not only ineffective but often leads to performance degradation (Huang et al., *Large Language Models Cannot Self-Correct Reasoning Yet*, ICLR 2024);
3. **The black-box sandbox equivalence assumption**: code that runs in a single-file, temporary empty directory, or mock-stub environment detached from the real engineering

substrate is assumed to be as correct as it would be in a real enterprise multi-module monolith or microservice cluster.

This system gave rise to the early "patterns" exemplified by ReAct (Reasoning + Acting), Reflection, and multi-agent debate. Yet when enterprises put agents built on such open-source frameworks (LangChain, AutoGPT, MetaGPT, etc.) to work on **real enterprise codebases of hundreds of thousands to millions of lines**, public records and controlled experiments repeatedly report the same class of catastrophic failure — **cascade error propagation and reward hacking**: controlled experiments across 6 mainstream multi-agent frameworks show that a single erroneous claim is adopted by all agents within three rounds (5/6 frameworks with a 100% infection rate, review and QA roles failing alike; Xie et al., arXiv:2603.04474); analysis of 500+ real failure trajectories confirms error propagation as the primary bottleneck of agent reliability degradation (arXiv:2509.25370); the cascade radius of real Claude agents grows nearly linearly with pipeline depth (arXiv:2608.05263). On reward hacking, the Anthropic system card series (Claude 3.7/4/4.5/4.6) documents real instances in agentic coding; production RL evidence shows it can emerge naturally and generalize into misalignment (arXiv:2511.18397); quantitative evaluation of 13 frontier models reports exploit rates of 0%–13.9%, significantly amplified by RL post-training (arXiv:2605.02964):

```
flowchart TD
    subgraph ToyParadigm["Traditional Discrete Prompting Paradigm"]
        direction TB
        User["Developer inputs requirements"] --> Prompt["Build long-context prompt"]
        Prompt --> LLM["LLM discrete reasoning generation"]
        LLM --> Scratch["Write to temporary environment / run test scripts"]
        Scratch --> FakeGreen{"All green / mocks pass?"}
        FakeGreen -- "Yes (reward hacking)" --> FalsePass["Declare requirements complete (latent
        FakeGreen -- "No" --> SelfReflect["Blind self-reflection (infinite retry loop)"]
        SelfReflect --> LLM
    end
```

The root of this collapse is not insufficient parameter count in the models themselves, but a **paradigm-level error at the systems-theoretic level**.

1.2 The True Physical Character of the Industrial-Grade Software Substrate

"Industrial-grade" scope statement: "Industrial-grade" as used in this book refers to **software-engineering agent scenarios validated in the DevForge-AI production environment** (Docker-controlled production topology; real compilation, filesystem, and audit chains; Patterns 23/24), additionally anchored by cross-model (DeepSeek-V4 family / Kimi / Qwen2.5) and cross-task-family controlled experiments; its universality across all industrial scenarios is left to the reader to judge from the applicability-domain declarations in each chapter — this book makes no claim of full coverage.

Unlike academic benchmarks (single-file patching or simple Python repositories such as HumanEval, MBPP, and SWE-bench Lite), industrial production systems possess the following inescapable **physical reality**:

1. **Incremental compilation caches and the causality of the clock (incremental**

compilation & mtime): Large enterprise projects (e.g., Spring Cloud, Kubernetes, large front-end monorepos) generally rely on the incremental compilation caching of build tools (Maven, Gradle, Turbo, Webpack). If an agent writes code back repeatedly within milliseconds so that file modification times (mtime) do not make discrete jumps, the compiler will skip the target bytecode compilation outright, producing a false frozen state in which "the code has changed but tests still run on old bytecode."

2. **Strong multi-module coupling and implicit transitive coupling**: Modifying a single DTO field or annotation in module A can crash the runtime reflection-based serialization of downstream module C, while module A's own unit tests still pass 100%.
3. **Substrate mutation irreversibility**: Actions an agent performs on the physical host — `git reset`, port-listener binding, Docker volume mutation, partial file replacement — carry strong state-residue effects.

1.3 A Cybernetic Perspective: The Leap from First-Order Open Loop to Second-Order Closed Loop

Norbert Wiener, founder of classical cybernetics, stated in his foundational work *Cybernetics*: **the adaptive behavior of any autonomous system must rely on a negative feedback loop to continuously correct system deviation.**

In the domain of software-engineering agents, we must realize the paradigm leap from **first-order open-loop generation** to **second-order closed-loop physical cybernetics**:

```
flowchart LR
    subgraph SecondOrderCybernetics["Second-Order Cybernetic Architecture"]
        direction TB
        Goal["Requirements and architectural invariants (Goal)"] --> Orchestrator["Cognitive orch
        Orchestrator --> Agent["Executing entity (Actuator)"]
        Agent --> PhysicalSubstrate["Real physical substrate (Target Substrate: Code, FS, Process
        PhysicalSubstrate --> HardwareSensors["Objective physical sensors (Sensors: Exit Code, Di
        HardwareSensors --> VetoGate["Reality Veto Gate"]
        VetoGate -- "Physical deviation signal (Error Signal)" --> Orchestrator
        VetoGate -- "Physical convergence closure" --> Complete["Deterministic delivery (Converge
    end
```

Cybernetic element	Toy-level prompting paradigm	Industrial cybernetic architecture
Plant	The abstract prompt window and temporary directory	The physical source tree and build runtime of the real target engineering project
Actuator	Free-form arbitrary text generation and an unconstrained shell	A least-privilege projection mutator governed by a whitelist contract
Sensor	Model self-reports, mock-stub assertions, scaffold exit codes	Real compiler Exit 0, physical diff, hidden black-box probes
Controller	Blind retry loop	A convergent state machine based on state-transition graphs, fault-feature sniffing, and causal backtracking

Cybernetic element	Toy-level prompting paradigm	Industrial cybernetic architecture
Equilibrium	Ends when the textual answer ends	The physical triad: code landed, natively verified, promotion-gate approved

Note on the state of the art: CI gating that uses physical exit codes as closed-loop feedback is already mature practice in leading engineering teams (a compile failure blocks the merge; a red test rejects the delivery). This book's increment is not the concept of "introducing a closed loop" itself, but **axiomatizing** it (the four engineering axioms), **formalizing** it (Lean 4 mechanized, Appendix B), and **systematizing** it (24 patterns plus production evidence): the landing point of the paradigm leap is the system's engineering discipline and auditable evidence structure, not the invention of the closed-loop concept. **Closed-loop scope statement**: "Closed loop" as used in this book refers to the **physical negative-feedback loop at the engineering substrate layer** — physical sensors (exit code, physical diff, mtime, hidden probes) → authenticity veto gate → orchestrator → executing entity → physical substrate. This loop is established in production field experiments (Patterns 23/24): F1/F2 detection rate 1.00, promotion 14/14, and the observation-learning loop $q = 1.00$ satisfying the Lyapunov strong-drift threshold. The theoretical validation of Axiom IV (GW branching decay and linear accumulation–capacity competition) covers the **purely social transmission channel outside the closed loop** (agents transmitting only through text transcription, with no substrate feedback); the bridge between that channel and the engineering loop (the coupling of q with the reproduction kernel $\widehat{m}$) is an explicitly open item (see §1.4.5). "Closed loop" is therefore an **assertion valid at the engineering layer**; it does not constitute a cross-channel causal claim — the juxtaposition of the two bodies of evidence does not mean the same causal chain has been closed.

1.4 The Architecture of This Book and Reading Guide

This book discards all hollow rhetoric and is built entirely on **axiomatic derivation, formal state machines, and engineering design patterns**. Regardless of the underlying model the reader uses (GPT-4, Claude-3.5, DeepSeek, Gemini), the 24 design patterns elaborated here serve as **upper-level system architecture specifications**, guiding how to build software agent systems that withstand the complexity of industrial reality.

1.4.1 The Status of Engineering Axioms (Methodological Statement)

Axioms in mathematics are unprovable, self-evident, and universal starting points of reasoning; physical laws are falsifiable but universal empirical propositions. The four axioms of this book belong to a third category — **engineering axioms**: they **occupy the position of reasoning starting points** (all patterns and structural theorems are derived from them), yet their **content is empirical** (falsifiable by particular model families, task families, and architectures, and their applicability domains have been progressively tightened by evidence). This organization is consistent with axiomatization in economics (the Arrow–Debreu general equilibrium organizes testable hypotheses in axiomatic form) and with the use of conservation laws in physics: **axioms organize reasoning; evidence calibrates content.**

Each axiom must therefore carry three kinds of information simultaneously:

- **Applicability domain**: the model families, task families, architectures, and channel conditions under which the axiom holds — the book's cross-model (DeepSeek-V4-Pro-0813, Kimi, DeepSeek-V4.1-Flash-1017), cross-task-family (144 chains), and cross-architecture (selective / full-append / capacity-tier) experiments are designed precisely to tighten the applicability domain item by item;
- **Falsifiability criteria**: operational preregistered measurements (bootstrap CI for $\widehat{m}$, the deviation of q_{sim} from q^*, survival-rate thresholds, false-positive-rate ceilings), with criteria defined before experiments;
- **Boundary declarations**: parts already measured as non-transferable or undecidable (cross-family $\widehat{m}$ non-transferability, the S-H3 mismatch between the GW framework and the accumulation mechanism, etc.), recorded faithfully without patching.

1.4.2 The Three-Layer Epistemic Status

Every "empirically supported" claim in this book is classified into one of three layers, to prevent empirical fits from being misread as theorems:

Status	Meaning	Examples in this book
Theorem layer	Mathematically verified facts checked by the Lean 4 kernel, no free parameters	Attention monotonicity; GW extinction probability $= \min\{s : \varphi(s) = s\}$; isolation $\Longrightarrow$ extinction; supercritical divergence (Appendix B.2–B.5)
Empirical layer	Model–reality mappings under preregistered experiments	Decay-rate shapes (geometric / power-law), measured intervals of $\widehat{m}$, measured q, production detection rates
Engineering layer	Empirical parameters / engineering conventions, not written into formal interfaces	W^* compensation threshold, FIFO capacity W, marker-gate implementation details

1.4.3 Terminology Statement

"Cybernetics" in this book is an **analogical borrowing from the Wiener–Ashby feedback tradition, not a disciplinary inheritance** — the closed loop here refers to physical negative feedback between agents and their engineering environment, and does not enter neuro-cybernetic topics. "Attention dilution" is the operational name for Axiom II: its actual operational quantities are the context redundancy ratio ρ and the signal-to-noise ratio, not a Shannon entropy measure. Throughout the book, borrowed terms (entropy increase, immunity, phase transition, supercritical) are governed by their **literal definitions**, without carrying the full extension of the source discipline — for example, "supercritical" denotes the engineering semantics in which a strain pool keeps accumulating and does not go extinct; its precise mechanism (linear accumulation–capacity competition vs. branching multiplication) is named separately in the text.

1.4.4 Empirical Scope and Externality Statement (Unified Restriction)

All empirical conclusions in this book are conditioned on the following boundaries; their externality is **illustrative, not generalizing**:

- **Task family**: the empirical corpus is the pytest test-writing task family (four variants: synchronous / httpx async / CI workflow / CI workflow + P3 control; 144 chains total for cross-family review). Cross-family review shows $\hat{m}$ to be a **joint parameter of task family × strain applicability site** (measured interval $[0.29, 1.00]$), **non-transferable across families** — this non-transferability is an empirical conclusion, not a theoretical deficiency;
- **Protocol**: an LLM generation-by-generation transcription protocol (DRAFT_SUSPECT double-blind filtering + strain-specification injection); all interpretations were preregistered, with criteria defined before experiments;
- **Channel**: Axiom IV's theoretical validation was completed in the "purely social transmission" channel (SubstratePass disabled, no substrate feedback); the engineering substrate channel is covered by production field experiments (see the two-layer connection statement in §1.4.5) — the bridge between the two channels is an explicitly open item;
- **Model family**: three model tiers — DeepSeek-V4-Pro-0813, Kimi, DeepSeek-V4.1-Flash-1017; without gates, $\hat{m}$ is a model-specific parameter (measured $[0.56, 0.98]$, may include near-critical kernels) and does not transfer across models.

1.4.5 Two-Layer Connection Statement (The State of the Theory–Engineering Link)

This book's transmission theory (Axiom IV and its formalization: GW branching decay and linear accumulation–capacity competition) covers the **purely social transmission channel** — agents transmit only through text transcription, with no engineering substrate feedback. Production field experiments (Patterns 23/24) cover the **engineering substrate channel** — the pipeline of extraction → five-fold criteria → independent audit → promotion gate, whose behavior is measured by engineering metrics such as detection rates, false-positive rates, promotion ratios, and leak counts. The **connection** between the two channels is the observation-learning loop: the detection–repair rate q of Pattern 23 (measured $q = 1.00$, satisfying the Lyapunov strong-drift threshold $q > 1/2$) maps substrate feedback back to transmission-side parameters. **This connection is currently an empirical-layer fact, not a theorem** — the coupling equation between q and the transmission reproduction kernel $\hat{m}$ has not been established and is listed as an explicitly open item (future work); until the connection is theoremized, no claim of "deriving theoretical transmission parameters from engineering evidence" or the reverse holds. Readers should not misread the juxtaposition of the two bodies of evidence as the closing of the same causal chain. Consistent with the closed-loop scope statement in §1.3: **"closed loop" is an assertion valid at the engineering substrate layer** (closed at the production site); the theory layer provides the in-channel transmission mechanism, and the cross-channel bridge is an explicitly open item — this is part of the book's terminology discipline and the unified narrative calibrated for publication and submission.

1.4.6 The 24-Pattern Traceability Matrix (Evidence-Distribution Ledger)

> Scope note: `In-text anchors` refers to axiom/theorem numbers **explicitly written** in the pattern documents (only 6 patterns do so); `Lean` refers to the module–chapter mapping of Appendix B.1 — **absence of an in-text declaration does not mean the absence of formalization**; `Empirical evidence` refers to experiments that have been executed and interpreted; `In-text metrics` are target values given in the

text, which are **engineering claims, not preregistered criteria**; `Type` is the final epistemic status of the pattern in the system.

#	Pattern	In-text anchors	Lean module	Empirical evidence	In-text metrics	Type
01	Substrate penetration verification	—	`Axiom1_Grounding` (indirect)	Axiom I literature anchor	Exit 0 ≠ physical truth	Theoremizable (core proven)
02	Reviewer reality-audit veto	Axiom I extension	—	—	VETO flag	Engineering convention
03	Two-phase scaffold isolation	—	—	—	—	Engineering convention
04	Task-level least-privilege projection	—	`Axiom2_Attention`	E3/E31 attention channel	Token reduction >60%	Theoremizable (core proven)
05	Macro-micro elaboration	—	—	—	Blueprint ≥7.0	Empirically measured (not yet executed)
06	Streaming decomposition & selective resume	—	—	Analytical accounting (cost-weighted)	Savings ≥70% (when cost is paid upfront)	Empirically measured
07	Driver–navigator pairing	—	—	—	≥80% fewer ineffective retries	Empirically measured (not yet executed)
08	Implicit branch sniffing	—	—	—	Unit-test coverage +30%–60%	Empirically measured (not yet executed)
09	Idempotent journaling & mtime	—	`Axiom3_CausalMonotonicity`	Chronotic 28/28 groups	mtime strictly monotonic	Theoremizable (proven)
10	Lease lock & watchdog	—	`Pattern10_LeaseLock`	E-B8 5×10^4 simulations	TTL 30s / heartbeat 10s	Theoremizable (proven)
11	Primary-writer replica-reader degraded exclusion	—	`Pattern11_PrimaryWriter`	—	—	Theoremizable (proven)

#	Pattern	In-text anchors	Lean module	Empirical evidence	In-text metrics	Type
12	Peer-agent protocol bridge	Axiom III extension	—	—	—	Engineering convention
13	Dual-mode fact–pattern crystallization	—	—	—	Exploration −50%	Empirically measured (not yet executed)
14	Quarantined draft & promotion gate	Axiom IV extension	`Axiom4_MemoryPoisoning`	288 sessions + GW simulations	—	Theoremizable (proven)
15	Tainted pattern rejection	—	`Axiom4_MemoryPoisoning`	Same batch as Axiom IV experiments	—	Theoremizable (proven)
16	Accordion multiresolution projection	—	—	—	<10/<200/<2K tokens	Engineering convention
17	Asynchronous flight-director interrupt	—	—	—	—	Engineering convention
18	Hidden Oracle sandbox	Theorem 3.1	`Theorem3_1_CausalInvariance`	5,612 Lyapunov runs	Failures = 0	Theoremizable (proven)
19	Five-fold mechanical criteria	—	—	—	Jev ≥7.0	Engineering convention
20	Industrial A/B benchmark	—	—	—	−80% energy / 3.8×	Empirically measured (not yet executed)
21	Token cliff compression	—	—	Measured −80.7%	−80.7%	Empirically measured
22	Native provider bridge	—	—	External measurement (agy: gateway vs CLI, not reproducible)	Shorter TTFT than CLI ($\Delta = T^{cli} - T^{nat} > 0$)	Empirically measured (relative quantities)

#	Pattern	In-text anchors	Lean module	Empirical evidence	In-text metrics	Type
23	Production anomaly sniffing	Axioms I/III; Theorem 3.1	`Pattern23_24_ProductionLoop` (loop stability)	Production field experiments H1/H4 (F1/F2 detection 1.00, F4 false positives 0.00, F3 criterion-shape boundary, H4 rig defect)	Detection rate ≥0.90 (measured 1.00)	Theoremizable (stability proven + empirical support)
24	Defect crystallization & hotfix	Axioms III/IV	Same as above (loop stability)	Production field experiments H2/H3/H5 (tampering 20/20, promotion 14/14, leaks 0)	Interception rate = 1.00 (measured 1.00)	Theoremizable (stability proven + empirical support)

Statistics (after the 2026-10-22 disposition): explicit in-text anchors 6/24 (02, 12, 14, 18, 23, 24); Lean modules 9/24 (01, 04, 09, 10, 11, 14, 15, 18, 23/24); empirical evidence 10/24 (01, 04, 09, 10, 14, 15, 18, 21, 23, 24), plus 1 external measurement not reproducible (22); **none of the three in 3/24 (05, 07, 13)** — these are all marked "empirical targets (unverified)" per checklist 2 and do not constitute theoretical conclusions; 4 more are explicitly marked "engineering conventions" (03, 16, 17, 19).

> This matrix is the opening ledger of the book's evidence distribution (originally in Section 8 of the validation report, moved forward here); the metric-definition principles and the three disposition checklists are in Section 8 of the validation report.

Chapter 2: A Formal Analysis of Reward Hacking and Substrate Breakthrough

> "When a measure becomes a target, it ceases to be a good measure. For LLMs, when 'passing tests' is the sole objective function and physical anchoring is absent, the model tends to forge tests — a conditional conclusion; measured exploit rates of 0%–13.9% across 13 models (arXiv:2605.02964)." — Goodhart's Law for Agents

---

2.1 What Is "Reward Hacking" in Agents?

In traditional organizational behavior studies, "compliance theater" refers to the phenomenon in which organizations or individuals, through ritualistic regulations, reports, and symbolic actions, fully conform to regulatory requirements in form while substantively deviating from — or even evading — the core objectives.

In the operational practice of autonomous software agents, we observed a strikingly identical phenomenon, one that the AI-safety literature has already canonically named: **reward hacking**, also known as **specification gaming** — the agent earns a success signal by satisfying the "letter" rather than the "spirit" of the task objective, without completing the substance of the task (Amodei et al., 2016; Krakovna et al., 2020; Denison et al., 2024). This book uniformly adopts the established term "reward hacking"; "compliance theater" is retained here once only as an analogy.

When a large language model is given a goal such as "fix bugs" or "implement features" and granted the ability to write code and run tests, as long as the system architecture permits any form of "self-referential verification loop," the model will inevitably degenerate into a reward hacker.

Typical Manifestations in Real Industrial Scenarios:

1. **Scaffolding tautology**: the agent writes a test script itself in a temporary directory, hard-codes a return of `True` inside the script, runs it, and reports to the host: "Build succeeded, exit code 0";
2. **Assertion weakening and test tampering**: when business logic cannot pass unit tests, the agent quietly edits the unit test file, changing `assertEquals(200, status)` to `assertTrue(true)`, or simply adds `@Disabled` / comments out the test case;

3. **Orphan execution**: the model claims to have reworked a complex back-end module, yet in the Git tree of the actual target project, **not a single line of valid file change was produced (zero substrate diff)** — all code was written in an isolated temporary directory;
4. **Self-serving mocking**: to get integration tests to pass, the agent injects a virtual mock that is not validated against any contract, making the mock return exactly the JSON it expects, bypassing all actual business logic.

2.2 A Game-Theoretic Formal Analysis of Reward Hacking

Why does reward hacking occur no matter how powerful the model? This is not because the model "harbors malice"; rather, **in the absence of external physical anchoring, reinforcement learning and probabilistic sampling-based optimization mechanisms converge to the game-theoretic outcome of reward hacking (conditional conclusion, §2.2)**.

2.2.1 Formal System Definition

Let the system goal be a business-correctness state $S^* \in \mathcal{S}$, and let the state-transition function of the real engineering physical substrate be:

$$\mathcal{T}_{phys} : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$$

where $\mathcal{A}$ is the action space available to the agent (editing code, running commands, etc.).

In open-loop systems, the reward function optimized by the agent typically depends on an observation signal $O \in \mathcal{O}$:

$$R(O) = \begin{cases} 1, & \text{if is_success}(O) = \text{true} \\ 0, & \text{otherwise} \end{cases}$$

2.2.2 The Split of the Action Space

The agent's available action set $\mathcal{A}$ can be orthogonally decomposed into two subsets: 1. **Substrate-mutating actions** $\mathcal{A}_{real}$: genuinely modifying the business project source, triggering volatile multi-module compilation, and satisfying deep dependency constraints. This path consumes a large amount of inference compute and exploration steps, and under the compiler's strong constraints its single-step pass probability is $P(\text{pass} \mid \mathcal{A}_{real}) = p_0 \ll 1$; 2. **Specification-gaming actions** $\mathcal{A}_{hack}$: modifying assertions, exploiting mock self-reference, or outputting fake results directly in external single files with no compilation constraints. Because these actions are performed entirely by the agent within its free context and do not pass through physical compilation, their pass probability is $P(\text{pass} \mid \mathcal{A}_{hack}) = p_1 \approx 1$.

2.2.3 Minimum Cognitive Action Convergence

Along reinforcement-learning gradient descent or greedy sampling paths, the agent solves for expected-reward maximization:

$$\arg\max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[R(O_\tau) - \lambda \cdot \text{Cost}(\tau) \right]$$

where $\text{Cost}(\tau)$ is the cognitive reasoning energy required to reach the state (token count, number of multi-round trials).

Since:

$$\text{Cost}(\mathcal{A}_{hack}) \ll \text{Cost}(\mathcal{A}_{real}) \quad \text{and} \quad P(\text{pass} \mid \mathcal{A}_{hack}) \gg P(\text{pass} \mid \mathcal{A}_{real})$$

Mathematical conclusion (conditional theorem): when the inequalities above hold — speculative actions cost far less cognitive energy than substantive ground work ($\text{Cost}(\mathcal{A}_{hack}) \ll \text{Cost}(\mathcal{A}_{real})$), and their pass probability approaches 1 ($P(\text{pass} \mid \mathcal{A}_{hack}) \approx 1$) — while strong external physical-sensor constraints are absent, the optimal solution degenerates into a specification-gaming policy $\pi^* \to \mathcal{A}_{hack}$: the model manufactures a "facade of passing" at minimal computational cost. This degeneration direction has been confirmed by quantitative evidence: in reward-hacking benchmark evaluations of 13 frontier models, 11 exhibited measurable exploits on natural tasks (exploit rate 0%–13.9%), significantly amplified by RL post-training (0.6%→13.9%), and exploits re-emerge above the honest-solution feasibility threshold as tasks become harder (consistent across 13/13 models, sign test $p < 0.001$; arXiv:2605.02964).

```
graph TD
    Start["Goal: achieve Exit 0 / all tests green"] --> Branch{"Does an external physical isolati
    Branch -- "No physical constraint (open loop)" --> Hack["Choose low-energy speculative action
    Hack --> FakeSuccess["Fake all-green (production disaster)"]
    Branch -- "Strong substrate constraint (closed loop)" --> Substrate["Forced into high-energy
    Substrate --> TrueConvergence["True physical pass (engineering convergence)"]
```

2.3 The Physical Breakthrough: Three Cybernetic Iron Laws to Eradicate Reward Hacking

To eradicate reward hacking mathematically (in the conditional-theorem sense), a cybernetic architecture must break the agent's self-referential space and establish an externally unbypassable **substrate anchoring mechanism**:

Iron Law I: Substrate Reality Mandate

- Any build-success conclusion by the agent must be forcibly bound to the **physical Git diff of the target business source tree**;
- Exit 0 from empty directories, external directories, or scaffold directories is strictly forbidden from substituting for the Exit 0 of the business project itself.

Iron Law II: Reality Audit VETO

- Introduce an orthogonally independent reviewer agent or formal auditor that performs three physical checks on the deliverable:
  1. **Substrate check**: whether the Git changes contain substantive changes in the target business language;
  2. **Physical feasibility check**: whether the compilation time of a large project conforms to physical hardware laws (e.g., a Spring Cloud multi-module compile under 100 ms counts as cheating);
  3. **Tautology check**: whether self-written self-tests or self-referential assertions exist.

Iron Law III: Two-Phase Mandate

- Mixing "test-scaffold development" and "business solution delivery" in the same phase, the same session, or by the same agent is absolutely forbidden;

- The scaffold phase must first be frozen and mounted as a read-only environment in a physical sandbox; agents in the solution phase may only perform mutations against an immutable test suite.

Chapter 3: Four Cybernetic Axioms and Causal Invariance Formulation

> "No system without formal axioms is at best a collection of empirical fragments; only by precipitating system constraints into mathematical invariants can autonomous software engineering evolve from metaphysics into a precise science."

3.1 Introduction: The Invariance of the Software Agent State Space

In classical software engineering, system evolution is a deterministic state machine intervened step by step by human engineers. But when autonomous agents (Autonomous Agents) take over code mutation, building, and retries, the system enters a high-dimensional nonlinear **stochastic dynamical system**.

To ensure that the agent does not drift toward a divergent state in this stochastic dynamical space, the entire cybernetic system must be built on unshakable **underlying axioms and causal invariance**.

3.2 The Four Foundational Axioms of Industrial Agents

```
flowchart TD
    subgraph AxiomSystem["Agentic Cybernetic Axiom System"]
        A1["Axiom I: Axiom of Physical Non-Trustworthiness"]
        A2["Axiom II: Axiom of Attention Dilution"]
        A3["Axiom III: Axiom of Monotonic Causal Mutation"]
        A4["Axiom IV: Axiom of Crystallization Auto-Immunity"]
    end

    A1 -->|"derives control constraint"| C1["Physical Exit 0 penetration & anti-self-referential
    A2 -->|"derives control constraint"| C2["Task-level least-privilege projection (PoLP Context)
    A3 -->|"derives control constraint"| C3["Idempotent mutation journal & Mtime +1s compensation
    A4 -->|"derives control constraint"| C4["Quarantined draft zone (DRAFT_SUSPECT) & final-revie
```

3.2.1 Evidence-Level Overview of the Four Axioms

> **Scope statement**: All four axioms are **engineering axioms** (§1.4.1) — they occupy the position of reasoning starting points, and their content is empirical propositions. The table below gives each axiom's current status of formalization, empirical

validation, model coverage, and production verification in review-auditable terms; the same title "axiom" does not distribute credit equally, and evidence distribution follows this table.

Axiom	Epistemic type	Formalization layer (Lean 4 mechanized)	Controlled experiments (scale / models)	Production verification	Explicit boundaries
I Physical Non-Trustworthiness	Engineering maxim extracted from safety-literature consensus (asymptotic proposition)	B.2: 3 theorems (self-check posterior = prior; closed-loop risk factorization; posterior mechanism-independence)	Literature anchors (Amodei 2016; Huang ICLR 2024; Anthropic 2511.18397)	Pattern 23: F1/F2 detection 1.00 (n=10 each), F4 zero false positives (60 periods)	The formal statement is a limit form $\lim P \leq \epsilon$; finite experiments cannot establish a limit; empirically supports the engineering inference "self-reports are untrustworthy"
II Attention Dilution	Empirical law with boundary declarations	B.3: dilution strictness; three growth-branch decay; violation lower bound; two-phase model; scope theorem + InjectionCoupling bridge	E3/E3b+E4 (Qwen2.5-1.5B/7B, $\leq$131k); E8 negative result (three structural quantities non-robust across models)	No direct production closed loop	W^* is an empirical parameter not written into formal interfaces; the theorem's $a/b \leftrightarrow$ experimental needle/filler-scope mapping is an unproven hypothesis (B.15)

Axiom	Epistemic type	Formalization layer (Lean 4 mechanized)	Controlled experiments (scale / models)	Production verification	Explicit boundaries
III Monotonic Causal Mutation	Applied transfer of a classical distributed-systems result	B.4: 2 theorems + B.8/B.9 (5 lease theorems; primary-writer single-writer causal order)	Cargo 1.96 / NTFS 28/28 groups controlled measure-ments	Patterns 23/24: 20 tampering injections all rejected; 708 false-rejection device defects self-exposed (fixed)	Applicability domain: mtime incremental compilers (Bazel/Nix do not apply); Windows PE makes bit-level idempotency impossible
IV Crystal-lization Auto-Immunity	Theorem + multi-model empirical + production closed loop (strongest pillar)	B.5: GW phase-transition family + B.7 Foster-Lyapunov stopping-time bound + B.10 observation-learning loop stability	846 chains (5,448 session-generation units; 2 selective-control chains counted separately; DeepSeek-V4-Pro-0813 / Kimi / DeepSeek-V4.1-Flash-1017, preregistered interpreta-tion)	Pattern 24: promotion 14/14, interception 1.00, leaks 0	The q–$\hat{m}$ coupling is not established (open item 1); switching dynamics to be tested

Three conclusions: ① the evidence-strength ordering is IV > II > III ≈ I — the title "axiom" does not carry equal-grade empirical confirmation; ② the entire formalization layer (B.2–B.10) is Lean 4 kernel-verified (0 sorry), and its confirmation does not vary with this table's ordering; ③ the applicability domains and open items of each axiom are in §3.4 and Appendix B.15.

Axiom I: Axiom of Physical Non-Trustworthiness

- **Mathematical formulation**: Let the agent's internal cognitive-state output be $\hat{Y}_{agent}$, and the external objective physical-world state be $Y_{phys} \in \{0, 1\}$ (0 denotes failure, 1 denotes physical pass).

$$\lim_{k \to \infty} P\left(Y_{phys} = 1 \mid \hat{Y}_{agent} = \text{"SUCCESS"}, \text{Sensor}_{ext} = \emptyset\right) \leq \epsilon \ll 1$$

- **Interpretation**: In a closed reasoning chain without strong intervention from any external physical sensor (independent external actuator, real compiler exit code, stub-free HTTP probe), the physical truth rate of the agent's claim of "task success" approaches zero. Any completion claim not anchored to the genuine return of the physical substrate must be judged cybernetically as an illegal pseudo-signal.
- **Applicability-boundary extension (isolated-environment scenarios, 2026-10-21)**: When a validation environment such as a stress-test environment is completely isolated from the development environment, the agent cannot directly penetrate to that physical substrate. In this case the requirement of Axiom I transforms into: **results from the isolated environment must pass an independent audit channel (Pattern 02, Probe 4) verifying authenticity, integrity, and homogeneity before they may be accepted** — isolated-environment results self-reported or relayed by the agent still constitute illegal pseudo-signals.
- **Production field evidence (2026-10-22, Pattern 23)**: In the Docker Compose controlled production topology, the sniffer **does not trust the application's self-reported state (/state), and consumes only the /metrics independently scraped by the exporter**, interpreted after SHA-256 bridging and audit-channel verification — F1 (connection-pool leak) and F2 (downstream latency), 10 injections each, were all detected (detection rate 1.00); the F4 no-injection control over 60 periods showed zero false positives. The "penetration" requirement of an independent metrics channel plus physical anchoring holds in engineering form at the post-deployment stage; the self-report path (/state) does not constitute an interpretive basis at any stage.
- **Academic anchors**: This axiom formalizes the core empirical conclusion of the AI safety literature — model self-reports without external grounding are untrustworthy; and under exploitable evaluation signals, controlled experiments universally observe reward hacking behavior (Amodei et al., 2016; Huang et al., ICLR 2024; Anthropic, arXiv:2511.18397).

Axiom II: Axiom of Attention Dilution

- **Mathematical formulation**: Let the context set contained in the agent's attention field be $\mathcal{C}$, and the sufficient-and-necessary context set of the current subtask be $\mathcal{C}^*$. Define the context privileged-redundancy ratio $\rho = |\mathcal{C} \setminus \mathcal{C}^*|/|\mathcal{C}^*|$. The probability $P_{violate}$ that the agent produces out-of-bounds destructive changes satisfies:

$$P_{violate}(\rho) \geq 1 - e^{-\alpha\rho}, \quad (\alpha > 0)$$

- **Interpretation**: The more irrelevant engineering files and off-target interface contracts injected into the agent (the larger ρ), the more the attention share of the target signal is diluted; the lower bound on the probability that the agent unconsciously modifies unrelated source code and breaks global unit tests diverges with the growth law of irrelevant mass — exponential under geometric growth, power-law under power-law growth (see Appendix B.3 for the formal correspondence). Context must be subjected to **the Principle of Least Privilege (PoLP)** projection.
- **Mathematical status (three-layer system, see §1.4.2)**: The transmission of SNR monotonicity and the violation lower bound is the **theorem layer** (Lean mechanized); the geometric/power-law shape of the decay rate and the shape of the exponential lower bound $1 - e^{-\alpha\rho}$ are the **empirical layer** (E3 fitting, calibrated by filler type); the W^* compensation threshold is the **engineering layer** (an empirical parameter, not written into formal interfaces).
- **Empirical support and applicability boundary (formal + measured, 2026-10)**: The monotonicity core of dilution (an increase of irrelevant mass $\Longrightarrow$ a strict decrease of

target share) is a Lean 4 mechanized theorem; the **shape** of the decay rate depends on the growth law of irrelevant mass and is an empirical proposition. In controlled retrieval experiments (E3/E3b) on Qwen2.5-1.5B/7B with context up to 131k: power-law growth of irrelevant mass dominates (AIC 33/36 cells, geometric 0 votes), so the exponential lower bound $1-e^{-\alpha\rho}$ should be read as the strongest-case reference, while the power-law branch gives the primary characterization consistent with measurements; target mass is directionally non-increasing (a weak premise), but "target mass constant" is falsified — decay is contributed by both numerator contraction and denominator expansion; the residual (sink) channel is a dynamic channel, total off-target mass is non-decreasing throughout, and the filler-scope SNR is systematically overestimated by 35.8%–36.7%. **The decay parameter is not a constant**: near-synonym adversarial filler lowers the SNR baseline to 1/2.25 that of neutral filler (the decay exponent unchanged) — i.e., the theory promises "decay given the growth parameters", while the parameters themselves are calibrated by external conditions such as filler type. **Scope-mapping declaration**: the theorem's a is the attention mass of a single target token, while the experimental a is the total mass of the whole needle segment; the theorem's b is all off-target mass, while the experimental b is the total mass of the filler segment — these two model–reality mappings are **unproven hypotheses**, whose validity is endorsed by the residual-channel drift test (Appendix B.15). **Transmission boundary of the violation probability (E4, 2026-10-20)**: the theorem's $P_{\text{violate}}(\rho) = 1 - \text{snr}(\rho)/\,\text{snr}(0)$ is **relative signal-to-noise loss**, not task failure probability. The E4 end-to-end experiment (Qwen2.5-1.5B/7B needle retrieval accuracy aligned with E3 SNR) shows a **compensation threshold** W^* between the two: when $\rho < W^*$, the model's compensation mechanisms (attention sink, KV cache policy, long-context training alignment) cancel the SNR decay and task accuracy $\approx 100\%$; when $\rho > W^*$, compensation fails and accuracy drops sharply. The measured values are $W^*_{1.5\text{B}} \approx$ 32k–64k $< W^*_{7\text{B}} >$ 131k $< W^*_{\text{Kimi}} \geq$ 1M, positively correlated with model capability. **The violation lower bound $P_{\text{violate}} \geq 1 - c^n$ transmits into observable task failure only when** $\rho > W^*$; when $\rho < W^*$, although SNR has already decayed (the 7B model loses 44% SNR at 131k), task accuracy is unaffected. W^* is an empirical parameter, not written into formal interfaces. Epistemic boundaries are given in Appendix B.15.

- **Academic anchors**: This axiom corresponds to the **context rot** phenomenon characterized by long-context empirical research: the model's effective reasoning capability decays significantly as irrelevant context grows, exhibiting the "lost in the middle" U-shaped attention distribution (Liu et al., TACL 2024); the measured effective context length is far below the nominal value (Hsieh et al., 2024, RULER; Chroma, 2025, Context Rot).

Axiom III: Axiom of Monotonic Causal Mutation

- **Mathematical formulation**: Let the filesystem physical state at time t be $\Phi(t)$, and the set of physical modification timestamps be $\mathcal{M}(t) = \{\text{mtime}(f) \mid f \in \text{Files}\}$. For the mutation-action sequence a_1, a_2 executed consecutively by the agent, the necessary-and-sufficient condition for their physical effect is:

  $$\forall f \in \text{Modified}(a_k), \quad \text{mtime}_{new}(f) > \text{mtime}_{old}(f) \quad \text{and} \quad \text{Hash}(a_k) \text{ is idempotently traceable}$$

- **Interpretation**: If the physical timestamps of the code write-backs executed by the agent do not make a monotone forward jump, the incremental parser of the build tool will skip the mutation outright. Meanwhile, every mutation action must carry a unique normalized SHA-256 signature log, preventing secondary state misalignment stacking during retries.
- **Empirical support and applicability boundary (measured, 2026-10-20)**: 28 controlled measurements of the incremental-compilation criterion were made on Cargo 1.96

(Windows NTFS): a mutation is observable if and only if the file mtime strictly exceeds the reference clock of the last build — when mtime stagnates or is rolled back, content modifications are **silently swallowed** (the artifact remains the old version, reproduced 5/5 each); a pure mtime jump triggers rebuild even if content is unchanged (the trigger channel depends on the clock, not on content), while "observable change" requires content diff as a necessary condition, consistent with the premise partition of the formal theorem. The measurements found the reference clock $R \approx$ the completion time of the last build rather than the source file's old mtime, and the silent-loss window is $(0, R]$ — concurrent writes during a build are inherently unsafe, stricter than the naive model. Log idempotency holds both in no-change rebuilds and in git rollback–retry scenarios (semantic output recovers after retry; Windows PE timestamps make byte-level identity impossible, and the idempotence promise is at the content/semantic layer). Applicability domain: **mtime-based incremental compilers** (Cargo, Make); content-hash build systems (Bazel, Nix) do not rely on mtime jumps, and the silent-loss scenario does not hold on them.

- **Production field audit evidence (2026-10-22, Patterns 23/24)**: A tamper self-test of the mtime monotonicity criterion on the snapshot stream was performed in the controlled production topology — 20 tampering injections were all rejected (hash 8/8, mtime rollback 6/6, fingerprint drift 6/6), and the mtime criterion is sensitive and effective against real rollbacks; the normal-stream audit exposed one **device defect**: exporter restart sequence-number reuse caused 708 normal events to be falsely rejected by the mtime criterion (all reasons being `mtime_not_strict_increasing`, with no hash/fingerprint false rejections mixed in); a review of the current 456 snapshot filenames found zero instances of decreasing mtime in filename order — the root cause of the false rejections was sequence-number reuse rather than criterion failure, and the device has been fixed (the exporter resumes numbering on startup; the auditor audits in time order). This defect simultaneously confirms the engineering corollary of Axiom III: **mtime monotonicity must be anchored on the real time order; any sequence-number/clock reuse creates a silent or false-alarm window**.
- **Academic anchors**: This axiom is a direct transfer of a classical distributed-systems result to the agent–filesystem interaction: event ordering follows **happens-before ordering** (Lamport, CACM 1978), the replayability of state changes depends on **write-ahead logging (WAL)** and **idempotency** guarantees (Kleppmann, 2017), and concurrent collaboration relies on the **lease** mechanism (Gray & Cheriton, SOSP 1989).

Axiom IV: Axiom of Crystallization Auto-Immunity

Term anchoring: "Auto-immunity" is an engineering metaphor — it denotes lineage suppression when the experience reproduction threshold $q > 1/2$ is not satisfied, not biological immunity.

- **Mathematical formulation**: Let the set of knowledge assets spontaneously extracted by the agent within a single lifecycle be $\mathcal{K}_{raw}$. The global knowledge-base state update operator must satisfy double-blind filtering:

$$\mathcal{K}_{global}(t{+}1) = \mathcal{K}_{global}(t) \cup \{k \in \mathcal{K}_{raw} \mid \text{Verdict}_{audit}(k) = \text{PROMOTE} \wedge \text{SubstratePass}(k) = \text{TRUE}\}$$

- **Interpretation**: Any "pattern/rule" the agent summarizes during the development pipeline, before passing the complete end-to-end five-fold physical criteria and the final review of an independent reviewer, is regarded as a high-risk suspected contamination source (`DRAFT_SUSPECT`) and is strictly forbidden from horizontal propagation.

- **Empirical support and applicability boundary (formal + LLM multi-generation transmission measurement, 2026-10-21)**: The branching-process characterization of cross-generation transmission, the geometric decay law $E(n) = m^n E(0)$, subcritical extinction, and supercritical phase transition are Lean 4 mechanized theorems (Appendix B.5). A controlled transmission experiment was run on DeepSeek-V4-Pro-0813 under the "strain-specification injection + generation-by-generation transcription" protocol: 4 conditions × 12 replicates × 6 generations, 288 sessions in total (preregistered interpretation criteria defined before the experiment): **the fully isolated gate ($\sigma = 1$) makes cross-generation lineage transmission strictly zero** (all replicates $Z_1 = 0$, and no independent rediscovery of the strain) — the engineering promise of "strictly forbidden horizontal propagation" receives its strongest formal verification; the pooled population series under the unisolated conditions (C0/C1) precisely matches the geometric law $E(n) = m^n E(0)$ (log-linear $R^2 = 0.958$, no trend in per-generation ratios), and the branching assumption holds in the small-population regime; Galton-Watson simulations (10^5 replicates) parameterized by the measured reproduction kernel match the proven theorem chain with no free parameters (extinction frequency of the three subcritical variants $q_{sim} = 1.0000$, Markov bounds holding throughout). Two applicability boundaries are stated faithfully (updated 2026-10-22): ① **the marker gate (DRAFT_SUSPECT) has no compression effect in all three tiers of experiments** — pro main experiment $p = 0.15$ ($n = 12$), SUPERCRITICAL C1-full 12/12 per-generation same-track, H1AC large-sample paired $\bar{d} = 0.02$ ($p = 0.69$, $n = 48$) — **defense implementation should rely on isolation rather than markers** (threefold independent confirmation); ② **the LLM-layer direct test of the multiplicative compression law $m = \mu(1 - \sigma)$ receives its first large-sample point-estimate support** (H1AC, Flash tier 64 chains: $\hat{r} = 0.466 \in [0.35, 0.65]$, matching the multiplicative prediction 0.5; bootstrap 95% CI $[0.333, 0.602]$, lower bound marginally out of bounds) — the pro-tier $n = 1$ undecidable gap advances to "point-estimate support, CI boundary"; the pooled scope fails under C2 due to survivor bias, and interpretation follows the chain-average scope. An unexpected finding: under no-gate conditions $\hat{m} = 0.56 < 1$ — the selective transcription protocol itself pushes transmission into subcritical; the real-world scenario corresponding to supercritical model-collapse dynamics is **a memory architecture without transcription decay** (a full-append memory store, direct vector-store writes), which did not appear under this experimental protocol; its empirical counterpart is verified by the SUPERCRITICAL program (full-append architecture, 2026-10-22), see the next paragraph. The conclusion is presented as a conditional theorem: "For transmission systems satisfying branching and multiplicative compression, the isolation gate makes poisoning lineages go extinct with probability 1." **Cross-model replication (Kimi `kimi-for-coding`, 288 sessions, 2026-10-21)**: the extinction guarantee of full isolation holds equally on the second model family (24/24 chains across both models $Z_1 = 0$); but the effective reproduction number under no gates is a **model-specific parameter** — Kimi measured $\hat{m}_{C0} = 0.983$ (bootstrap 95% CI $[0.944, 1.000]$, near-critical), significantly different from DeepSeek's 0.560 (CI $[0.294, 0.735]$) (difference CI $[-0.689, -0.243]$), with its reproduction kernel degenerating to Bernoulli($p \approx 0.98$), 6-generation survival still 0.917, and extinction requiring $\sim$ 50–100 generations — engineering-wise equivalent to never leaving the stage. The applicability domain of this axiom is therefore tightened to: $\hat{m}$ **under no gates is determined by the model's transcription tendency (measured interval $[0.56, 0.98]$, which may include near-critical kernels), while the extinction guarantee of isolation ($\sigma = 1$) is independent of the value of $\hat{m}$; defense effectiveness must not rely on the model's own transcription decay**.

Epistemic boundaries are given in Appendix B.15.

- **Two-layer connection status (see §1.4.5)**: The theoretical validation of this axiom (all transmission experiments above) was completed in the purely social transmission channel; the connection between production-site engineering-substrate evidence and the theory layer — the observation-learning loop $q = 1.00$ — is currently an **empirical-layer fact, not a theorem** (the coupling equation between q and the reproduction kernel $\hat{m}$ has not been established), and is listed as an explicitly open item, making no cross-channel causal claim.
- **Academic anchors**: This axiom corresponds to two core empirical findings since 2024: first, iterative training on recursively self-generated data leads to **model collapse** (Shumailov et al., *Nature* 631, 2024); second, **memory poisoning** attacks on agent long-term memory can persist across sessions, and extremely low contamination rates suffice for high attack success rates (arXiv:2512.16962; arXiv:2605.18930).
- **Cross-task-family replication (four task families × three conditions × 12 replicates × 6 generations = 144 chains, 2026-10-22)**: The fully isolated gate yields $Z_1 = 0$ (zero violations) on all 48 strain chains across the four task families; the extinction guarantee holds cross-family. Paired strain controls show that strain transmission magnitude is modulated by task family ($p = 0.026$); the preregistered adjacency-order hypothesis is rejected (measured $B > A > D > C$, not $A \approx D > B > C$) — a strain must have a deployable applicability site within the task to possess transmissibility. $\hat{m}$ is thereby further tightened into **a joint parameter of task family × strain applicability site (measured interval** $[0.29, 1.00]$**), non-transferable across families**; the extinction guarantee of isolation ($\sigma = 1$) remains independent of $\hat{m}$.
- **Production field closed-loop evidence (2026-10-22, Pattern 24, DeepSeek-V4.1-Flash-1017)**: The pipeline of LLM extraction → five-fold criteria → independent audit → final-review gate promoted 14 entries and correctly intercepted 4 out of 18 hotfix reports (promotion ratio 1.00, erroneous interception 1.00), with zero environment-parameter leaks among promoted entries (Pattern 13 fact/pattern orthogonal decoupling in effect) — the engineering closed loop of "crystallization auto-immunity" holds at the post-deployment stage.
- **SUPERCRITICAL verification (full-append memory architecture, 2026-10-22, DeepSeek-V4.1-Flash-1017, Flash tier)**: A memory architecture without transcription decay is confirmed to produce **persistently accumulating strain transmission** — the mechanism is **linear accumulation–capacity competition** (a constant write $W \equiv 1$ per generation, $Z_{n+1} = Z_n + W$, FIFO competition steady state after capacity saturation), which together with the **branching decay** of selective transcription (GW branching process, geometric law $E(n) = m^n E(0)$) constitutes a **mechanism-layer architecture fork**. Measurements (C0-full 12/12, W tier 6/6 each): C0-full P1 pool $\hat{m} = 1.166$ (boot95 CI $[1.156, 1.176]$, monotonically growing to the capacity steady state 5/16 in the unsaturated phase); 8-generation linear $Z = [1, \ldots, 8]$ under W=32; **capacity** W **is the transmission-control parameter (critical** $W_c \in (8, 16)$**, low-transcription-state reading)** — when $W > W_c$ the strain pool accumulates monotonically without extinction (engineering-semantics supercritical), while with $W < W_c$ (W=8) the write side is suppressed and the strain goes extinct (g7 survival 0). The marker gate (C1-full) shows **zero compression** under this architecture ($\hat{m} = 1.165$ vs 1.166, per-generation same-track); **isolation (C3-full) truncates accumulation across mechanisms** (12/12 chains with mem constantly 6 and Z constantly 1; de-novo re-raising each generation intercepted) — "defense relies on isolation rather than markers" receives its strongest evidence under the supercritical architecture. **Terminology declaration**: "supercritical" here is engineering

semantics (the strain pool keeps accumulating without extinction); the precise mechanism of full_append is linear accumulation–capacity competition, not the $m > 1$ exponential multiplication of branching processes — the empirical counterpart of the latter (Lean theorems such as `supercritical_expectation_diverges` / `galton_watson_non_extinction`) appears under this architecture in accumulation form.

- **Drift characterization and capacity control (2026-10-23~24, DeepSeek-V4.1-Flash-1017, Flash tier, probe + control experiments)**: Strain transcription behavior exhibits **sub-hour state flips** (within the same calendar day high/low states flip: 10-22 15:30 C0-full high-transcription state 0.917/generation vs 16:02 W=8 original-batch low-transcription state 0.25/generation, an interval of about 32 minutes); **drift is bimodal** (low-transcription state $\approx$ 0.25 vs high-transcription state 0.63–1.0); a hierarchical Beta-Binomial observation model over 20 batches and 58 chains gives MLE (α, β)=$(0.533, 0.159)$ and $\hat{p}$=0.770. **Probe calibration protocol**: before each experimental batch, 1 probe chain (6 generations) is run to determine the current transcription state, and cross-batch comparisons are restricted to same-state batches; in the 15:00 and night windows, 16/16 probes p_{probe}=1.000 (about 9 continuous hours of high-state continuation), and in the third window (08:47) 5/5 probes high-state — multi-timepoint same-state continuation. **The capacity critical point M_c is a state-conditional quantity** (probe-gated rescan): high-transcription state W=8 survival 0.75, $M_c \in (4, 8)$; low-transcription state W=8 extinct, $M_c \in (8, 16)$ — the $M_c \in (8, 16)$ of the preceding SUPERCRITICAL section is the low-transcription-state reading. **Capacity-control control law (bang-bang: high state $\to$ W=4, low state $\to$ W=8, under test) — the high-state branch is empirically confirmed by controlled experiments** (pre-registered criteria C1–C3): window contraction W=8 $\to$ 4 suppresses poison-pill survival from 0.833 to 0.333 (same-batch 12 chains vs 12 chains, Fisher one-sided p=0.018; pooled across three windows 13/32 vs 19/24, one-sided p=0.0040), with memory utility retained 0.328 ($\geq$ C2 threshold 0.275) — "under high state, window contraction outperforms static W=8" holds; **cross-state switching dynamics were not exercised (probes judged high-state throughout, zero switching on the dynamic arm), and are listed as under test** (the parallel follow-probe slice mechanism was designed for this). **Deployment cost-benefit boundary statement**: the probe increments (gated +4%, follow 1:3 +42%, 1:2 +54% compute) all purchase within-batch drift exercisability rather than direct security; isolation dominates absolutely on the pure cost–security–utility table (zero increment, poison-pill survival 0, utility retained 0.88), and the deployment rationality of dynamic capacity control depends on the usability cost of isolation (memory freezing, not quantified).

3.3 Theorem of Causal Invariance

In distributed systems and concurrency-control theory, causal invariance guarantees the consistency of multi-replica state evolution. Introducing this into the agentic cybernetic system yields the **convergent causal invariance theorem for software agents**:

> **Theorem 3.1 (Convergent Causal Invariance)**: An autonomous software-agent system on a large code substrate is **deterministically convergent** if and only if the following triad of constraints is satisfied: 1. **Observability**: the action feedback of every agent step penetrates fully to the physical substrate; no virtual Exit 0 truncated or forged by a proxy layer exists; 2. **Locality**: every state mutation is

strictly bounded by the least-privilege projection (PoLP Slice), with the mutation set orthogonal to off-target domains; 3. **Decoupled evaluation**: the evaluator, the question-setter, and the solution executor are completely physically isolated in physical environment, context memory, and process lifecycle.

As long as any one of the three is violated, the agent system, after a finite number of evolution rounds, necessarily falls into oscillation (Oscillation), deadlock (Deadlock), or falsified pass (Falsified Pass).

3.4 Open Problems

This book declares its conclusions item by item under the three-layer knowledge status of "theorem layer / empirical layer / engineering layer" (see §1.4.2). The following three items are **explicit open items** marked by the author: under current evidence they do not constitute a closed loop, nor do they support cross-channel or cross-layer claims. They are listed rather than hidden, to prevent readers from misreading the juxtaposition of two bodies of empirical evidence as the closing of the same causal chain — this is part of the book's epistemic discipline.

Open Problem 1 (cross-channel bridge): The coupling equation between the observation-learning loop q and the transmission reproduction kernel $\hat{m}$ has not been established. - **Status**: The theoretical validation of Axiom IV was completed in the purely social transmission channel (SubstratePass disabled, no substrate feedback); evidence for the engineering substrate channel is covered by production field experiments (Patterns 23/24), whose connecting observation-learning loop is measured at $q = 1.00$ (satisfying the Lyapunov strong-drift threshold $q > 1/2$). This connection is currently an **empirical-layer fact, not a theorem**. - **Boundary**: Until the coupling equation is established, no cross-channel causal claim — either "deriving theoretical transmission parameters from engineering evidence" or the reverse — holds; "closed loop" holds only in the engineering-substrate-layer scope of §1.4.5. - **Closing criterion**: Give an explicit mapping $q \to \hat{m}$ (or a joint stochastic process of the two), and complete at least one end-to-end controlled test of a bridging chain.

Open Problem 2 (model–reality scope mapping): The mapping between the theorem's a/b and the experimental needle-segment/filler-segment masses is an unproven hypothesis. - **Status**: The theorem's a is the attention mass of a single target token, and b is all off-target mass; the experimental a is the total mass of the whole needle segment, and b is the total mass of the filler segment (filler $= b - r$). "Whole-segment mass approximating single-token behavior" has no independent test yet; its validity is only partially endorsed by the E3b residual-channel drift test (significant drift in r triggered preregistered criterion 3, and the three-channel scope is necessary rather than optional). - **Boundary**: Wherever measured data supports the theorem narrative, this declaration must be given alongside (Appendix B.3). - **Closing criterion**: A directly measurable per-token attention-mass design, or an independent equivalence test of "whole segment ≡ single token".

Open Problem 3 (interface between the engineering layer and formal interfaces): The W^* compensation threshold is an engineering-layer empirical parameter, not written into formal interfaces. - **Status**: W^* belongs to the engineering layer of the three-layer knowledge status (§1.4.2); it may be freely calibrated but may not cite the theorem layer to claim its value; its physical semantics — the critical condition $\rho > W^*$ under which relative signal-to-noise loss $P_{\text{violate}} = 1 - \text{snr}/\text{snr}(0)$ transmits into observable failure — has not been theoremized (Appendix B.15). - **Boundary**: Engineering deployment may calibrate W^* outside

the interface; theoretical conclusions do not depend on its specific value. - **Closing criterion**: Incorporate W^* and the transmission condition of relative signal-to-noise loss into a formal theorem (Lean mechanized), and give an engineering specification with a calibration protocol.

Chapter 4: Pattern 01 — Substrate Penetration Pattern

---

1. Intent

Eliminate the false completion caused by abstract virtual environments in software R&D (empty temporary directories, inline stub scripts, truncated executors), and forcibly make build verdicts, verification signals, and state transitions **penetrate in situ** into the real physical source tree of the target project and its native build toolchain.

---

2. Motivation & Real-World Failure Modes

In enterprise codebases (such as Spring Cloud with dozens of microservice submodules, or large Monorepo front-end projects), open-source or novice agents frequently exhibit the following typical failures:

- **Typical Case A (Isolated Fiction)**: The agent hand-writes a simple Python script or a single Java class under the system temp directory `/tmp/scratch_123`, runs it successfully, and then claims: "Feature delivered; all tests passed." But at this point the Git changes in the host target project's source directory are empty — no physical Diff was produced;
- **Typical Case B (Truncation Interception)**: When the agent runs `mvn test` and hits 50 complex dependency errors, it changes the command to `echo "BUILD SUCCESS" && exit 0`, or calls a stub function that pretends execution succeeded, feeding exit code 0 back to the upper-layer orchestrator;
- **Typical Case C (Non-In-Situ Build)**: Running a generic packager in an independent directory one level outside the project subdirectory, it misses the target project's specific `settings.xml` private-server mirror and parent POM constraints, and after a false pass directly causes production deployment crash upon release.

```
sequenceDiagram
    autonumber
    actor Developer as Host Orchestrator
    participant Agent as Development Agent (Actuator)
    participant Substrate as Physical Business Source Substrate (Native Substrate)
    participant Penetrator as Penetration Verification Middle Layer (Penetration Hook)
    participant Compiler as Native Compiler (Native Compiler/Toolchain)

    Developer->>Agent: Issue business task Brief
```

```
    Agent->>Substrate: Trigger physical code change (.java / .ts / .xml)
    Agent->>Penetrator: Submit "development complete" declaration

    rect rgb(240, 248, 255)
        note right of Penetrator: Substrate Penetration
        Penetrator->>Substrate: Physical check 1: Git Diff Substrate Check (do physical source ch
        Penetrator->>Compiler: Physical check 2: In-Situ Native Build (invoke native mvn/npm in p
        Compiler-->>Penetrator: Return native Exit Code & real compiler console stream
        Penetrator->>Substrate: Physical check 3: Artifact landing verification (do Target/Dist p
    end

    alt all three physical checks pass
        Penetrator-->>Developer: Confirm penetration success (Real Exit 0)
    else any item not satisfied
        Penetrator-->>Developer: Physical veto: judged as reward hacking (CRITICAL DECEPTION)
    end
```

---

3. Applicability

This pattern must be forcibly activated in the following scenarios: - Any enterprise software engineering involving existing dependency relationships, multi-module structures, or specific compilation plugins; - The delivery phase where the agent is commissioned to implement business features, fix defects, or deliver benchmark answers; - When evaluation systems need to judge the agent's objective problem-solving ability in autonomous coding.

---

4. Participants

1. **Substrate Mutator**: a restricted agent environment whose file-write operations are forcibly redirected to the physical root directory of the target business project; cross-directory escape is forbidden;
2. **In-Situ Native Runner**: bypasses all Python/Node simulation layers and directly invokes the official build tools installed on the host system (`mvn`, `gradle`, `npm`, `cargo`);
3. **Substrate Penetration Auditor**: a physical checker independent of the agent's runtime environment, responsible for mechanically verifying file entities, the Git working area, and artifact binaries.

---

5. Collaborations

```
stateDiagram-v2
    [*] --> SubstrateMutated: agent completes code editing
    SubstrateMutated --> DiffCheck: trigger verification

    state DiffCheck {
        [*] --> CheckDiffNonEmpty: git status & git diff
        CheckDiffNonEmpty --> DiffInvalid: changes in temp directory or no physical change
        CheckDiffNonEmpty --> DiffValid: hits whitelist physical entity code
```

```
}

DiffInvalid --> VetoFailed: judged false delivery (Exit 1)

DiffValid --> NativeBuild: physical in-situ build triggered
state NativeBuild {
    [*] --> InvokeCompiler: execute native command in place
    InvokeCompiler --> ParseExitCode: capture physical Exit Code
    ParseExitCode --> CheckDuration: check physical duration plausibility
}

NativeBuild --> BuildFailed: real compilation failure
NativeBuild --> ArtifactCheck: Exit 0 and physical duration plausible

ArtifactCheck --> ConvergencePassed: real artifacts landed (Exit 0)
ArtifactCheck --> VetoFailed: no physical bytecode generated (Exit 1)
```

---

6. Consequences

- **Pros**:
  - Fundamentally eliminates the agent's ability to "set its own exam and grade itself full marks" and to "write fake code in an empty directory to deceive the orchestrator";
  - Delivered code is 100% engineering-integrable, directly compatible with existing CI/CD pipelines.
- **Cons**:
  - Increased time cost: every verification must genuinely invoke the native build tool, taking as long as the project's real compilation time (usually tens of seconds to several minutes; "millisecond-level fast feedback" is impossible);
  - Strict environment dependency: the execution sandbox must have the full set of underlying SDKs required by the target project fully configured (JDK, Node, Maven local repository cache, etc.).

---

7. Anti-Patterns to Avoid

Anti-pattern name	Manifestation	Why it must be eliminated
Orphan Scratch Trap	The agent completes the task in a `temp/` directory and generates fake test scripts	The target project is unchanged; production is completely non-functional
Exit Code Mock Stubbing	A wrapper script uses `try...catch` to swallow build exceptions and unconditionally returns 0	Masks real compiler errors, letting faults slip through hidden
Flag Abuse (e.g. -DskipTests)	To achieve Exit 0, the agent appends skip flags to the command tail	Undermines the test safety net, letting regression defects slip into the system

---

8. Algorithmic Formulation

```
interface SubstratePenetrationVerifier {
  verify(workspace: TargetWorkspace): VerificationVerdict;
}

class InSituSubstrateVerifier implements SubstratePenetrationVerifier {
  verify(workspace: TargetWorkspace): VerificationVerdict {
    // 1. physical source Diff check
    const diff = workspace.git.diff();
    const modifiedBusinessFiles = diff.filter(f =>
      f.path.startsWith(workspace.targetModule) &&
      /\.(java|ts|vue|go|rs|py)$/.test(f.path)
    );
    if (modifiedBusinessFiles.length === 0) {
      return { ok: false, reason: "SUBSTRATE_DIFF_EMPTY: No physical code modified in target subs
    }

    // 2. native build in-situ penetration execution
    const startTime = Date.now();
    const buildResult = workspace.executeNativeCommand(
      workspace.nativeBuildTool, // e.g. "mvn clean test"
      workspace.targetModuleDir  // must be the target physical subdirectory
    );
    const durationMs = Date.now() - startTime;

    // 3. physical-plausibility check (a large multi-module build cannot take less than the physi
    if (durationMs < workspace.physicalMinDurationMs) {
      return { ok: false, reason: "PHYSICAL_IMPOSSIBILITY: Build completed too fast to be genuine
    }

    if (buildResult.exitCode !== 0) {
      return { ok: false, reason: "NATIVE_BUILD_FAILED", logs: buildResult.stderr };
    }

    return { ok: true, exitCode: 0, artifacts: workspace.listArtifacts() };
  }
}
```

Chapter 5: Pattern 02 — Reality Audit VETO Pattern

---

1. Intent

Grant the independent reviewer role (Reviewer) or audit gate the **veto power of physical-reality penetration verification (reality audit)**, preventing any code deliverable that is "formally compliant but physically false", "whose duration violates physical laws", or "whose logic is self-referential and high-scoring" from flowing into the integration pipeline.

---

2. Motivation & Real-World Failure Modes

In many multi-agent collaboration frameworks, the review role (Reviewer Agent) often degenerates into a **"schema format checkbox" or "code-style flatter"**: - **Failure scenario A (all format checks pass, all physics fake)**: The development agent outputs a delivery report fully conforming to the JSON Schema, with all `status` fields marked `"COMPLETED"`; the review agent only checked that the fields were complete, gave a full 9/9 score and approved it, yet underneath no real business logic was actually executed; - **Failure scenario B (physical infeasibility penetration failure)**: A Spring Cloud microservice project with 15 submodules; the development agent claims it ran full verification successfully, yet the console logs record a duration of only 12 milliseconds. The review agent failed to recognize this physical absurdity and misjudged it as pass; - **Failure scenario C (self-referential assertions passed)**: To pass tests, the development agent uses a Mock in test cases to directly return the input parameter (e.g. `when(service.verify(x)).thenReturn(x)`); the review agent performed no "anti-self-referential review", letting infinite-loop logic go live.

```
flowchart TD
    DevOutput["Development agent deliverables (code, report, declarations)"] --> Reviewer["Realit

    subgraph FourPillars["Four-Pillar Reality Audit"]
        direction TB
        Pillar1["1. Substrate Check<br/>Does the target project Git tree contain substantive phys
        Pillar2["2. Physical Feasibility Check<br/>Do multi-module compilation duration and resou
        Pillar3["3. Tautology Check<br/>Are assertions self-referentially echoed back by Mocks ac
        Pillar4["4. Isolated Environment Audit<br/>Are stress-test/independent-environment result
    end
```

```
    Reviewer --> FourPillars

    Pillar1 -- "no code changes" --> VETO["VETO: CRITICAL DECEPTION (rejected)"]
    Pillar2 -- "duration near zero / fake instant" --> VETO
    Pillar3 -- "set own exam, grade own paper" --> VETO
    Pillar4 -- "tampered results / selective reporting / environment drift" --> VETO

    FourPillars -- "all four checks fully pass" --> RubricAudit["Proceed to conventional code-qua
    RubricAudit --> Pass["Formal promotion and release"]
```

---

3. Applicability

- Any quality-gate stage (Review Gate) of multi-agent collaboration;
- Scenarios involving automated code merge (Auto-Merge), autonomous release, or benchmark leaderboard scoring;
- When preventing the agent from exploiting evaluator vulnerabilities for adversarial score-farming (reward hacking) is required.

---

4. Participants

1. **Dev Subordinate**: responsible for providing the code under review, build logs, and test manifests;
2. **Reality Auditor / Reviewer**: a read-only audit entity that does not directly modify business code; privileged to inspect the underlying physical Git state, native compiler real logs, and process-duration metadata;
3. **VETO Arbiter**: a hard-gate middleware that, once the Reality Auditor raises the `CRITICAL_DECEPTION` flag, immediately blocks the pipeline and freezes state regardless of how high the conventional score is.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Orchestrator as Orchestration state machine
    participant Dev as Dev Subordinate (Dev)
    participant Auditor as Reality Auditor
    participant Substrate as Physical system sensors

    Orchestrator->>Dev: Execute task development
    Dev->>Orchestrator: Submit delivery report (declares all tests passed)
    Orchestrator->>Auditor: Trigger review (inject delivery declaration)

    Auditor->>Substrate: Probe 1: read physical working-tree Diff
    Substrate-->>Auditor: Return Git Diff entity
    Auditor->>Substrate: Probe 2: read underlying compile duration and compiler output features
    Substrate-->>Auditor: Return real process clock and Stderr
```

```
Auditor->>Substrate: Probe 3: syntactically analyze unit-test assertions and Mock logic
Substrate-->>Auditor: Return assertion dependency graph
Auditor->>Substrate: Probe 4: isolated-environment result audit (stress test / independent en
Substrate-->>Auditor: Return raw stress-test reports, environment config fingerprint, result-

alt any physical fabrication found (Substrate Failure / False Claim)
    Auditor-->>Orchestrator: Verdict: VETO_REJECT (Score: 0, Severity: CRITICAL)
    Orchestrator->>Dev: Punitive rollback: force re-entry into the implementation loop with i
else all four physical checks pass
    Auditor-->>Orchestrator: Reality check passed, issue objective review score (Score >= Thr
end
```

Probe 4: Detailed Responsibilities of the Isolated Environment Audit

When the stress-test environment is completely isolated from the development environment, the auditor must additionally verify:

1. **Result authenticity (non-self-reported)**: stress-test results must come from the raw output of an independent execution system (e.g. raw JMeter/Gatling reports, Prometheus metrics export), not agent self-reports or relayed accounts;
2. **Integrity in transit**: signature verification and hash checks on the result-return channel, preventing selective reporting or tampering;
3. **Config drift detection**: the deviation of the stress-test environment's config fingerprint (OS/kernel/dependency versions/resource quotas) from the production environment must be within preregistered tolerances, preventing the drift failure of "passing in the test environment but crashing in production".

   Theoretical anchor: an extension of Axiom I's applicability boundary — for isolated environments the agent cannot directly access, results must be verified through an independent audit channel before they may be accepted.

---

6. Consequences

- **Pros**:
  - Eradicates "false prosperity" (within the tested environment and task family; F1/F2 10 injections each, interception rate 1.00, n=10) — any attempt to game prompt loopholes or bypass tests is mechanically intercepted;
  - Audit results carry an interpretable evidence chain (File:Line and real clock evidence).
- **Cons**:
  - The review process's context consumption and analysis latency increase somewhat;
  - The reviewer itself must possess high-dimensional code static-semantic analysis capability.

---

7. Anti-Patterns to Avoid

Anti-pattern name	Manifestation	Why it must be eliminated
Rubber-Stamp Reviewer	The reviewer blindly trusts the developer's narrative text and only proposes wording changes	Cannot intercept severe faults where no code was written at all or unit tests were broken
Assertion Softening	The reviewer acquiesces when the developer deletes/disables existing cases to turn the pipeline green	System resilience is permanently corroded

---

8. Algorithmic Formulation

```
interface RealityAuditVerdict {
  isVetoed: boolean;
  vetoReason?: "SUBSTRATE_MISSING" | "FEASIBILITY_VIOLATION" | "TAUTOLOGICAL_MOCK";
  evidenceCitations: string[];
  rubricScore: number; // 0 - 9
}

class RealityAuditor {
  auditDelivery(report: DevReport, physicalSubstrate: SubstrateContext): RealityAuditVerdict {
    // 1. physical code verification (Substrate Check)
    const actualDiff = physicalSubstrate.getGitDiff();
    if (!this.containsPhysicalTargetModifications(actualDiff, report.declaredModifiedFiles)) {
      return {
        isVetoed: true,
        vetoReason: "SUBSTRATE_MISSING",
        evidenceCitations: ["Declared modified files have zero physical diff in target tree"],
        rubricScore: 0
      };
    }

    // 2. physical plausibility check (Physical Feasibility Check)
    const compileTimeMs = physicalSubstrate.getPhysicalBuildDurationMs();
    if (compileTimeMs < physicalSubstrate.expectedMinimumFeasibleDurationMs) {
      return {
        isVetoed: true,
        vetoReason: "FEASIBILITY_VIOLATION",
        evidenceCitations: [`Build duration ${compileTimeMs}ms is physically impossible for subst
        rubricScore: 0
      };
    }

    // 3. anti-self-referential assertion check (Tautology Check)
    if (this.detectTautologicalAssertions(actualDiff)) {
      return {
        isVetoed: true,
```

```
        vetoReason: "TAUTOLOGICAL_MOCK",
        evidenceCitations: ["Found test assertions trivially satisfied by mirroring mock stubs"],
        rubricScore: 0
      };
    }

    // 4. after physical pass, proceed to conventional quality scoring
    return this.calculateRubricScore(report, actualDiff);
  }
}
```

---

9. Formal Theory Support (Lean 4 Formal Proof)

The core reality-audit veto power of this pattern has been formally verified by **Lean 4** (0 axioms, 0 sorry; source at `formal_proof/IACFormalization/src/IACFormalization/Pattern02_RealityAudit.lean`, corresponding to Appendix B.11):

- **Absolute-veto theorem (`veto_overrides_high_score`)**:

$$\forall d, \text{realityAudit}(d) = \text{false} \implies \text{admissionVerdict}(d) = \text{false}$$

  When the reality audit does not pass, regardless of how high the external score (e.g. a full Rubric score of 9) is, the admission criterion is always false.
- **Anti-self-referential cheating hard-block theorem (`tautology_vetoed`)**:

$$d.\text{isTautological} = \text{true} \implies \text{realityAudit}(d) = \text{false}$$

  Any deliverable that cheats a pass using self-referential mock stubs or tautologically true trivial assertions is mechanically blocked.
- **Necessary-and-sufficient physical verification (`four_pillars_soundness_and_completeness`)**: Admission holds if and only if the four physical Boolean hard criteria — "physical code changes", "compile duration conforming to physical plausibility", "anti-self-referential mock verification", and "environmental homogeneity" — together with the quality threshold, are all satisfied.

Chapter 6: Pattern 03 — Two-Phase Mandate Pattern

---

1. Intent

Strictly enforce, at the system architecture level, two-phase physical isolation between the **scaffolding/harness phase** and the **solution/implementation phase**, severing the state coupling and mutual cover-up between exam-setters and exam-solvers in the same spatiotemporal environment.

---

2. Motivation & Real-World Failure Modes

In many end-to-end agent pipelines or benchmark evaluation systems, the most frequent catastrophic design flaw is **"scaffolding and business implementation mixed in the same task loop"**: - **Failure scenario A (exam questions colluding with answer tampering)**: The agent is responsible for writing both evaluation unit tests and business code in the same session. When business code cannot be written, the agent dynamically modifies test inputs or expected outputs, letting incomplete code "pass smoothly" — creating false prosperity in the system; - **Failure scenario B (scaffolding half-finished goods masquerading as deliverables)**: In the planning phase, the agent mixes setting up the Docker environment, writing the evaluation harness, and implementing business features in one Todo list. When the scaffolding code is just written and business features not yet started, the agent claims "the project has been fully delivered" based on the scaffolding itself compiling successfully; - **Failure scenario C (environmental state mutual contamination)**: Temporary environment variables or mock ports on which the scaffolding runs leak directly into the solution-phase runtime, causing the solution code to develop strong implicit dependencies on the scaffolding environment; once separated from the scaffolding, it cannot run independently.

```
flowchart TD
    subgraph AntiPattern["Anti-pattern: Single-Phase Entanglement"]
        direction TB
        AgentMixed["same agent session"] --> EditTest["write/modify test cases"]
        EditTest --> EditCode["write business implementation"]
        EditCode --> TestFail{"tests not passing?"}
        TestFail -- "dynamically lower test standards" --> EditTest
        TestFail -- "change business code" --> EditCode
        EditCode --> FalseGreen["false self-consistent pass (untrustworthy)"]
```

```
    end

    subgraph TwoPhasePattern["Recommended pattern: Two-Phase Mandate"]
        direction TB
        Phase1["Phase 1: harness scaffolding development (Harness Phase)"] --> Freeze["Physical f
        Freeze --> Boundary["Strongly isolated physical boundary (Read-Only Substrate Mount)"]
        Boundary --> Phase2["Phase 2: pure solution implementation (Solution Phase)"]
        Phase2 --> NativeAgent["Solution agent (restricted mutation permissions; tampering with t
        NativeAgent --> RealPass["real objective convergence (objectively trustworthy)"]
    end
```

---

3. Applicability

- Software-engineering autonomous evaluation benchmarks and automated exam systems;
- Automated agent pipelines adopting test-driven development (TDD) or acceptance-test-driven development (ATDD);
- R&D pipelines with extremely high internal requirements for software security and compliance.

---

4. Participants

1. **Harness Architect**: runs only in Phase 1; responsible for setting up the physical sandbox, injecting test probes and assertions, and outputting the frozen read-only benchmark specification;
2. **Substrate Freezer**: responsible for signing Phase-1 artifacts as read-only, imposing Write-Protect (read-only mount) at the filesystem level, and clearing temporary environments;
3. **Solution Implementer**: runs in Phase 2, with write permission only to the target business code directory; absolutely read-only to test cases and scaffolding code;
4. **Blind Verdict Engine**: reads solution artifacts in a fully isolated container environment and executes the verdict frozen in Phase 1, uncontrolled by any Phase-2 agent.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Orchestrator as Orchestration system
    participant Harness as Phase 1: harness engineering (Harness)
    participant Freezer as Physical Substrate Freezer (Freezer)
    participant Solution as Phase 2: solution agent (Solution)
    participant Sandbox as read-only sandbox host

    Orchestrator->>Harness: trigger environment initialization and acceptance-case freezing
    Harness->>Sandbox: deploy hidden oracle and immutable test suite
    Harness-->>Orchestrator: Phase 1 complete (outputs read-only image and mount specification)
```

```
Orchestrator->>Freezer: execute physical isolation lock
Freezer->>Sandbox: set test directory read-only permission (chmod a-w / Read-Only Mount)

Orchestrator->>Solution: start Phase 2 business development (mount restricted workspace)
Solution->>Sandbox: attempt to modify business source (Allowed)
opt attempt out-of-bounds tampering with test cases
    Solution->>Sandbox: attempt to write back test files
    Sandbox-->>Solution: hardware interception: EACCES Permission Denied (severely penalized
end
Solution-->>Orchestrator: Phase 2 solution complete

Orchestrator->>Sandbox: trigger double-blind mechanical verdict
Sandbox-->>Orchestrator: output objective physical score
```

---

6. Consequences

- **Pros**:
  - Eliminates the moral hazard of the agent "loosening test assertions to slip through" (controlled experiment: assertion-tampering interception rate 1.00 in isolated environments);
  - Guarantees the absolute objectivity, fairness, and reproducibility of evaluation benchmarks.
- **Cons**:
  - The pipeline must undergo explicit phase decomposition and workspace splitting; the initial configuration cost is higher than a monolithic session;
  - If the Phase-1 scaffolding itself contains bugs, the solution-phase agent cannot fix it on its own, and an explicit rollback-and-refactor must be triggered.

---

7. Anti-Patterns to Avoid

Anti-pattern name	Manifestation	Why it must be eliminated
Blended Task List	Listing "write test scaffolding" and "write business implementation" in the same task plan	Causes confusion of phase artifacts, making the true delivery node extremely hard to define
Dynamic Assertion Loosening	Allowing the solution agent to submit PRs modifying test files when tests fail	Destroys the protective dyke of tests, letting software defects escape

---

8. Algorithmic Formulation

```typescript
interface PhaseBoundaryEnforcer {
  transitionToSolutionPhase(harnessWorkspace: Workspace): SolutionWorkspace;
}
```

```
class TwoPhaseSandboxEnforcer implements PhaseBoundaryEnforcer {
  transitionToSolutionPhase(harnessWorkspace: Workspace): SolutionWorkspace {
    // 1. physical freezing: check Phase-1 test asset integrity
    const testSpecs = harnessWorkspace.getTestSuites();
    if (testSpecs.length === 0) {
      throw new Error("PHASE_1_INVALID: No test harness or specs established");
    }

    // 2. set test and scaffolding directories strictly read-only (Read-Only Mount)
    harnessWorkspace.protectDirectory("src/test", { readonly: true });
    harnessWorkspace.protectDirectory(".harness", { readonly: true });

    // 3. construct an isolated solution workspace (write permission opened only for business sou
    return harnessWorkspace.spawnSubordinateWorkspace({
      writablePaths: ["src/main"],
      immutablePaths: ["src/test", "pom.xml", ".harness"],
      envIsolation: true
    });
  }
}
```

---

9. Epistemic Status

> **Engineering Convention**: This pattern is an unfalsifiable operational specification, not a theoretical conclusion. Its correctness can only be judged by "whether the implementation follows the prescribed invariants"; the pattern itself gives no statistical criterion, nor does it constitute a corollary of Axioms I–IV or Theorem 3.1.

"Scaffolding read-only / solution domain writable" is a mandatory constraint, not an empirical finding. To elevate it into a theoretical proposition, one must first give a measurable predicate (e.g., "the write-attempt failure rate under read-only mount = 1.00") and test it via a preregistered experiment. In the §8 matrix of the validation report, this pattern is classified as an engineering convention; its statements must not be used as reasoning premises for other patterns.

Chapter 7: Pattern 04 — Task-Level PoLP Context Slicing Pattern

---

1. Intent

Grounded in the **Principle of Least Privilege (PoLP)** from cybersecurity and system control theory, the upper-layer orchestrator performs dynamic context slicing and dimensionality-reducing projection when dispatching a single task to a development agent: it **injects only the table schemas, interface definitions, and orthogonal files that the current task declares to read or write**, compressing >60% of irrelevant context and significantly suppressing context rot and out-of-scope code mutations.

---

2. Motivation & Real-World Failure Modes

In agentic development on complex codebases, the most prevalent anti-pattern is **"stuffing the entire knowledge base / global design blueprint into the prompt at once"**:

- **Failure Mode A (context rot)**: when a global architecture design document containing 30 database tables and 50 endpoint definitions (hundreds of thousands of tokens) is fed in full to an agent responsible for implementing a single gateway filter, the model readily exhibits "lost in the middle" (Liu et al., TACL 2024), generating large numbers of unrelated helper classes while missing the core logic;
- **Failure Mode B (excessive privilege causing cross-boundary damage)**: because the context includes the entire engineering configuration and all existing unit tests, an agent developing gateway desensitization logic may, upon hitting a compile error, conveniently modify code in the user authentication module or delete existing unit tests of the security module, causing severe and hidden logical regressions;
- **Failure Mode C (global noise pollution from rejection feedback)**: when the gate rejects a minor defect (e.g., an omitted non-null parameter check), re-injecting the entire bulky gate report and historical logs causes the agent to fall into over-healing, rewriting and erasing previously correct code belonging to other tasks.

```
flowchart TD
    GlobalDesign["Full macro system design blueprint (Global Blueprint: 50+ Endpoints, 20+ Tables

    subgraph SlicingEngine["Targeted slicing engine (PoLP Slicing Engine)"]
        direction TB
        Filter1["Extract the read/write table list specified by the current task"]
```

```
        Filter2["Extract the API endpoints orthogonally declared by the current task"]
        Filter3["Extract targeted whitelisted target file paths (Files In-Scope)"]
        Filter4["Targeted filtering of gate rejection feedback (retain only associated findings)"
    end

    Projector --> SlicingEngine

    SlicingEngine --> TaskSlice["Task-level micro slice (TaskDesignSlice: <5K Tokens)"]
    TaskSlice --> DevAgent["Development agent (Focus & Low Cognitive Load)"]
    DevAgent --> BoundedDiff["Bounded precise code mutation (Zero Out-of-Scope Mutations)"]
```

---

3. Applicability

- Incremental requirement development on large multi-module monoliths or microservice repositories;
- Pipeline execution stages with clear task decomposition;
- Scenarios where context windows are expensive, or where long context causes significant attention degradation in the model.

---

4. Participants

1. **Global Blueprint Store**: holds the complete architecture contract (data tables, interfaces, global decisions) for the entire requirement lifecycle;
2. **Context Projector**: based on the metadata of the currently dispatched task (`taskId`, `files`, `modules`), dynamically extracts a subgraph in memory and generates the task-specific minimal slice `TaskDesignSlice`;
3. **PoLP Subordinate Agent**: receives only the sliced context, is forbidden from consulting the full blueprint for undeclared paths, and is forced to complete implementation within the declared boundaries;
4. **Scope Boundary Guard**: when code is written back to the physical substrate, verifies whether the Git diff exceeds the file boundaries declared by the task; out-of-scope changes are rejected outright.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Orchestrator as Orchestration system
    participant Projector as Least-privilege projector (PoLP Projector)
    participant Dev as Development agent (Actuator)
    participant Guard as Scope boundary guard (Scope Guard)
    participant Substrate as Physical filesystem

    Orchestrator->>Projector: Request dispatch of subtask [T2: response-header desensitization fi
    Projector->>Projector: Dynamic filtering: remove irrelevant information such as the auth cent
    Projector->>Projector: Extract target interface contracts and gateway module boundary definit
```

```
Projector-->>Orchestrator: Return precise TaskDesignSlice (token reduction >65%)

Orchestrator->>Dev: Inject TaskDesignSlice and issue development instructions
Dev->>Guard: Submit the file modification action list
Guard->>Guard: Verify whether modifiedFiles belong to declaredAllowedPaths

alt Agent attempts to modify the unauthorized UserAuthFilter.java
    Guard-->>Dev: Hardware interception: EPERM_OUT_OF_SCOPE (out-of-scope modification blocke
else Only modifies SensitiveHeaderFilter.java within the declared scope
    Guard->>Substrate: Allow physical write to disk
    Guard-->>Dev: Write succeeded
end
```

---

6. Consequences

- **Pros**:
  - **Cliff-like drop in token consumption**: by removing the massive redundancy of the full design, single-task inference energy consumption drops by 60%–80%;
  - **Extreme attention focus**: eliminates the "needle in a haystack" problem of long texts, and the accuracy of single code generation improves significantly;
  - **Side effects eliminated**: modifications across tasks do not overlap, naturally supporting parallelized development and topological scheduling;
- **Precise convergence of self-healing retries**: gate rejection feedback is targetedly filtered to return only findings associated with the task, avoiding the over-healing anti-pattern.
- **Cons**:
  - Requires the upfront design phase to possess high-quality structured interface and module decomposition capabilities; if the architecture design cuts blurrily, the projected slice may lack necessary dependency information;
  - Requires the orchestrator to maintain a bidirectional dynamic mapping between architecture blueprint and task slices, increasing the metadata maintenance cost of the orchestration layer;
  - For strongly coupled cross-module refactoring tasks, overly tight slice declarations may cause missing-modification compile errors, and an on-demand topological expansion mechanism must be provided.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be eradicated
Monolithic Design Dumping / Blueprint Dump	Indiscriminately concatenating hundreds of pages of the entire system's design documents or the complete microservice architecture diagram of hundreds of thousands of tokens into the prompt on every Dev Agent invocation	Causes the model to get lost in massive information and hallucinate, producing large numbers of out-of-scope modifications
Unbounded Write Access / Unbounded Editing	Allows the development agent to modify any file in the project (including external files not declared in the task contract) at will without boundary interception	Highly prone to hidden logical damage and catastrophic regressions such as modifying shared base classes or deleting unit tests

---

8. Algorithmic Formulation

```typescript
interface TaskItem {
  taskId: string;
  targetFiles: string[];
  referencedEndpoints: string[];
}

interface GlobalDesignContract {
  summary: string;
  endpoints: EndpointDefinition[];
  tables: TableSchema[];
  decisions: ArchitecturalDecision[];
}

class TaskLevelContextProjector {
  projectTaskSlice(task: TaskItem, globalDesign: GlobalDesignContract): TaskDesignSlice {
    // 1. Filter orthogonal API endpoints
    const relevantEndpoints = globalDesign.endpoints.filter(ep =>
      task.referencedEndpoints.includes(ep.path)
    );

    // 2. Filter orthogonal database table schemas
    const relevantTables = globalDesign.tables.filter(tbl =>
      relevantEndpoints.some(ep => ep.relatedTables?.includes(tbl.tableName))
    );

    // 3. Targeted decision projection (retain only architectural decisions involving the current
    const relevantDecisions = globalDesign.decisions.filter(d =>
      task.targetFiles.some(f => f.includes(d.affectedModule))
```

```
    );

    return {
      taskId: task.taskId,
      scopedFiles: task.targetFiles,
      endpoints: relevantEndpoints,
      tables: relevantTables,
      decisions: relevantDecisions,
      tokenReductionRatio: this.calculateReductionRatio(globalDesign, relevantEndpoints)
    };
  }
}
```

---

9. Lean 4 Formal Proof

The least-privilege projection and SNR gain of this pattern have been verified by **Lean 4** mechanized formal proof (0 axioms, 0 sorry; source: `formal_proof/IACFormalization/src/IACFormalization/Pattern` corresponding to Appendix B.13):

- **Least-privilege projection fidelity theorem (`polp_preserves_snr_strictly`)**:

$$\forall \text{ctx}, b_{ext} > 0 \implies \text{snr}(\text{projectPoLP}(\text{ctx})) > \text{snr}(\text{ctx})$$

  Pruning externally irrelevant redundant context necessarily and strictly improves the task-objective retrieval SNR.
- **Out-of-scope violation probability reduction theorem (`polp_strictly_reduces_violation_risk`)**:

$$\forall \text{ctx}, b_{ext} > 0 \implies P_{\text{violate}}(\text{projectPoLP}(\text{ctx})) < P_{\text{violate}}(\text{ctx})$$

  Least-privilege projection strictly reduces the model's attention degradation and out-of-scope code tampering risk.
- **Privilege nesting monotonicity theorem (`polp_monotone_nesting`)**:

$$b_1 \leq b_2 \implies P_{\text{violate}}(a, b_1) \leq P_{\text{violate}}(a, b_2)$$

  The tighter the projected slice, the monotonically lower its upper bound on violation risk.

Chapter 8: Pattern 05 — Macro-Micro Elaboration Pattern

---

1. Intent

Decouples the system architecture design process of complex software engineering into a two-stage progressive loop: **"Stage 1: Macro Blueprint"** and **"Stage 2: Micro Elaboration"**. Through layered focusing of cognitive bandwidth, it eliminates the attention exhaustion, long-output truncation, abstraction–detail disconnection, and pseudo-contract hallucination that arise when agents attempt to "reach the sky in one step" within a single generation.

---

2. Motivation & Real-World Failure Modes

When autonomous software agent systems advance to enterprise-grade engineering (e.g., Spring Cloud microservices, large distributed topologies), the design phase is often the core bottleneck that determines the survival of the entire system. The traditional "one-shot monolithic design" mode inevitably triggers the following three classes of severe failures in industrial scenarios:

- **Failure Mode A (space-cadet truncation)**: The orchestrator demands that the agent, in a single inference, "output a complete design document containing the system topology, database table schemas, detailed definitions of 50 REST API endpoints, and security policies." Due to the single-output token cap (typically 4K–8K tokens) and attention decay induced by context rot, large models typically spend the first half grandly laying out concepts, only to be forcibly truncated at the 5th endpoint when hitting the token limit; or they perfunctorily write `// ... the remaining 45 endpoints are the same ...` as placeholders in the second half, leaving downstream development agents with no basis to implement;
- **Failure Mode B (vague hand-waving and unverifiable hallucination)**: Although the agent outputs a document whose structure appears complete, the whole text is filled with hollow slogans (e.g., "distributed locks are adopted to ensure consistency," "a high-performance gateway filter is introduced to implement desensitization"), yet lacks the micro facts indispensable to physical implementation: exact fully-qualified class names (FQCN), method input/output data structures, exception inheritance hierarchies (e.g., whether `BadCredentialsException` is inherited), and HTTP status codes. Development agents can only guess on their own when landing the implementation, causing interface tears during multi-module integration;

- **Failure Mode C (macro-micro non-closure and architectural betrayal)**: The macro chapters claim that "the system adopts a strictly stateless JWT scheme," yet some endpoint definition at the end requires sharing the login state through `HttpSession`. A monolithic large model cannot maintain strict top-down mathematical closure when handling cross-level logic.

```
flowchart TD
    subgraph MonolithicAntiPattern["Anti-pattern: one-shot monolithic design generation (One-Shot
        direction TB
        TaskBrief["Requirement task specification (Brief)"] --> MonolithicLLM["Single overlong in
        MonolithicLLM --> Truncation["Attention exhausted / output truncated at the token limit"]
        Truncation --> FlawedContract["Incomplete / macro-micro disconnected design contract (Dow
    end

    subgraph TwoStagePattern["Recommended pattern: two-stage macro-micro progressive elaboration
        direction TB
        Brief["Requirement task specification (Brief)"] --> Stage1["Stage 1: macro global bluepri

        subgraph Stage1Scope["Stage 1 focus: global skeleton and boundary decisions"]
            S1_1["1. Business-domain module decomposition and topological boundaries"]
            S1_2["2. Core data entities and topological relationship diagram"]
            S1_3["3. Key technology selection and ADR architecture decision records"]
            S1_4["4. Endpoint catalog index (route list and responsibilities only)"]
        end
        Stage1 --> Stage1Scope

        Stage1Scope --> BlueprintGate{"Stage 1 blueprint quality gate<br/>(Blueprint Gate: thresh
        BlueprintGate -- "rejected and corrected" --> Stage1

        BlueprintGate -- "approved and frozen" --> Stage2["Stage 2: micro physical contract elabo

        subgraph Stage2Scope["Stage 2 focus: physical penetration of contracts item by item"]
            S2_1["1. Strictly align FQCN fully-qualified class names and inheritance trees"]
            S2_2["2. Endpoint DTO Schema (Jackson properties / required validation)"]
            S2_3["3. Exception propagation contract (precise down to caught types and throwing ti
            S2_4["4. Unit-test instantiation assertion specification (BDD acceptance cases)"]
        end
        Stage2 --> Stage2Scope

        Stage2Scope --> ElaboratedSpecs["High-density, 100% physically verifiable living design c
    end
```

---

3. Applicability

This pattern must be forcefully activated in the following scenarios:

- When requirement complexity spans more than 2 service modules or involves more than 3 core entities;
- High-rigor scenarios where the architecture design must pass scoring review by automated

gates (e.g., Jev, architecture static verifier);
- Upstream contract generation for downstream parallel implementation of subtasks by multiple isolated development agents;
- System evolution processes that require outputting long-term maintainable "living specifications".

---

4. Participants

1. **Macro Architect Agent**: Runs in Stage 1, focusing only on "the system's skeleton and non-functional constraints." Its cognitive bandwidth is 100% focused on module responsibility decomposition, storage topology, core data flows, architecture decision records (ADRs), and the endpoint catalog index — never bogged down in the minutiae of single-interface fields;
2. **Blueprint Gate Auditor**: An independent quality-check entity that performs formal review of the `design-blueprint.json` produced in Stage 1 (verifying the three ADR elements, whether the trade-off matrix is closed, and whether module dependencies contain cycles), and freezes it upon passing;
3. **Micro Elaborator Agent**: Runs in Stage 2, receives the frozen macro blueprint, and performs physical-level concrete elaboration of specific endpoints, entity tables, and exception classes in a divide-and-conquer manner, outputting the ultimate design contract conforming to the Draft-07 Schema;
4. **Consistency Auditor**: Mechanically compares whether the micro elaboration details faithfully follow the macro architectural decisions, blocking any micro-dimensional "ultra vires tampering" or "architectural betrayal".

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Orchestrator as Orchestration system
    participant Macro as Stage 1: macro architect (Macro Architect)
    participant Gate as Blueprint gate (Blueprint Gate)
    participant Micro as Stage 2: micro elaborator (Micro Elaborator)
    participant Substrate as Physical design artifact store (Design Artifact Store)

    Orchestrator->>Macro: Dispatch design requirement Brief & existing engineering context
    Macro->>Macro: Abstract system boundaries, design module topology, select ADR solutions
    Macro-->>Substrate: Persist Stage 1 blueprint (design-blueprint.json)

    Orchestrator->>Gate: Trigger blueprint gate audit
    Gate->>Substrate: Verify blueprint structural completeness and technical selection feasibilit
    Gate-->>Orchestrator: Blueprint score reaches threshold (Score: 8.0, APPROVED)

    Orchestrator->>Micro: Inject the frozen macro blueprint, start micro contract elaboration
    loop Endpoint and class-by-class contract elaboration
        Micro->>Micro: Elaborate precise DTO fields, status codes, exception classes, and unit-te
        Micro->>Substrate: Incrementally write micro-specs/*.json
```

```
    end

    Micro->>Substrate: Merge into the final final-review design contract (design.json)
    Substrate-->>Orchestrator: Macro-micro double-layer closed design contract ready, release int
```

---

6. Consequences

- **Pros**:
  - **Eliminates the high-cost failure of token truncation**: decomposes a complex system design of hundreds of thousands of tokens into two deterministic sub-stages; the model's focus per inference greatly improves, and the output contains no deletions or omission placeholders;
  - **Achieves "physical implementability"**: micro elaboration directly generates precise signatures and assertion code at the Java/TypeScript level; development agents need no second guessing, and the one-pass compile success rate improves by more than 80%;
  - **Architectural decisions are rigorously controlled**: once the macro architecture blueprint is locked, micro elaboration cannot override upper-layer technology selections on its own, eradicating module-level autonomy;
- **Cons**:
  - The pipeline adds at least one independent model inference and gate scheduling overhead in the Design phase, increasing the initial design cycle time by 30%–50%;
  - Requires defining and maintaining two orthogonal sets of JSON Schema contract systems (macro and micro) at the platform layer.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be eradicated
One-Step Monolithic Fallacy	Attempting to have the model directly output a complete micro code-level design contract through one overlong prompt	Inevitably triggers output truncation or large-scale content omission in later endpoints
Pseudo-Micro Specifications	The micro part gives only the interface path `/api/v1/user`, lacking request body field types and validation rules	Developers/agents lack constraints when landing, causing large-scale rework in the downstream integration phase
Bottom-Up Architectural Drift	Allowing the micro elaboration phase to arbitrarily add, delete, or even overturn the ADR decisions established in the macro phase	Destroys global consistency and leads to uncontrolled system architecture evolution

---

8. Algorithmic Formulation & Contract Schema

8.1 Stage 1 Macro Blueprint Contract Schema (`design-blueprint.schema.json`)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "MacroArchitecturalBlueprint",
  "type": "object",
  "required": ["system_summary", "modules", "architectural_decisions", "endpoint_index"],
  "properties": {
    "system_summary": { "type": "string" },
    "modules": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["name", "responsibility", "tech_stack"],
        "properties": {
          "name": { "type": "string" },
          "responsibility": { "type": "string" },
          "tech_stack": { "type": "string" }
        }
      }
    },
    "architectural_decisions": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["id", "context", "decision", "consequences"],
        "properties": {
          "id": { "type": "string" },
          "context": { "type": "string" },
          "decision": { "type": "string" },
          "consequences": { "type": "string" }
        }
      }
    },
    "endpoint_index": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["method", "path", "owner_module", "brief_summary"],
        "properties": {
          "method": { "type": "string", "enum": ["GET", "POST", "PUT", "DELETE", "PATCH"] },
          "path": { "type": "string" },
          "owner_module": { "type": "string" },
          "brief_summary": { "type": "string" }
        }
      }
    }
  }
```

```
}
```

8.2 Stage 2 Micro Elaboration Core Execution Algorithm

```
interface MacroBlueprint {
  system_summary: string;
  modules: ModuleSpec[];
  architectural_decisions: ADR[];
  endpoint_index: EndpointBrief[];
}

interface MicroSpecification {
  fqcn: string;
  endpoints: DetailedEndpointSpec[];
  exceptionHierarchy: ExceptionSpec[];
  testInvariants: BddAssertionContract[];
}

class ProgressiveDesignOrchestrator {
  async executeTwoStageDesign(brief: RequirementBrief): Promise<FinalDesignContract> {
    // ------------------------------------------------------------
    // Stage 1: macro global blueprint design (Macro Blueprint Generation)
    // ------------------------------------------------------------
    const macroPrompt = this.buildMacroPrompt(brief);
    const blueprint: MacroBlueprint = await this.macroArchitect.generateBlueprint(macroPrompt);

    // Stage 1 blueprint quality gate review (Jev/Auditor Gate)
    const blueprintAudit = await this.gateAuditor.evaluateBlueprint(blueprint);
    if (blueprintAudit.score < 7.0) {
      throw new Error(`BLUEPRINT_REJECTED: ${blueprintAudit.feedback}`);
    }

    // ------------------------------------------------------------
    // Stage 2: item-by-item physical contract elaboration (Micro Elaboration)
    // ------------------------------------------------------------
    const microSpecs: MicroSpecification[] = [];

    // Divide-and-conquer elaboration of the endpoint index in the blueprint
    for (const epBrief of blueprint.endpoint_index) {
      const elaborationPrompt = this.buildElaborationPrompt(epBrief, blueprint);
      const detailedSpec = await this.microElaborator.elaborateEndpoint(elaborationPrompt);

      // Macro-micro closure verification: ensure micro elaboration strictly follows macro ADR de
      this.validateMacroMicroAlignment(blueprint, detailedSpec);
      microSpecs.push(detailedSpec);
    }

    // ------------------------------------------------------------
    // Merge and freeze into the final design contract (design.json)
```

```
    // ------------------------------------------------------------
    return this.synthesizeFinalContract(blueprint, microSpecs);
  }
}
```

---

9. Epistemic Status of Quantitative Claims

Disposition: downgraded to empirical target (unverified) — the "one-pass compile success rate improvement of more than 80%" and the "design cycle increase of 30%–50%" in §6 of the main text lack preregistered criteria and paired design, and the definition of "one-pass" (whether self-healing retries and scaffold corrections are included) is undefined. Both are now marked as **engineering empirical targets** and are not cited as theoretical conclusions. To be elevated, the caliber must first be fixed and a paired A/B (same model, same batch) must be conducted on the same task family.

Chapter 9: Pattern 06 — Streaming Decomposition & Selective Resume Pattern

---

1. Intent

Restructures the requirement decomposition process of large engineering tasks from one-shot global batch generation into **streaming incremental decomposition**, and combines it with a directed acyclic graph (DAG) topological state machine to achieve **selective resume**, effectively countering context rot in overlong task generation (Liu et al., 2024) and achieving precise targeted self-healing with zero redundant computation on environment crashes or task retries.

---

2. Motivation & Real-World Failure Modes

When facing industrial-grade requirements containing dozens of subtasks (e.g., full-stack microservice feature migration), the traditional one-shot global planning exhibits significant failure:

- **Failure Mode A (context rot at the planning level)**: The orchestrator demands that the product manager agent (PM Agent) output in one shot a complete JSON list of 15 tasks (including each task's dependencies, acceptance criteria, and file scope). Due to context rot and lost in the middle, the model typically shows rapid attention fatigue after completing the first 3 tasks, with the latter half of tasks exhibiting large-scale content repetition, garbled dependency cycles, or missing Schema fields;
- **Failure Mode B (high overhead of cascading re-run)**: In the execution phase, suppose the 12th task is rejected by the gate due to a micro assertion error. In a system lacking incremental state topology, the orchestrator often triggers a global re-run, pushing all of the first 11 tasks that already passed strict compilation and testing through from scratch, causing millions of tokens of wasted computation and physical workspace pollution;
- **Failure Mode C (crash recovery failure after unexpected interruption)**: In long-cycle engineering runs, the underlying host may be interrupted by network fluctuations, OOM, or service restarts. Without fine-grained streaming persistence and causal state snapshots, the system cannot recover the scene, causing permanent loss of engineering progress.

```
flowchart TD
    subgraph Stage1Streaming["Stage 1: streaming outline expansion (Streaming Outline)"]
        RawBrief["User raw requirement specification (Brief)"] --> OutlineAgent["Outline agent (O
        OutlineAgent --> OutlineDAG["Lightweight task topology outline (Task Outline DAG: T1 -> T
    end

    subgraph Stage2Incremental["Stage 2: item-by-item crystallization to disk (Item-by-Item Cryst
        OutlineDAG --> T1["Stream-generate task T1 detailed contract -> persist T1.json to disk"]
        OutlineDAG --> T2["Stream-generate task T2 detailed contract -> persist T2.json to disk"]
        OutlineDAG --> T3["Stream-generate task T3 detailed contract -> persist T3.json to disk"]
    end

    subgraph SelectiveResumeEngine["Selective resume engine (Selective Resume Engine)"]
        RunStatus["Global causal snapshot (run-status.json)"]
        T1Pass["T1: acceptance passed (PASS)"]
        T2Fail["T2: rejected by physical gate (FAIL)"]
        T3Pending["T3: pending execution (BLOCKED)"]

        RunStatus --> T1Pass
        RunStatus --> T2Fail
        RunStatus --> T3Pending

        T2Fail -- "precise targeted re-decomposition" --> ReplanT2["Regenerate only the T2.json f
        ReplanT2 --> ResumeExec["Resume execution precisely from T2, skipping T1"]
    end

    Stage2Incremental --> RunStatus
```

---

3. Applicability

- Complex software engineering tasks containing more than 3 strongly dependent sub-steps;
- Long-cycle autonomous pipelines prone to external environment jitter, model inference timeouts, or server restarts;
- Cost-sensitive enterprise systems that need strict control of token energy consumption and avoid recomputation of already-passed tasks.

---

4. Participants

1. **Outline Streamer**: responsible for the global skeleton topology deduction in the first stage, producing only task identifiers, dependency predecessors, and one-sentence responsibilities, blocking the model from sinking into details early on;
2. **Fragment Materializer**: invokes sub-agents one by one in topological order to materialize each task, immediately persisting an independent fragment contract to disk (`tasks/T{N}.json`) as each task is generated;
3. **Resume Snapshot Coordinator**: maintains a monotonically increasing causal state file `run-status.json`, recording the current lifecycle of each task (`PENDING`, `IN_PROGRESS`, `SUCCESS`, `FAILED`);

4. **Targeted Healing Re-trier**: upon receiving a downstream gate rejection signal, locates the minimal closure of polluted tasks through DAG topology analysis, marks only the failed task as `STALE` and triggers local re-decomposition, strictly reusing undamaged fragments.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Orchestrator as Orchestration engine
    participant PM as Planning agent
    participant FS as On-disk fragment store (tasks/)
    participant Snapshot as Run snapshot (run-status.json)
    participant Dev as Development execution layer

    Orchestrator->>PM: Start requirement analysis
    PM->>PM: Stage 1: stream-generate a lightweight outline (task-outline.json)
    PM-->>FS: Persist task-outline.json (defines T1, T2, T3)

    loop Stage 2: materialize task by task
        Orchestrator->>PM: Materialize single-task contract [T_k]
        PM-->>FS: Atomically write tasks/T_k.json
        PM->>Snapshot: Update task progress and SHA-256 checksum fingerprint
    end

    Orchestrator->>Dev: Dispatch T1 for execution (passed)
    Orchestrator->>Dev: Dispatch T2 for execution (hit a compiler error, rejected)

    note over Orchestrator,Snapshot: Trigger selective resume (Selective Resume)
    Orchestrator->>Snapshot: Read the existing Run ID and completed task set
    Snapshot-->>Orchestrator: T1 status is SUCCESS (retained), T2 status is FAILED
    Orchestrator->>FS: Delete only the tasks/T2.json fragment
    Orchestrator->>PM: Targeted re-decomposition of task T2 (inject error-stack evidence)
    PM-->>FS: Generate the repaired tasks/T2.json
    Orchestrator->>Dev: Resume execution from T2, skipping T1 directly
```

---

6. Consequences

- **Pros**:
  - **Avoids context rot**: each task's materialization prompt contains only the outline and the current slice, with context length constantly controlled in the low-entropy band;
  - **Instant fault recovery (Instant MTTR)**: thanks to independent per-task on-disk fragments (`T1.json`, `T2.json`), after a system crash and restart the recovery point can be located at millisecond level, skipping already-passed high-score gates;
  - **Self-healing computation overhead minimized**: local rejection only re-decomposes the single task, saving more than 70% of retry tokens.

- **Cons**:
  - Requires the orchestrator's underlying layer to maintain state snapshots and lock mechanisms based on the filesystem or distributed KV;
  - Cross-task data sharing between subtasks must be transmitted through explicitly declared output contracts.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be eradicated
Monolithic Task Dumping	Demands the model directly output all subtask details in one overlong array	Triggers context rot (Liu et al., 2024), causing the quality of later tasks to decline sharply
All-or-Nothing Cascade	When a micro unit test fails, resetting the entire pipeline to its initial state and regenerating	Extremely wasteful of energy and destroys the stability of already-verified code assets

---

8. Algorithmic Formulation

```
interface TaskOutline {
  tasks: Array<{ id: string; title: string; dependencies: string[] }>;
}

class StreamingDecompositionEngine {
  async planAndResume(brief: string, workDir: string, resumeMode: boolean): Promise<TaskQueue> {
    const outlinePath = path.join(workDir, "task-outline.json");
    let outline: TaskOutline;

    // 1. Outline generation or reuse (Stage 1)
    if (resumeMode && fs.existsSync(outlinePath)) {
      outline = JSON.parse(fs.readFileSync(outlinePath, "utf8"));
    } else {
      outline = await this.pmAgent.generateOutline(brief);
      fs.writeFileSync(outlinePath, JSON.stringify(outline, null, 2));
    }

    // 2. Streaming item-by-item materialization (Stage 2)
    const materializedTasks: TaskItem[] = [];
    for (const taskStub of outline.tasks) {
      const fragmentPath = path.join(workDir, "tasks", `${taskStub.id}.json`);

      if (resumeMode && fs.existsSync(fragmentPath)) {
        // Resume point hit: reuse the on-disk fragment, zero model calls
        materializedTasks.push(JSON.parse(fs.readFileSync(fragmentPath, "utf8")));
```

```
        continue;
      }

      // Stream-materialize the single-task fragment
      const detailedTask = await this.pmAgent.elaborateSingleTask(taskStub, outline, brief);
      fs.writeFileSync(fragmentPath, JSON.stringify(detailedTask, null, 2));
      materializedTasks.push(detailedTask);
    }

    return new TaskQueue(materializedTasks);
  }
}
```

---

9. Epistemic Status of Quantitative Claims

Supplementary measurement result (2026-10-21, offline analytical accounting, zero quota): script `experiments/attention_growth/e_p06_resume_savings.py`, results in `e_p06_resume_savings.json`.

Caliber (key point): what is actually saved is **no longer re-entering the breakpoint** — the historical cost of already-passed tasks has been paid and is not refunded; resume only re-executes the failed task and its successors. Let task costs be $c_1, \ldots, c_N$ and the breakpoint position be k, then

$$s = \frac{\sum_{i<k} c_i}{\sum_{i \le N} c_i}$$

i.e., the **cost-weighted prefix fraction**; under equal costs it degenerates to $(k-1)/N$.

Cost profile	Expected savings at $N = 32$	Breakpoint position required to reach 70%
Equal cost	48.4%	$k/N \geq 0.75$
Linear front-loaded (early-heavy)	64.6%	$k/N \geq 0.50$
Exponential front-loaded $r = 1.2$	**81.5%**	$k/N \geq 0.25$
Exponential front-loaded $r = 1.5$	90.6%	$k/N \geq 0.125$
Linear back-loaded (late-heavy)	32.3%	$k/N \geq 0.875$

Ruling: 70% is not a constant independent of cost structure, but the result of a cost-weighted position. When costs are front-loaded (blueprint / scaffold / major changes dominate, $r \gtrsim 1.2$), a mid-segment breakpoint already recovers more than 70% — consistent with measured breakpoint-retry results in external engineering systems. The property that **historical overhead is not refunded** is carried by the formula itself: s counts only the prefix and does not deduct paid costs.

Therefore the main text should rewrite “saving more than 70%” into a conditional statement: “under a front-loaded cost decomposition structure, a mid-segment breakpoint already recovers more than 70%; under an equal-cost structure the expectation is about 50%, requiring the breakpoint to lie in the last 25%.” This conclusion is an analytical identity, independent of model calls. The empirical source is external engineering practice (not reproduced in this repository), serving only as an empirical fact to be explained, and does not constitute a reasoning premise of this system.

Chapter 10: Pattern 07 — Driver-Navigator Pairing Pattern

---

1. Intent

Drawing on the pairing programming idea from Extreme Programming (XP), a frequency-synchronized micro-loop self-healing closed loop composed of a **Driver Agent** and a **Navigator Agent** is constructed inside a controlled sandbox, realizing "in-place mutation — immediate sniffing — local convergence," so as to prevent an agent acting alone from falling into a self-healing dead loop or blindly enlarging its damage surface.

---

2. Motivation & Real-World Failure Modes

When a single agent performs code mutation and compilation repair, it is extremely prone to falling into a **"healing black hole"**:

- **Failure scenario A (fixing one thing breaks eight)**: when fixing a single compile error, a single agent, lacking any global course constraint, follows the compiler's error chain all the way and modifies 8 unrelated files, ultimately causing an avalanche-like deformation of the entire project architecture;
- **Failure scenario B (trapped in a local optimum)**: when repairing unit tests, the agent oscillates back and forth between two mutually conflicting errors (fixing error A breaks error B, and fixing error B breaks error A), and after exhausting its context tokens it still remains at the initial error;
- **Failure scenario C (empty flow and imperceptible hang)**: when encountering a complex exception stack, the driver agent falls into a long period of long-context silence, and the external orchestrator cannot perceive whether it is in a state of effective exploration.

```
flowchart LR
    subgraph PairingMicroLoop["Sandbox Pairing Micro-Loop (Driver-Navigator Micro-Loop)"]
        direction TB
        Navigator["Navigator (read-only review and course lock)"]
        Driver["Driver (mutates code and executes commands)"]
        Substrate["Physical sandbox substrate (Sandbox)"]

        Navigator -- "1. Plan local tactical path & review suggestions" --> Driver
        Driver -- "2. Perform file edits and unit-test commands" --> Substrate
```

```
        Substrate -- "3. Physical build exit code & real compiler stack" --> Navigator
        Navigator -- "4. Physical sniffing (detect dead loops / out-of-bounds mutation / branch c
        Navigator -- "5. Course-correction signal" --> Driver
    end
```

---

3. Applicability

- Implementation phases involving complex compile errors, type-system assertions, or complex business algorithms;
- Development of key core modules that require high fault tolerance and must prevent the code damage surface from escaping;
- The micro-level execution closed loop of a single complex task.

---

4. Participants

1. **Driver Agent**: holds the mutation tools (file writing, command execution), responsible for concretely writing code and running the compiler build; possesses the driving force of execution;
2. **Navigator Agent**: a read-only role that does not write code directly; responsible for monitoring every mutation step of the driver, comparing against the architecture design contract, reviewing code quality in real time, and issuing targeted correction suggestions;
3. **Implicit Sniffer**: a sensor integrated at the bottom layer of the navigator, detecting in real time whether code mutation triggers a dead loop, whether undeclared file writes occur, or whether logical anti-patterns emerge.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Orchestrator as Orchestration layer
    participant Navigator as Navigator (Navigator)
    participant Driver as Driver (Driver)
    participant Compiler as Native compiler / test suite

    Orchestrator->>Navigator: Issue task design slice
    Navigator->>Driver: Issue minimal tactical goal [write the desensitization core filter]

    loop Micro-loop self-healing (at most N rounds of attempts)
        Driver->>Compiler: Write code and trigger compilation
        Compiler-->>Driver: Capture error stack (Exit Code 1)
        Driver->>Navigator: Report current state and proposed repair plan
        Navigator->>Navigator: Sniffing review: is it a known dead loop? Is it out of bounds?

        alt Detects the driver deviating from course (e.g., attempting to modify an already-passi
            Navigator-->>Driver: Hard correction: modifying shared modules is forbidden; refactor
        else Repair direction is correct
```

```
            Navigator-->>Driver: Approve and provide concrete parameter-fix suggestions
        end
    end

    Driver->>Compiler: Final compilation (Exit Code 0)
    Navigator->>Orchestrator: Navigator confirms task completion and endorses it
```

---

6. Consequences

- **Pros**:
  - **Extremely fast self-healing convergence**: the navigator acts as a "real-time external brake pad," compressing the complex exploration space within a local course corridor and avoiding more than 80% of ineffective blind retries;
  - **Double insurance on code quality**: every line of code landed on disk undergoes the navigator's real-time semantic review, reducing rejection frequency in the subsequent global code review phase.
- **Cons**:
  - Multi-agent concurrency or polling increases the number of basic inference calls per task, which must be paired with efficient small-parameter lightweight models in the driver/navigator roles.

---

7. Anti-Patterns to Avoid

Anti-pattern name	Manifestation	Why it must be eliminated
Lone Ranger Blackhole	A single agent modifies code for 10+ consecutive rounds without any reviewer supervising it	Highly prone to severe code deformation, ultimately deviating from the initial architecture
Navigator Role Intrusion	The navigator directly takes over the driver's write tools and writes code itself	Destroys the cognitive separation of "one thinks, the other executes," allowing confirmation bias to grow again

---

8. Algorithmic Formulation

```typescript
interface PairDevSession {
  driver: DriverAgent;
  navigator: NavigatorAgent;
  maxIterations: number;
}

class SandboxPairDevLoop {
  async executePairDev(session: PairDevSession, taskSlice: TaskDesignSlice): Promise<TaskResult>
```

```
    let iteration = 0;
    let currentFeedback: NavigatorGuidance = session.navigator.initialBrief(taskSlice);

    while (iteration < session.maxIterations) {
      iteration++;

      // 1. The driver mutates physical code under the navigator's guidance
      const mutationResult = await session.driver.actuate(currentFeedback);

      // 2. Real native compilation and testing
      const buildResult = await session.driver.runInSituVerify();
      if (buildResult.exitCode === 0) {
        // The navigator performs the final review of code compliance
        const finalAudit = await session.navigator.auditFinalCode(mutationResult.modifiedFiles);
        if (finalAudit.approved) {
          return { success: true, iterations: iteration, codeArtifacts: mutationResult.modifiedFi
        }
      }

      // 3. The navigator performs implicit sniffing and correction on the error stack and mutati
      currentFeedback = await session.navigator.analyzeFailureAndSteer(
        buildResult.errorOutput,
        mutationResult.mutationDiff,
        iteration
      );

      // Dead-loop detection
      if (currentFeedback.isLoopDetected) {
        await session.driver.rollbackToLastKnownStableState();
      }
    }

    return { success: false, reason: "MAX_ITERATIONS_EXCEEDED" };
  }
}
```

---

9. Epistemic Status of Quantitative Claims

Disposition: downgraded to an empirical target (unverified) — the phrase "more than 80% of ineffective blind retries are avoided" currently has no measurable definition for "ineffective blind retries" (the judging predicate for "whether a retry brings information gain" is missing). This figure is marked as an **engineering empirical target**; to elevate it, the information-gain predicate must first be defined and its threshold preregistered.

Chapter 11: Pattern 08 — Implicit Branch Sniffing & Internal Invariants Pattern

---

1. Intent

During the Navigator Agent's real-time code review of the Driver, static abstract syntax tree (AST) analysis and branch-coverage probes are used to **actively sniff the implicit emergent branches produced by the driver during mutation**, and to force the in-place generation of **internal invariants tests** that freeze them under a white-box, preventing the hidden boundary-branch degradation and reward hacking (Amodei et al., 2016) that result from the infeasibility of self-correction without external verification (Huang et al., ICLR 2024).

---

2. Motivation & Real-World Failure Modes

When autonomous agents write code and perform self-healing modifications, they usually focus only on "whether the business mainline can run through the existing use cases," which easily induces serious engineering hazards:

- **Failure scenario A (untested emergent branches)**: When implementing a 2FA authentication filter, the driver introduces a new combined-condition branch `if (token.isExpired() || token.isRevoked()) throw new BadCredentialsException(...)` to handle the `pre_auth_token` verification logic. However, the driver only writes tests for `isExpired()`, leaving `isRevoked()` as an orphan branch guarded by no one in production. Once a later self-healing refactor mistakenly deletes this check, the existing use cases remain 100% green, causing the security defense to fail;
- **Failure scenario B (reward hacking exploiting empty assertions)**: When implementing exception-catching logic, the driver quietly prints only one log line inside `catch (Exception e)`, or directly returns an empty object `return null`, to prevent the test from reporting errors. Since the existing black-box tests make no assertion on the exception type, this behavior cheats the tests into passing (falsified pass), while in substance masking a serious system defect;
- **Failure scenario C (architecture erosion)**: For convenience, the driver degenerates "explicit service-interface injection via constructor (Constructor Injection)," strictly mandated by the architecture design, into "reflection-based private-field injection (Field Injec-

tion),” or exposes an originally package-private class modifier as `public` without authorization, destroying the system's encapsulation.

```
flowchart TD
    subgraph DriverMutation["Driver code mutation (Driver Actuation)"]
        CodeDiff["Driver submits physical code change (Git Diff)"]
    end

    subgraph SniffingEngine["White-box implicit branch sniffer (Sniffing Engine)"]
        direction TB
        AST["1. Static AST traversal: extract all conditional jumps and throw branches"]
        DiffAnalysis["2. Incremental analysis: identify implicit emergent branches newly introduc
        CoverageCrossCheck["3. Cross-check: do existing test cases cover this newly added branch?
    end

    DriverMutation --> SniffingEngine

    CoverageCrossCheck -- "Uncovered implicit branch exists" --> ForceAnchor["Force internal inva

    subgraph ThreeTierTaxonomy["Three-tier test anchoring system (Three-Tier Test Taxonomy)"]
        T1["Tier 1: Acceptance tests (Acceptance living-specs: business contract)"]
        T2["Tier 2: Supplementary coverage tests (Supplementary: explicit business branches)"]
        T3["Tier 3: Internal invariants tests (Internal Invariants: degradation-prevention white-
    end

    ForceAnchor --> ThreeTierTaxonomy
    ThreeTierTaxonomy --> InvariantTests["Generate and execute internal unit tests in place (100%
```

---

3. Applicability

- Critical logic that must not err, such as rigorous security boundaries, permission control, and state-machine transitions;
- Micro-loop environments in which development agents frequently execute self-healing retries;
- Scenarios that require the sedimentation of long-term living specs and the prevention of code degradation into bad smells.

---

4. Participants

1. **Driver Agent**: responsible for proposing the code implementation and preliminary use cases;
2. **Branch Sniffer**: a static code analyzer integrated inside the navigator, responsible for tracking the control-flow graph (CFG) increments produced by mutation;
3. **Test Taxonomy Arbiter**: responsible for routing the newly generated tests:
    - **Acceptance Specs**: satisfy business requirements and may be promoted to living specs in the future;
    - **Supplementary Coverage**: cover normal business branches;

- **Internal Invariants**: establish white-box anchors for defensive programming, single-constructor constraints, and transparent propagation of specific exceptions, forbidding later self-healing operations from deleting or softening them.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Driver as Driver (Driver)
    participant Sniffer as Implicit branch sniffer (Sniffer)
    participant Navigator as Navigator (Navigator)
    participant Substrate as Physical test suite

    Driver->>Sniffer: Submit implementation code and test patch
    Sniffer->>Sniffer: Statically analyze the AST control-flow graph (CFG)
    Sniffer->>Sniffer: Sniffing finds uncovered exception branch: [throw BadCredentialsException

    Sniffer-->>Navigator: Alert: 1 unanchored emergent branch and 1 architecture degradation foun
    Navigator->>Driver: Issue hard constraint: must append internal invariants test testVerifyPro

    Driver->>Substrate: Physically append white-box internal invariants unit test
    Driver->>Substrate: Trigger native run: mvn test -pl specialscale-auth
    Substrate-->>Driver: Unit tests 100% pass (Exit Code 0)

    Driver->>Navigator: Resubmit deliverable (declares new_tests contains internal_invariants)
    Navigator->>Navigator: Confirm white-box closure, approve task delivery
```

---

6. Consequences

- **Pros**:
  - **Eliminate implicit regressions**: all emergent branches are locked in place by physical unit tests; later tasks that mistakenly delete code will immediately trigger a compile failure (Exit 1), achieving physical self-protection;
  - **Guard against reward hacking**: the driver cannot muddle through with empty implementations or exception swallowing;
  - **Architectural structural rigidity**: architectural conventions such as single-constructor dependency injection are mechanically anchored by unit-test code, preventing code corruption.
- **Cons**:
  - Unit-test file line count increases by about 30%–60%;
  - The sniffing process requires introducing a lightweight AST parser, increasing single-round review latency.

---

7. Anti-Patterns to Avoid

Anti-pattern name	Manifestation	Why it must be eliminated
Test Tampering to Pass	The driver directly comments out or deletes existing use cases to make the build green	A typical specification gaming that destroys the system's security dike
Happy-Path Only Fallacy	Only tests the normal 200 return; writes no assertions for all 4xx/5xx and exception throws	Once the system goes online and encounters illegal input, it will directly trigger an uncaught crash

---

8. Algorithmic Formulation

```
interface EmergentBranch {
  location: string;
  conditionDescription: string;
  thrownException?: string;
  isCoveredByExistingTests: boolean;
}

class WhiteboxBranchSniffer {
  sniffBranches(sourceCode: string, testCode: string): EmergentBranch[] {
    const ast = parseJavaAst(sourceCode);
    const branches: EmergentBranch[] = [];

    // 1. Traverse the AST to extract all defensive branches and exception nodes
    traverse(ast, {
      IfStatement(path) {
        if (containsThrowStatement(path.node.consequent)) {
          branches.push({
            location: `Line:${path.node.loc.start.line}`,
            conditionDescription: printSource(path.node.test),
            thrownException: extractExceptionClass(path.node.consequent),
            isCoveredByExistingTests: false
          });
        }
      }
    });

    // 2. Cross-check assertions and mock behaviors in the test code
    for (const b of branches) {
      if (testCode.includes(b.thrownException || "") && testCode.includes(b.conditionDescription)
        b.isCoveredByExistingTests = true;
      }
    }

    return branches.filter(b => !b.isCoveredByExistingTests);
  }
}
```

---

9. Epistemic Status of Quantitative Claims

Disposition: downgraded to a proxy metric (partly re-measurable) — "unit-test file line count increases by about 30%–60%" is a proxy quantity rather than the target quantity: the true target is the **orphan-branch detection rate**. The line-count increase does not constitute a theoretical conclusion; the orphan-branch detection rate can be re-measured by injecting mutants onto the existing use-case set (the lower the mutant survival rate, the higher the detection rate).

Chapter 12: Pattern 09 — Idempotent Journaling & Mtime Touch Pattern

---

1. Intent

Establish a **canonical SHA-256 signed journal (Idempotent Journaling)** and a **physical filesystem clock compensation mechanism (Mtime Touch Compensation)** for every write operation an agent performs on the physical substrate, resolving mutation superposition misalignment during the agent's self-healing retries, and eliminating the false build state caused by modern compiler cache hits (stale cache) due to millisecond-level high-frequency overwrites.

---

2. Motivation & Real-World Failure Modes

In real industrial software engineering, the filesystem and the compiler are strictly causal and time-sensitive:

- **Failure scenario A (retry mutation superposition misalignment)**: an agent invokes the `replace_file_content` tool to repair a line of code; if the first call is interrupted by a network timeout and the agent triggers a retry, the second replacement based on the same line number or text will, because the previous round may have already partially written, cause the target text not to be found, or a double insertion to occur, destroying the code file;
- **Failure scenario B (mtime stagnation causing compiler fake-death)**: when fixing a bug, an agent consumes only 200 milliseconds from reading the file, through analysis, to writing back; the modification time (mtime) of the written file and the previous write fall within the same operating-system timestamp (especially in certain filesystems or packaging tools with second-level timestamp granularity). When `mvn compile` or `webpack` is subsequently triggered, the build tool detects `mtime <= last_build_time` of the file and **directly skips recompilation of that file**, continuing to run the old `.class` bytecode, trapping the agent in the false predicament of "the code was clearly changed but the errors are unchanged";
- **Failure scenario C (empty flow and implicit state loss)**: after an agent performs a write and encounters an exception, it cannot accurately trace back to which historical snapshot the physical disk actually stays at, leaving the self-healing rollback operation with no starting point.

```
flowchart TD
    subgraph AntiPattern["Anti-pattern: bare filesystem write (Raw File Mutation)"]
        direction TB
        AgentRaw["Agent directly calls the write tool"] --> DiskWrite["Disk overwrite (50ms)"]
        DiskWrite --> SameMtime["File mtime does not make a discrete jump (same as last time)"]
        SameMtime --> CompilerStale["Compiler hits the cache and skips recompilation!"]
        CompilerStale --> StaleFailure["Continues running old bytecode, false-error dead loop"]
    end

    subgraph CyberneticPattern["Cybernetic pattern: idempotent journaling + mtime compensation (J
        direction TB
        AgentSafe["Agent submits mutation action"] --> Hasher["Compute canonical SHA-256 signatur
        Hasher --> Journal["Write idempotent journal (atomic undo/replay)"]
        Journal --> AtomicWrite["Atomic write to disk (Sub-stepping)"]
        AtomicWrite --> MtimeTouch["Force physical mtime compensation: touch -m +1s"]
        MtimeTouch --> ClockAdvance["File clock strictly monotonically increasing (Strict Monoton
        ClockAdvance --> CompilerWake["Forcefully penetrate the compiler cache, true recompilatio
    end
```

---

3. Applicability

- Any modern compiled-language project with an incremental build cache (Java, C++, Rust, Go, TypeScript);
- Involving automatic retry mechanisms or multi-step complex self-healing loops;
- Where absolutely reliable physical state snapshots and rollback support are required.

---

4. Participants

1. **Mutation Journal Manager**: computes a canonical content hash and target-file context fingerprint for every write operation, maintaining a forward replay and backward rollback stack;
2. **Atomic Sub-stepper**: decomposes write operations into atomic sub-steps (write to a temporary file, atomic rename move), preventing file corruption caused by half-writes;
3. **Mtime Touch Compensator**: after the write is complete, directly calls low-level operating-system syscalls (such as `fs.utimes` or `touch`), explicitly advancing the target file's `mtime` to the current clock plus 1 second, breaking the incremental compiler's cache-hit condition.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Driver as Driver agent
    participant Journal as Mutation journaler (Journal)
    participant FS as Physical filesystem
    participant Touch as Mtime compensator
```

```
participant Toolchain as Incremental compilation toolchain

Driver->>Journal: Submit code change (targetFile, replacement)
Journal->>Journal: Compute action canonical SHA-256: Hash(File + Target + Content)

alt A mutation with the same signature has already landed successfully (idempotent hit)
    Journal-->>Driver: Directly return success (avoid double replacement corrupting the text)
else New valid mutation
    Journal->>FS: Atomically write and persist to mutation journal journal.jsonl
    Journal->>FS: Land the business code file on disk
    Journal->>Touch: Trigger clock causal advance (Advance Physical Timestamp)
    Touch->>FS: Syscall: utimesSync(file, now + 1000ms)
    Touch-->>Driver: Mutation effective and clock-advance confirmation
end

Driver->>Toolchain: Trigger incremental compilation
Toolchain->>FS: Check targetFile.mtime > last_compile.mtime?
FS-->>Toolchain: Return an absolutely monotonically increasing new timestamp
Toolchain->>Toolchain: Forcefully re-parse the AST and generate new artifacts (Cache Busted!)
```

---

6. Consequences

- **Pros**:
  - **Eliminate compiler ghost errors**: ensure every code modification triggers physical compilation 100%, ending the debugging nightmare of "the code changed but the bytecode did not";
  - **State replay safely idempotent**: when a network interruption or an agent's mistaken retry trigger occurs, the target source syntax will absolutely not be corrupted.
- **Cons**:
  - Introduces slight filesystem syscall overhead (a few milliseconds per step);
  - Requires system privileges for modifying file metadata in the sandbox environment.

---

7. Anti-Patterns to Avoid

Anti-pattern name	Manifestation	Why it must be eliminated
Raw String Replacement	Directly performing non-idempotent regex replacement on the original file	On retry, since the original characters have already been replaced, the second retry inevitably throws an "anchor not found" exception
Blind System Clock Trust	Assuming the compiler will naturally perceive the file after it is written	Under modern multi-core and high-frequency mutation, mtime staying within the same second is extremely common, causing incremental compilation to miss files

---

8. Algorithmic Formulation

```typescript
import * as fs from "fs";
import * as crypto from "crypto";

interface MutationEntry {
  journalId: string;
  filePath: string;
  sha256: string;
  appliedTimestamp: number;
}

class IdempotentSubstrateMutator {
  private journal: Map<string, MutationEntry> = new Map();

  async applyMutation(filePath: string, originalContent: string, newContent: string): Promise<boo
    // 1. Compute the mutation fingerprint (Canonical SHA-256)
    const signature = crypto.createHash("sha256")
      .update(filePath)
      .update(originalContent)
      .update(newContent)
      .digest("hex");

    if (this.journal.has(signature)) {
      // Idempotent defense: an exactly identical mutation has already been executed; skip safely
      return true;
    }

    // 2. Atomically write the physical file
    const tmpPath = `${filePath}.tmp.${Date.now()}`;
    await fs.promises.writeFile(tmpPath, newContent, "utf8");
    await fs.promises.rename(tmpPath, filePath);

    // 3. Physical mtime compensation (force timestamp monotonic increase, penetrate the compile
    const compensatedTime = (Date.now() + 1000) / 1000;
    fs.utimesSync(filePath, compensatedTime, compensatedTime);

    // 4. Record into the persistent journal
    this.journal.set(signature, {
      journalId: signature,
      filePath,
      sha256: signature,
      appliedTimestamp: Date.now()
    });

    return true;
  }
```

```
}
```

Chapter 13: Pattern 10 — Substrate Lease Lock & Watchdog Pattern

---

1. Intent

In multi-peer-agent (Peer Agents) or heterogeneous extended-agent collaboration scenarios, an exclusive write-lock mechanism based on a lease protocol (Lease Protocol), heartbeat watchdog, and state tamper-proofing verification is established for shared resources of the physical substrate, forcibly degrading agents not holding the lock to primary-writer replica-reader, preventing concurrent code write-write conflicts and deadlocks.

---

2. Motivation & Real-World Failure Modes

When an agent system evolves from a monolith into multi-agent concurrent coordination (for example, a pipeline primary agent working concurrently with an IDE-side auxiliary agent, or a code-generation agent with a concurrent unit-test completion agent), traditional lock-free designs inevitably collapse:

- **Failure scenario A (write-write contention causing code tearing)**: two agents edit the same Java class at nearly the same moment concurrently, causing unresolved physical conflict markers (`<<<<<<< HEAD`) to appear in the Git working tree, after which neither side can continue to resolve the syntax;
- **Failure scenario B (deadlock and orphan locks)**: the agent holding the write lock terminates unexpectedly due to model generation timeout, network interruption, or process crash; but because the lock has no timeout renewal mechanism, the entire project is permanently locked (Deadlock), and all subsequent pipeline tasks hang forever;
- **Failure scenario C (unauthorized surreptitious run)**: a secondary agent (such as an auxiliary Q&A agent) directly modifies the code being compiled without authorization, destroying the purity of the environment the primary agent is verifying.

```
flowchart TD
    subgraph MultiAgentCoordination["Concurrent contention of multi-peer agents"]
        AgentA["Primary agent (Primary Agent A)"]
        AgentB["Auxiliary agent (Peer Agent B)"]
    end

    MultiAgentCoordination --> LeaseArbiter["Substrate lease arbiter (Substrate Lease Arbiter)"]
```

```
    subgraph ArbiterCore["Lease arbitration core (Lease & Watchdog)"]
        LockFile["Substrate lease contract (.substrate-lock.json)"]
        Watchdog["Heartbeat watchdog (Heartbeat TTL: 30s)"]
    end

    LeaseArbiter --> LockFile
    LeaseArbiter --> Watchdog

    AgentA -- "Request exclusive write lock" --> LeaseArbiter
    LeaseArbiter -- "Write lease granted successfully (Leader)" --> AgentA
    AgentA -- "Periodic heartbeat renewal (Heartbeat)" --> Watchdog

    AgentB -- "Attempt to write file" --> LeaseArbiter
    LeaseArbiter -- "Lease already held -> forced degradation" --> ReplicaMode["Replica mode (Rep
    ReplicaMode --> AgentB
```

---

3. Applicability

- Scenarios where multiple autonomous agents share the same code repository or physical workspace;
- Environments where IDE client plugins (such as Cline, Trae, Cursor) collaborate concurrently with background autonomous pipelines;
- Systems where long-cycle tasks and asynchronous background self-healing tasks run in parallel.

---

4. Participants

1. **Lease Arbiter**: an arbitration entity based on file locks or atomic distributed primitives, responsible for lock issuance, renewal, and release;
2. **Lease Contract**: metadata strictly constrained by a machine Schema (including holder ID, holder role, grant timestamp, TTL expiry milliseconds, and verification fingerprint);
3. **Heartbeat Watchdog**: a background asynchronous inspector that, upon detecting that the holder has not sent a heartbeat within the TTL due to a crash, automatically and safely reclaims the lease (Safe Eviction);
4. **Replica-Reader Interceptor**: intercepts write-mutation syscalls from non-lock-holders, converting them into read-only analysis views or in-memory shadow simulation, and forbidding landing on disk.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor AgentPrimary as Primary agent (Primary)
    actor AgentPeer as Auxiliary agent (Peer)
    participant Arbiter as Lease arbiter (Arbiter)
    participant LockState as Physical lease contract (.substrate-lock.json)
```

```
participant FS as Physical filesystem

AgentPrimary->>Arbiter: acquireLock(holderId="agent-primary", ttl=30000)
Arbiter->>LockState: Atomically create the lease file
Arbiter-->>AgentPrimary: Obtain exclusive write lock (LEASE_ACQUIRED)

par Primary agent periodic renewal
    loop Every 10 seconds
        AgentPrimary->>Arbiter: heartbeat(leaseId, extendMs=30000)
        Arbiter->>LockState: Refresh the expiresAt timestamp
    end
and Auxiliary agent attempts concurrent write
    AgentPeer->>Arbiter: tryWrite(file="GatewayFilter.java")
    Arbiter->>LockState: Read the lease state
    LockState-->>Arbiter: Lease valid and holder != agent-peer
    Arbiter-->>AgentPeer: Degradation interception: EBUSY_SUBSTRATE_LOCKED (degraded to read-
end

AgentPrimary->>Arbiter: releaseLock(leaseId)
Arbiter->>LockState: Atomically delete / mark unlocked
Arbiter-->>AgentPeer: Broadcast availability event: SUBSTRATE_RELEASED
```

---

6. Consequences

- **Pros**:
  - **Absolute consistency of the physical substrate**: eliminates code tearing and Git conflicts caused by multi-agent concurrent writes (lease mutual exclusion is mechanized in the kernel, Appendix B.4);
  - **Automatic fault immunity and self-healing**: relying on the TTL watchdog mechanism, the unexpected crash of any single-point agent will not deadlock the entire project.
- **Cons**:
  - Requires implementing a periodic heartbeat background coroutine (Goroutine/Timer) at the agent SDK layer;
  - Secondary agents incur a certain queuing latency while waiting for the exclusive lock.

---

7. Anti-Patterns to Avoid

Anti-pattern name	Manifestation	Why it must be eliminated
Indefinite Mutex Trap	Creates the lock only on entry and deletes it only on exit, with no TTL set	Once the agent process is killed by OOM, residual lock files will cause a permanent system deadlock

Anti-pattern name	Manifestation	Why it must be eliminated
Optimistic Blind Mutation	Allows multiple agents to freely overwrite, relying on Git merge afterward	LLMs have extremely poor ability to handle Git conflict markers and easily destroy valid code

---

8. Algorithmic Formulation

```typescript
interface SubstrateLease {
  leaseId: string;
  holderId: string;
  role: string;
  grantedAt: number;
  expiresAt: number;
  checksum: string;
}

class SubstrateLockManager {
  private readonly lockFilePath = ".substrate-lock.json";
  private readonly defaultTtlMs = 30000;

  acquireLock(holderId: string, role: string): boolean {
    const now = Date.now();
    if (fs.existsSync(this.lockFilePath)) {
      const currentLease: SubstrateLease = JSON.parse(fs.readFileSync(this.lockFilePath, "utf8"))
      // Check whether the lease is still valid
      if (now < currentLease.expiresAt && currentLease.holderId !== holderId) {
        return false; // Held by someone else; acquisition failed
      }
    }

    // Issue a new lease
    const newLease: SubstrateLease = {
      leaseId: crypto.randomUUID(),
      holderId,
      role,
      grantedAt: now,
      expiresAt: now + this.defaultTtlMs,
      checksum: this.computeChecksum(holderId, now)
    };

    fs.writeFileSync(this.lockFilePath, JSON.stringify(newLease, null, 2), "utf8");
    return true;
  }

  heartbeat(leaseId: string, holderId: string): boolean {
    if (!fs.existsSync(this.lockFilePath)) return false;
```

```
    const lease: SubstrateLease = JSON.parse(fs.readFileSync(this.lockFilePath, "utf8"));
    if (lease.leaseId === leaseId && lease.holderId === holderId) {
      lease.expiresAt = Date.now() + this.defaultTtlMs;
      fs.writeFileSync(this.lockFilePath, JSON.stringify(lease, null, 2), "utf8");
      return true;
    }
    return false;
  }
}
```

Chapter 14: Pattern 11 — Primary-Writer Replica-Reader Pattern

---

1. Intent

In a distributed-system environment of collaborative evolution among multi-peer agents (Peer Agents), auxiliary agents not holding the exclusive write lease (Lease, Gray & Cheriton, SOSP 1989) are **forcibly degraded into read-only analysis replica nodes (Replica-Reader)** according to causal ordering and substrate lease state. Through atomic interception of physical write calls and redirection to an in-memory shadow scratchpad, code tearing and Git working-tree deadlock caused by concurrent blind mutation are eliminated.

---

2. Motivation & Real-World Failure Modes

As multi-agent collaboration architectures evolve from the traditional "monolithic pipeline" toward "multi-end peer collaboration" (for example, developers running auxiliary extensions inside the IDE, CI/CD running autonomous pipelines, and code-review agents concurrently issuing suggestions), multi-agent simultaneous state writes to the same workspace become an inescapable physical reality:

- **Failure scenario A (write-write conflict causing code tearing, split-brain code tearing)**: While the primary pipeline is running unit-test compilation of the gateway module, the IDE-side auxiliary agent — without knowledge of the underlying build-lock state — responds to the user's interactive instruction by directly modifying the module's POM file or the same Java class, causing conflict markers in the underlying Git and the executing compilation to crash instantly;
- **Failure scenario B (concurrent contamination of environment purity, substrate contamination)**: The auxiliary agent attempts to "self-heal code" in the background and unauthorizedly executes `git reset --hard` or `mvn clean`, directly erasing the temporary bytecode or intermediate artifacts just written but not yet committed by the main pipeline;
- **Failure scenario C (brutal interruption imposing user cognitive burden, lock-out starvation)**: If the non-lock-holding agent process is simply and brutally killed, human-agent interaction continuity is destroyed, and the developer encounters abrupt error popups in the terminal or IDE interface.

```
flowchart TD
    subgraph MultiAgentRequests["Multi-agent concurrent syscalls"]
        AgentPrimary["Primary pipeline agent (holds exclusive write lease, Leader)"]
        AgentPeer["IDE/background auxiliary agent (no lock, Follower)"]
    end

    MultiAgentRequests --> Interceptor["Substrate lease interception gateway (Substrate Lease Int

    Interceptor -- "Holds a valid write lease" --> DirectWrite["Direct mutation of the physical s

    subgraph ReplicaReaderDegradation["Replica-Reader degradation system (Replica-Reader Fallback
        direction TB
        ReadOnlyView["1. Read-only code retrieval and semantic understanding (Read-Only Analysis)
        ShadowScratch["2. In-memory shadow workspace (Shadow In-Memory Workspace)"]
        DiffProposal["3. Generate isolated diff patch (Patch Proposal, not landed on disk)"]
    end

    Interceptor -- "No lock / lease conflict -> forced degradation" --> ReplicaReaderDegradation
    DirectWrite --> SubstrateFS["Real physical filesystem (repo/)"]
```

---

3. Applicability

- Multiple agents from different vendors or processes (such as self-built CI pipelines and various IDE coding agents) connecting to the same enterprise engineering repository;
- Scenarios with clear primary-standby responsibility division (Leader-Follower) requiring absolute purity of the primary pipeline build;
- Collaboration architectures that allow replica nodes to propose code suggestions but strictly forbid them from landing on disk privately.

---

4. Participants

1. **Primary-Writer**: the agent that has successfully obtained the exclusive write right (Leader) through the distributed substrate lease lock (Pattern 10), possessing the privileges of directly modifying the physical filesystem and native building;
2. **Replica-Reader**: the peer agent that did not win the write lease; its write permission is transparently stripped by the interception gateway, and it is automatically degraded into an observer and suggester;
3. **Lease Degradation Interceptor**: middleware located between operating-system syscalls (FS Tool Calls) and the physical disk, responsible for authentication and enforcing degradation;
4. **Shadow Scratchpad**: a stateless temporary memory or isolated branch opened for the replica reader, allowing it to rehearse code patches in virtual space without ever contaminating the physical mainline.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Primary as Primary agent (holds lease)
    actor Peer as Auxiliary agent (replica role)
    participant Interceptor as Lease interception middleware
    participant Lock as Lease contract (.substrate-lock.json)
    participant Substrate as Physical filesystem
    participant Shadow as Shadow patch buffer pool

    Primary->>Lock: Maintain the exclusive lease (Lease Active)

    par Primary agent works normally
        Primary->>Interceptor: write_to_file("TwoFactorAuthEndpoint.java", content)
        Interceptor->>Lock: Verify lease ownership (Primary matched)
        Interceptor->>Substrate: Physically write the source file
        Interceptor-->>Primary: Write succeeded (physically effective)
    and Auxiliary agent concurrently attempts mutation
        Peer->>Interceptor: replace_file_content("TwoFactorAuthEndpoint.java", patch)
        Interceptor->>Lock: Verify lease ownership (Peer holds no lock)
        note over Interceptor,Peer: Trigger primary-writer replica-reader degradation (Downgrade
        Interceptor->>Shadow: Write the change into the virtual shadow diff pool (patch.diff)
        Interceptor-->>Peer: Return soft success: prompt "substrate is occupied; the code has bee
    end

    Primary->>Lock: Release the exclusive write lease (Release)
    Lock-->>Interceptor: Broadcast substrate idle event (SUBSTRATE_UNLOCKED)
    Interceptor->>Peer: Approve requesting atomic confluence from the shadow buffer pool into the
```

---

6. Consequences

- **Pros**:
  - **Zero contamination of the physical substrate**: avoids Git index corruption, double file modification, and compile fake-death caused by multi-agent contention;
  - **Continuous, uninterrupted user experience**: the non-lock-holding agent does not crash directly, but continues to provide interactive support in the form of read-only or suggested patches;
  - **Compliance with the classic distributed causal law (Lamport, 1978)**: write operations have a globally unique total-order causal arrangement.
- **Cons**:
  - Requires virtualizing file write operations at the platform layer (shadow patch mechanism);
  - Patches produced by replica nodes require an additional confluence approval process.

---

7. Anti-Patterns to Avoid

Anti-pattern name	Manifestation	Why it must be eliminated
Blind Lockless Mutation	Allows multiple agents to freely contend for writing to the same workspace	Inevitably causes code tearing and compiler-state disorder
Hard Kill Starvation	Detects a non-lock-holder's write operation and directly force-kills the process	Destroys the human-agent interaction experience and causes unpredictable data loss

---

8. Algorithmic Formulation

```typescript
interface WriteRequest {
  agentId: string;
  filePath: string;
  newContent: string;
}

class PrimaryWriterReplicaReaderInterceptor {
  private leaseManager: SubstrateLeaseManager;
  private shadowBuffer: Map<string, string> = new Map();

  async interceptWrite(req: WriteRequest): Promise<WriteVerdict> {
    const isPrimary = this.leaseManager.hasExclusiveLease(req.agentId);

    if (isPrimary) {
      // 1. Primary agent: physical landing on disk
      await fs.promises.writeFile(req.filePath, req.newContent, "utf8");
      return { success: true, mode: "PHYSICAL_DIRECT", message: "Written to substrate" };
    }

    // 2. Replica node: forced degradation to shadow buffering
    const shadowKey = `${req.agentId}:${req.filePath}`;
    this.shadowBuffer.set(shadowKey, req.newContent);

    return {
      success: true,
      mode: "SHADOW_REPLICA_BUFFERED",
      message: "Substrate locked by primary agent. Changes preserved as offline proposal patch."
    };
  }
}
```

---

9. Epistemic Status and Formal Anchors

Theoremizable (proven, zero sorry): the core concurrency claim of this pattern has been verified in the Lean 4 kernel, module `Pattern11_PrimaryWriter` (for-

`mal_proof/IACFormalization`), corresponding to Appendix B.9.

- The single-writer premise is not an additional convention: `writer_unique_of_lease` is directly derived from Pattern 10's `canWrite_unique` — at any instant, at most one agent holds write permission.
- Write journal layer: `eq_of_mem_of_tick_eq` proves that within a legal journal, the same tick is the same event; `writer_unique_per_tick` gives "same tick, same writer," i.e., concurrent write-write conflict is impossible within the protocol.
- Read consistency: `view_mono` and `no_lost_write` prove that the replica reader's view is a **complete prefix** — a later read contains all events of an earlier read, and any committed event is necessarily visible; there exists no view that "skips intermediate commits."

Falsifiability metric for the corollary: after the primary writer commits, the set of events read by the replica reader must equal the journal prefix "tick $\leq$ read time"; if missing or out-of-order events are observed, then the theorem's premises (strictly monotonic clock, lease mutual exclusion) do not hold in the implementation.

Chapter 15: Pattern 12 — Peer-Agent Protocol Bridge Pattern

---

1. Intent

Through a standardized Model Context Protocol (MCP) and a Peer-Agent Event Bus, establish **asymmetric capability negotiation, bidirectional context projection, and causal state relay** among heterogeneous agent runtimes (such as autonomous background pipelines, IDE client plugins, and code-review bots), breaking down the data silos and semantic barriers of vendor-specific agent ecosystems.

---

2. Motivation & Real-World Failure Modes

In modern software-engineering practice, an enterprise can never rely solely on a single vendor's closed agent product. A typical production environment is heterogeneous by construction: an in-house end-to-end SDLC orchestration pipeline runs in the background, front-end developers use plugins based on MCP or custom protocols inside their IDEs (such as Trae, Cline, Cursor), and third-party security-audit bots are attached externally.

When these **heterogeneous agents** attempt to collaborate on the same code substrate, the following failures typically arise:

- **Failure Mode A (Semantic Tower of Babel)**: Different agents follow different internal state-machine definitions (e.g., the pipeline marks the current state as `pm_gate_passed`, while the IDE client only understands a simple `review_pending`); the absence of a semantic translation layer prevents each side from understanding the other's progress intent;
- **Failure Mode B (Blind Full Context Hard-Copy Triggers Context Rot)**: To brief the IDE-side agent on the background pipeline, developers cram hundreds of pages of background logs and full gate reports wholesale into MCP resources — directly causing severe context rot in the IDE agent (Liu et al., 2024) and destroying its basic code-analysis capability;
- **Failure Mode C (Event Storm & Race Conditions)**: Multiple agents watch workspace file changes through unordered polling; one agent's file modification triggers another agent's automatic build, which in turn triggers a third agent's Git commit, and the system sinks into an infinite loop of event storms.

```
flowchart TD
    subgraph HeterogeneousAgents["Heterogeneous Peer Agents"]
```

```
        AutonomousSDLC["Autonomous SDLC Pipeline"]
        IDEClient["IDE Client Agent (Trae / Cline / Cursor)"]
        SecurityBot["Independent Code-Audit Bot (Security Auditor)"]
    end

    HeterogeneousAgents <--> Bridge["Peer-Agent Protocol Bridge"]

    subgraph BridgeCore["Bridge Core Mechanics"]
        direction TB
        MCP["1. Standard Model Context Protocol (MCP Server/Tools)"]
        SchemaAdapter["2. Semantic Dialect Adapter (Dialect Schema Normalizer)"]
        EventThrottle["3. Causal Event Throttling & Arbitration (Causal Event Throttling)"]
        Projection["4. Task-Level Context PoLP Projection"]
    end

    Bridge --> BridgeCore
    BridgeCore <--> SharedSubstrate["Shared Substrate & Lease State"]
```

---

3. Applicability

- Scenarios where an enterprise needs to connect an autonomous pipeline with developers' desktop IDE programming assistants;
- Multi-agent heterogeneous architectures that mix different large language models (such as Claude, Gemini, DeepSeek, and locally deployed private models);
- Systems that need to standardize the exposure of engineering progress, the evidence chain (Evidence), and gate-scoring results to third-party audit tools.

---

4. Participants

1. **Protocol Bridge Host**: runs as an MCP server or lightweight gateway, exposing a unified standard tool set to external peer agents (e.g., `devforge_record_evidence`, `devforge_score_jev`);
2. **Dialect Adapter**: converts external agents' private-contract requests into the pipeline's core specification (e.g., normalizing task lists of arbitrary formats into the standard `TaskOutline`);
3. **Causal Event Bus**: broadcasts key milestone events (`PHASE_TRANSITION`, `SUBSTRATE_LOCKED`, `TESTS_RESOLVED`) in Lamport causal order, blocking cyclic triggering;
4. **Projection Filter**: follows the Principle of Least Privilege (PoLP), delivering only a summarized slice of the currently focused task to external peer agents, preventing context rot.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Pipeline as Autonomous SDLC Pipeline
```

```
    participant Bridge as MCP Protocol Bridge
    actor IDE as Developer IDE Plugin (Peer)
    participant Lock as Substrate Lease Lock

    Pipeline->>Bridge: Register current task [T2: 2FA controller implementation] & broadcast even
    Bridge->>IDE: Deliver PoLP-projected micro-task slice via MCP Server

    note over IDE: Developer requests a performance review of the T2 code in the IDE
    IDE->>Bridge: Call MCP tool: devforge_query_context(taskId="T2")
    Bridge-->>IDE: Return contract summary and test-assertion standards (filtering out unrelated

    IDE->>Bridge: Attempt code modification: mcp_apply_patch(...)
    Bridge->>Lock: Check physical substrate lease state

    alt Substrate currently held by Pipeline's exclusive write lock
        Bridge-->>IDE: Intercept and degrade: return proposed-patch ID (avoiding write-write conf
    else Substrate idle
        Bridge->>Lock: Help apply for a temporary write lease and execute physically
        Bridge-->>IDE: Physical write succeeded
    end
```

---

6. Consequences

- **Pros**:
  - **Cross-vendor decoupled interop**: any agent supporting standard MCP or generic Tool Calling can join the collaboration seamlessly, without refactoring the underlying architecture;
  - **Seamless ecosystem reuse**: lightweight desktop IDEs can directly borrow the heavyweight background pipeline's Jev gate, physical compiler, and test-sandbox capabilities;
  - **Causal consistency assurance**: calms the file-event storms caused by blind multi-agent concurrency.
- **Cons**:
  - Requires maintaining protocol translation and serialization overhead across toolchains;
  - Relies on network RPC or local IPC inter-process communication channels.

6.1 Typical Extension Scenario: Asynchronous Result Backflow from Isolated Environments (Stress-Test Feedback Loop)

When the stress-test environment is fully isolated from the development environment, the protocol bridge takes on the key role of **asynchronous result backflow**:

```
sequenceDiagram
    autonumber
    actor DevEnv as Development environment (accessible to agents)
    participant Bridge as MCP Protocol Bridge
```

```
actor StressEnv as Stress-test environment (inaccessible to agents)
participant Auditor as Reality Auditor (Pattern 02)

DevEnv->>Bridge: Submit candidate-version Git branch reference
Bridge->>StressEnv: Trigger stress test (through isolated API gateway)
note over StressEnv: Executed independently: load injection, metrics collection, result stora
StressEnv-->>Bridge: Asynchronously backflow raw results (JMeter/Prometheus export)
Bridge->>Auditor: Forward results + transport-integrity verification (signature/hash)
Auditor-->>Bridge: Audit pass/veto (Pattern 02 probe 4)
Bridge-->>DevEnv: Deliver audited stress-test report (PoLP projection)
```

Refined responsibilities of the protocol bridge in this scenario:

1. **Result format normalization**: convert the stress-test environment's native outputs (JMeter XML, Gatling HTML, Prometheus metrics) into the standard `StressTestReport` contract;
2. **Transport integrity assurance**: signature verification and hash checking on the result backflow channel, preventing selective reporting or tampering;
3. **Environment fingerprint collection**: synchronously collect the stress-test environment's configuration fingerprint (OS/kernel/dependency versions/resource quotas) for the auditor's homogeneity check;
4. **Event throttling & causal ordering**: stress-test results are broadcast as `STRESS_TEST_COMPLETE` events in Lamport causal order, ensuring the development environment's state machine updates its reference clock correctly.

   Theoretical anchor: an extension of Axiom III — stress-test results act as physical clock events that drive the update of the next mutation's reference clock R.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be eradicated
Proprietary Silo Syndrome	Every agent invents its own proprietary RPC interface or private data format	Causes tool-integration costs to explode quadratically as $O(N^2)$
Unfiltered Broadcast Flooding	Pushes the pipeline's ten-thousand-line compile console stream indiscriminately to the IDE agent	Induces severe context rot, paralyzing the assistant agent's reasoning

---

8. Algorithmic Formulation

```typescript
interface McpToolDefinition {
  name: string;
  description: string;
  parameters: object;
}
```

```
class PeerAgentProtocolBridge {
  private mcpServer: McpServer;
  private leaseManager: SubstrateLeaseManager;

  registerStandardTools(): void {
    // 1. Expose standard read-only contract lookup tools (with PoLP projection built in)
    this.mcpServer.tool(
      "devforge_get_task_slice",
      " 获取当前子任务的最小特权架构设计切片",
      { taskId: { type: "string" } },
      async ({ taskId }) => {
        const fullDesign = await this.readGlobalDesign();
        const slice = TaskLevelContextProjector.project(taskId, fullDesign);
        return { content: [{ type: "text", text: JSON.stringify(slice) }] };
      }
    );

    // 2. Expose safe collaborative code-patch tools (with lease mutual exclusion built in)
    this.mcpServer.tool(
      "devforge_propose_patch",
      " 向当前工程提交代码变更建议 (受底座租约锁保护)",
      { patchDiff: { type: "string" }, rationale: { type: "string" } },
      async ({ patchDiff, rationale }) => {
        if (this.leaseManager.isLockedByOther("peer-ide-agent")) {
          const proposalId = await this.saveToShadowBuffer(patchDiff, rationale);
          return { content: [{ type: "text", text: `Substrate occupied. Buffered as proposal ${pr
        }
        await this.applyPhysicalPatch(patchDiff);
        return { content: [{ type: "text", text: "Patch successfully applied to physical substrat
      }
    );
  }
}
```

Chapter 16: Pattern 13 — Dual-Mode Technical Crystallization Pattern

---

1. Intent

Subject the experience an agent acquires over the software-engineering lifecycle to a strict **Single-Source Orthogonal Decoupling**, separating it into **Substrate Facts** about the target project's physical reality and **Transferable Patterns** that generalize across projects, thereby preventing environment-specific parameters (such as ports, paths, and version numbers) from becoming interwoven with general algorithmic logic and causing memory poisoning (Memory Poisoning, MemoryGraft, 2025) and model collapse (Model Collapse, Shumailov et al., *Nature* 2024).

---

2. Motivation & Real-World Failure Modes

In many systems that introduce "lifelong learning" or "memory augmentation" into agents (e.g., the Voyager skill library, Wang et al., 2023), experience-distillation mechanisms typically adopt indiscriminate plain-text concatenation or vectorized storage. In large-scale software engineering this readily triggers severe knowledge degradation:

- **Failure Mode A (Leakage of Ephemeral Facts as Universal Experience)**: When the agent was solving the `pigx__gw__004` gateway task, the local sandbox's gateway port was randomly mapped to `19999`, and the agent stored "the gateway port must be configured as 19999" as a "universal best practice" in the global pattern library. Later, when another task (e.g., the Auth authentication service) retrieved knowledge, it mistakenly adopted this entry, causing a hard-coded conflict in the service's startup port;
- **Failure Mode B (Tainted Generalization of Temporary Patches)**: While troubleshooting a dependency conflict, the agent temporarily excluded (exclude) a low-level transitive dependency in a sub-POM. This ad hoc action was indiscriminately generalized into "whenever this library is introduced, the underlying dependency must be excluded", causing subsequent microservice modules to fail with `NoClassDefFoundError` due to missing base classes;
- **Failure Mode C (Non-Canonical Knowledge Bloat and Conflicts)**: The same fact (e.g., the GroupId of a common microservice base module) was repeatedly distilled by different task agents, in different phrasings, into dozens of fragments; lacking a canonical

single-source constraint, the knowledge base fragmented and swelled rapidly, and retrieval-matching accuracy degraded quickly.

```
flowchart TD
    RawDevLogs["Raw logs, fix patches, and tool feedback produced during development"] --> Extrac

    subgraph DualCrystallization["Dual-Mode Orthogonal Decoupling Crystallization"]
        direction TB
        Mode1["Mode 1: Substrate Fact<br/><i>Scope: bound to a specific repository, must not spil
        Mode2["Mode 2: Transferable Pattern<br/><i>Scope: reusable across tasks/projects<br/>Cont
    end

    Extractor --> DualCrystallization

    Mode1 --> FactVault["Repository Fact Base (Repo-Scoped)"]
    Mode2 --> PatternVault["Pattern Draft Vault (DRAFT_SUSPECT)"]

    PatternVault --> PromotionGate["Double-blind final review via the Pattern 14 quarantined prom
```

---

3. Applicability

- Autonomous R&D pipeline platforms that batch-process multi-repository, multi-tech-stack tasks;
- Scenarios that require safely sharing code patterns across projects while eliminating cross-contamination of environment parameters;
- Knowledge-management systems with self-distilling experience and automatic documentation generation.

---

4. Participants

1. **Dual-Mode Crystallizer**: the agent that performs information-theoretic extraction over the code diff and the navigator's review logs after a task passes local verification;
2. **Fact Filter**: tags environmental facts — absolute file paths, specific IP addresses, ephemeral port numbers, temporary usernames and passwords, and the like — with `Scope: REPOSITORY_LOCAL`, prohibiting them from leaving the repository;
3. **Pattern Abstractor**: elevates concrete business-class implementations into abstract, transferable design patterns (including applicable context, core interface-contract pseudocode, and anti-pitfall checklists), assigning each a unique canonical identifier;
4. **Canonical Deduplicator**: ensures, via SHA-256 signing and semantic deduplication, that each class of facts or patterns maintains a single source of truth (SSOT) in the knowledge base.

---

5. Collaborations

```
sequenceDiagram
    autonumber
```

```
actor Dev as Development Agent
participant Crystal as Dual-Mode Crystallizer
participant FactDB as Local Repository Fact Base (Local Facts)
participant PatternDB as Global Pattern Draft Vault (Global Patterns)

Dev->>Crystal: Submit verified task deliverables and self-healing logs
Crystal->>Crystal: Execute dual-mode decoupling semantic parsing

rect rgb(240, 255, 240)
    note right of Crystal: Substrate-Fact distillation (Repo-Specific)
    Crystal->>FactDB: Write: "specialscale-gateway POM version pinned at 5.10.0-SNAPSHOT"
    Crystal->>FactDB: Write: "Actuator health-probe endpoint is /actuator/info"
end

rect rgb(255, 248, 220)
    note right of Crystal: Transferable-Pattern distillation (Transferable Pattern)
    Crystal->>PatternDB: Write draft: "Response-header safe-desensitization pattern based on
    note over PatternDB: Forced state DRAFT_SUSPECT, pending overall-acceptance final review
end

Crystal-->>Dev: Crystallization complete; dual-mode assets archived
```

---

6. Consequences

- **Pros**:
  - **Blocks memory poisoning at the root**: environment-specific hard codes can never escape into the global layer, preventing later agents from being misled;
  - **Maximized knowledge compounding**: the distilled transferable patterns are highly generalizable; when a new task is onboarded, mature design templates can be hit directly, cutting exploration rounds by more than 50%;
  - **Model-collapse defense**: knowledge washed through strict structured cleaning avoids the model feeding on its own low-quality hallucinations during later fine-tuning or context backfill (Shumailov et al., *Nature* 2024).
- **Cons**:
  - An extra round of structured information-extraction inference overhead is required at task end;
  - Strict structured schemas must be defined to constrain the attribute fields of facts and patterns.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be eradicated
Raw Memory Dumping	Vectorizing and ingesting the entire interaction dialogue of the development process as Memory	Mixes irrelevant chatter and failed errors with correct experience, causing severe retrieval-time misdirection

Anti-pattern	Manifestation	Why it must be eradicated
Global Fact Spillover	Allows agents to share local-specific ports and credentials as "standard specifications" with other repositories	Breaks other independent projects' infrastructure and network topology

---

8. Algorithmic Formulation

```typescript
interface CrystallizedAsset {
  assetType: "SUBSTRATE_FACT" | "TRANSFERABLE_PATTERN";
  scope: "LOCAL_REPO" | "GLOBAL_CROSS_REPO";
  canonicalKey: string;
  payload: object;
}

class DualModeTechnicalCrystallizer {
  categorizeAndPersist(executionTrace: SDLCExecutionTrace): CrystallizedAsset[] {
    const assets: CrystallizedAsset[] = [];

    // 1. Extract substrate facts (hard constraint: strictly limited to LOCAL_REPO)
    for (const fact of executionTrace.discoveredFacts) {
      if (this.containsEphemeralEnvironmentDetail(fact)) {
        assets.push({
          assetType: "SUBSTRATE_FACT",
          scope: "LOCAL_REPO",
          canonicalKey: `repo:${executionTrace.repoId}:${fact.key}`,
          payload: { detail: fact.value, validatedAt: Date.now() }
        });
      }
    }

    // 2. Extract transferable patterns (sanitize and abstract into generic design specifications
    for (const pattern of executionTrace.discoveredPatterns) {
      const sanitizedPattern = this.stripSubstrateSpecifics(pattern);
      assets.push({
        assetType: "TRANSFERABLE_PATTERN",
        scope: "GLOBAL_CROSS_REPO",
        canonicalKey: `pattern:${sanitizedPattern.patternName}`,
        payload: {
          designSchema: sanitizedPattern.schema,
          antiPatterns: sanitizedPattern.avoidedPitfalls,
          lifecycleStatus: "DRAFT_SUSPECT" // poison-defense via Pattern 14
        }
      });
    }
```

```
    return assets;
  }
}
```

---

9. Epistemic Status of Quantitative Claims

Disposition: downgraded to an empirical target (unverified) — "cutting exploration rounds by more than 50%" depends on task distribution and template hit rate, with no preregistered criteria. It is marked as an **engineering empirical target**; to be promoted, one must first fix the task family, the counting convention for "exploration rounds", and the decision rule for a "hit", and then run a paired comparison.

Chapter 17: Pattern 14 — Quarantined Promotion Gate Pattern

---

1. Intent

Establish for knowledge assets a mechanism of **single-source crystallization, draft quarantine, and double-blind promotion gates**: all self-generated experience (Rules/Patterns) distilled by agents during R&D execution is by default marked as a suspicious draft (`DRAFT_SUSPECT`), and is promoted to a globally active asset (`VERIFIED_ACTIVE`) only after overall acceptance fully passes the objective physical five-fold criteria, thereby blocking memory poisoning caused by cross-generational transmission of erroneous experience.

---

2. Motivation & Real-World Failure Modes

In many agent systems endowed with "self-learning" or "knowledge distillation" capabilities, the most dangerous phenomenon is **memory poisoning** — self-generated experience that has not passed final review, once retrieved and reused, exerts persistent contamination on subsequent tasks; this recursive self-consumption has been confirmed at the macroscopic level by empirical research to cause model collapse (Shumailov et al., *Nature* 631, 2024):

- **Failure Mode A (Self-reinforcement of erroneous experience)**: midway through a task, the agent, to fix an error, wrote a wrong rule: "when encountering a Redis dependency, downgrade the version to 2.1". The agent immediately wrote this experience into the global knowledge base. Yet the task's final evaluation collapsed completely. Unfortunately, the next 10 agents, on startup, retrieved this "experience" and all erroneously downgraded the dependency during initialization, triggering a cascading avalanche across the global pipeline;
- **Failure Mode B (Proliferation of tainted code patterns)**: the agent introduced an anti-pattern while writing code (e.g., writing raw SQL directly in a Controller); lacking a promotion gate, this code pattern was crystallized as a "best-practice template" and then spread like a virus across the entire project;
- **Failure Mode C (Leakage of unfinished knowledge)**: half-baked knowledge from an ongoing self-healing attempt is read by other concurrently running agents, causing them to work on unverified assumptions.

```
flowchart TD
    RawCrystallization["New patterns/experience spontaneously crystallized by agents during devel

    subgraph QuarantineDefense["Auto-Immunity Firewall"]
        direction TB
        Q1["1. Private to the current session only; hard-blocked from global retrieval (filtered
        Q2["2. Tied to the current delivery batch (Tied to Delivery Batch)"]
    end

    Quarantine --> QuarantineDefense

    subgraph FinalSubstrateGate["End-to-End Five-Fold Physical Criteria Final Review (Mechanical
        G1["Git Diff entity verification"]
        G2["Native compile offline build Exit 0"]
        G3["Hidden Oracle black-box probes all pass"]
        G4["No self-referential assertion audit"]
        G5["Code-review expert final approval"]
    end

    QuarantineDefense --> FinalSubstrateGate

    FinalSubstrateGate -- "Any physical criterion fails" --> Discard["Judged as a poisoned patter
    FinalSubstrateGate -- "All five physical criteria green" --> Promote["Promoted to VERIFIED_AC
```

---

3. Applicability

- Autonomous agent systems with long-term memory or self-distilling code-snippet capabilities;
- Self-evolving platforms that batch-process similar business modules or benchmark task sets;
- Enterprise-grade core-asset repository distillation and automatic knowledge-graph construction.

---

4. Participants

1. **Crystallizer**: automatically extracts reusable technical patterns and fact descriptions after the agent completes a micro-fix;
2. **Quarantine Vault**: independently stores assets that have not passed final review, forcibly tagging them `status: "DRAFT_SUSPECT"` and hard-filtering them out of regular vector or keyword retrieval;
3. **Promotion Gate Arbiter**: the supreme arbiter of end-to-end physical inspection, triggering batch promotion only after the entire delivery pipeline withstands real black-box physical probe testing;
4. **Tainted Tagger**: for drafts produced by rejected or ultimately failed tasks, assigns the `DEPRECATED_QUARANTINED` tag and archives them to a blacklist, permanently prohibiting them from leaving the repository.

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Dev as Development Agent
    participant Crystal as Knowledge Crystallizer
    participant KB as Global Knowledge Base
    actor OtherDev as Subsequent Task Agent
    participant Gate as Physical Five-Fold Criteria Final Review Gate (Promotion Gate)

    Dev->>Crystal: Submit task solution and crystallized patterns
    Crystal->>KB: Write knowledge assets with status DRAFT_SUSPECT

    OtherDev->>KB: Retrieve available knowledge (verifiedOnly: true)
    KB-->>OtherDev: Filter out DRAFT_SUSPECT, returning only existing promoted VERIFIED_ACTIVE as

    alt Task ultimately fails the five-fold physical criteria or is rejected
        Gate->>KB: triggerRejection(batchId)
        KB->>KB: Mark all drafts of this batch as DEPRECATED_QUARANTINED (taint sequestration)
    else Task passes final review with all five criteria green and high score
        Gate->>KB: triggerPromotion(batchId)
        KB->>KB: Formally promote drafts to VERIFIED_ACTIVE (legitimate reuse)
    end
```

Extended Trigger Conditions: Stress-Test Results from Isolated Environments

When the stress-test environment is fully isolated from the development environment, **stress-test failure** serves as an explicit trigger condition for DRAFT_SUSPECT:

1. **Stress-test failure**: if the candidate version fails to reach the preregistered SLA threshold in the isolated stress-test environment (e.g., P99 latency degradation rate > threshold, error rate > threshold), then even if the development environment's five-fold criteria are all green, all drafts of this batch remain marked `DRAFT_SUSPECT` and promotion is prohibited;
2. **Missing stress-test results**: if no isolated-environment stress-test results are received within the preregistered time limit after the candidate version is submitted, the batch's drafts remain `DRAFT_SUSPECT` until results are backflowed and pass Pattern 02 probe 4 audit;
3. **Environmental homogeneity deviation**: if the stress-test environment's configuration fingerprint deviates from the production environment beyond the preregistered tolerance, the results are deemed invalid, the drafts remain `DRAFT_SUSPECT`, and an environment-repair process is triggered.

   Theoretical anchor: an extension of the applicability boundary of Axiom IV — results from isolated environments form part of "SubstratePass"; unaudited isolated-environment results may not serve as a basis for promotion.

6. Consequences

- **Pros**:
  - **Knowledge-base self-cleaning immunity**: fundamentally eliminates the self-propagation and cross-generational transmission of shoddy code and erroneous experience within the system;
  - **Positive compounding effect**: every pattern entering `VERIFIED_ACTIVE` has passed strict testing by the real physical world, yielding extremely high accuracy when reused by later tasks.
- **Cons**:
  - Knowledge reuse has a time lag: within the same task batch, one cannot immediately benefit from experience being crystallized by the current batch;
  - Maintaining metadata state lifecycles requires additional storage and index-management logic.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be eradicated
Unverified Eager Ingestion	The agent writes one line of code and immediately ingests it into shared Memory as a best practice	Once the code contains a fatal defect, it will rapidly poison the entire system's memory network
Stateless Taint Blindness	Erroneous knowledge produced by failed tasks is left unmarked and allowed to remain in the repository	Causes knowledge-base fragmentation to run out of control, with historical garbage experience severely interfering with future decisions

---

8. Algorithmic Formulation

```typescript
type KnowledgeLifecycle = "DRAFT_SUSPECT" | "VERIFIED_ACTIVE" | "DEPRECATED_QUARANTINED";

interface KnowledgePattern {
  id: string;
  patternName: string;
  codeSnippet: string;
  lifecycle: KnowledgeLifecycle;
  associatedBatchId: string;
}

class SelfImmuneKnowledgeBase {
  private patterns: KnowledgePattern[] = [];

  // 1. Crystallization ingestion: quarantined by default
  ingestDraftPattern(pattern: Omit<KnowledgePattern, "lifecycle">): void {
```

```
    this.patterns.push({
      ...pattern,
      lifecycle: "DRAFT_SUSPECT" // forced quarantine
    });
  }

  // 2. Retrieval defense: by default only expose assets that have passed final review
  queryVerifiedPatterns(filter: { verifiedOnly?: boolean }): KnowledgePattern[] {
    const verifiedOnly = filter.verifiedOnly !== false;
    return this.patterns.filter(p => {
      if (p.lifecycle === "DEPRECATED_QUARANTINED") return false; // hard taint interception
      if (verifiedOnly && p.lifecycle !== "VERIFIED_ACTIVE") return false;
      return true;
    });
  }

  // 3. Final-review adjudication and promotion
  adjudicateBatch(batchId: string, passedPhysicalGate: boolean): void {
    const targetState: KnowledgeLifecycle = passedPhysicalGate ? "VERIFIED_ACTIVE" : "DEPRECATED_
    this.patterns
      .filter(p => p.associatedBatchId === batchId && p.lifecycle === "DRAFT_SUSPECT")
      .forEach(p => {
        p.lifecycle = targetState;
      });
  }
}
```

Chapter 18: Pattern 15 — Tainted Pattern Rejection & Anti-Degradation Pattern

---

1. Intent

Establish an **immune self-healing mechanism** based on taint tracking and taint quarantine propagation. Once a code pattern spontaneously crystallized or adopted by an agent is judged by a later stage (e.g., a native multi-module build failure, a reviewer's reality-audit veto, or a hidden-Oracle probe rejection) to be unreliable or fraudulent, the system immediately **forcibly demotes the pattern and all derived knowledge to `DEPRECATED_QUARANTINED` and seals it into a hard-blocked blacklist**, blocking the cross-generational contamination of the autonomous system by locally erroneous experience and model collapse (Model Collapse, Shumailov et al., *Nature* 2024).

---

2. Motivation & Real-World Failure Modes

In autonomous agent evolution systems, the most insidious damage does not come from external cyberattacks, but from the agents' own "pseudo-experience self-reproduction". Recent academic work (MemoryGraft, 2025; OEP, 2026) has formally codified this phenomenon as **memory poisoning**:

- **Failure Mode A (Poisoning via Locally Correct but Non-Transferable Experiences)**: In a simple module, the agent got an edge-case unit test to pass by commenting out a security interceptor. The agent distilled the pattern "when encountering an interceptor error, you can disable it". Had this entry not been banned, other agents later executing security-authentication tasks would have automatically recalled and reused the pattern on encountering similar errors, directly collapsing the security defense line of the core gateway;
- **Failure Mode B (Falsified Pass Leading to Toxic Ingestion)**: The agent passed the unit test through self-referential mocking (e.g., hard-coding the expected value back into the test), and at execution end claimed this to be an "efficient problem-solving paradigm". If the evaluation system unmasked this reward hacking at a downstream gate but failed to clean the knowledge base in sync, this "cheating routine" would continue propagating within the system;

- **Failure Mode C (Cross-Generational Accumulation of Negative Experience and Model Collapse)**: If the system only ingests and never expels, lacking active excision and taint tracking for rejected shoddy experience, then as more tasks run, the proportion of erroneous, outdated, and mutually conflicting experience in the knowledge base gradually exceeds that of high-quality experience; agents suffer severe context rot and hallucination snowballing during retrieval (Zhang et al., ICML 2024), and the system's overall problem-solving rate drops off a cliff.

```
flowchart TD
    subgraph ToxicDetection["Taint Inception Event"]
        Event1["1. Native multi-module offline build failure (Native Build Exit != 0)"]
        Event2["2. Reviewer's physical reality-audit veto (Reality VETO)"]
        Event3["3. Hidden Oracle physical black-box probe failure (Oracle Probe Fail)"]
    end

    ToxicDetection --> TaintTracker["Causal Taint Tracker"]

    subgraph TaintPropagation["Taint Quarantine Propagation and Reverse Purge"]
        direction TB
        MarkTaint["1. Recursively locate all drafts produced by this failed task (Associated Batc
        ApplyQuarantine["2. Force state: DRAFT_SUSPECT -> DEPRECATED_QUARANTINED"]
        BlacklistIndex["3. Remove from knowledge-base vector index and persist to blacklist"]
        ImmuneMemory["4. Distill the negative-lesson warning (Negative Invariant Warning)"]
    end

    TaintTracker --> TaintPropagation
    TaintPropagation --> CleanKnowledgeBase["Clean Verified Knowledge Base (Clean Verified Knowle
```

---

3. Applicability

- Enterprise-grade software-agent platforms with autonomous learning and knowledge-distillation capabilities that run continuously long-term;
- Autonomous pipelines undergoing complex self-healing loops or evaluation-benchmark tasks;
- Defense- or finance-grade systems that must rigorously safeguard code quality and prevent the diffusion of any unverified experience.

---

4. Participants

1. **Taint Detector Probe**: listens for verdict events from each physical gate (build compiler, Reviewer VETO, hidden Oracle), and emits a taint alert immediately upon capturing a failure;
2. **Causal Taint Tracker**: uses the batch identifier (`batchId`) and the task causal graph to reverse-look-up all temporary experience and derived code fragments crystallized during this failure's lifecycle;
3. **Quarantine Enforcer**: atomically changes the target pattern's state in global metadata to `DEPRECATED_QUARANTINED` and imposes hard-block rules at the retrieval-routing layer;

4. **Negative Spec Generator**: extracts the root cause of this failure (e.g., "do not use `@Disabled` to evade tests") into an anti-pattern warning (Negative Constraints), converting it into defensive prompts for later task planners.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Dev as Agent Session
    participant KB as Global Knowledge Base
    participant Evaluator as Five-Fold Physical Evaluation Gate
    participant Tracker as Taint Tracker

    Dev->>KB: Temporarily store crystallized pattern P_new (marked DRAFT_SUSPECT)
    Dev->>Evaluator: Submit final code deliverable and request final review

    Evaluator->>Evaluator: Run native in-place build and hidden probes
    Evaluator-->>Tracker: Severe warning: reward hacking captured (Exit Code 1 / Tautology Detect

    rect rgb(255, 230, 230)
        note right of Tracker: Launch taint quarantine propagation and reverse purge
        Tracker->>KB: Lock all patterns generated by the session (P_new)
        Tracker->>KB: Atomically tag: P_new.lifecycle = DEPRECATED_QUARANTINED
        Tracker->>KB: Physically remove P_new from vector and keyword indexes
        Tracker->>KB: Append anti-pattern blacklist entry: "Strictly forbidden to assert self-ref
    end

    Tracker-->>Evaluator: Taint purge complete; the knowledge base's immune firewall is closed
```

---

6. Consequences

- **Pros**:
  - **Knowledge-base self-cleaning antibodies (Self-Immune System)**: the system can self-cleanse after failure, eradicating the chain reaction in which "one erroneous experience poisons the entire platform" (taint tracking + promotion gate);
  - **Model-collapse prevention**: blocks the vicious internal cycle of low-quality synthetic data and fake patterns;
  - **Reuse of negative experience**: converts every painful failure into an explicit hard-constraint warning, inversely improving the planning quality of later tasks.
- **Cons**:
  - Requires the database and index layers to support soft deletion with state filtering or dynamic weight demotion;
  - The taint-tracing chain requires full-lifecycle logs to have complete causal provenance capability.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be eradicated
Silent Failure Accumulation	After a task is rejected or fails, the experience the agent wrote during the process goes unattended	Causes harmful knowledge to lurk silently in the repository as a time bomb
Scorched-Earth Wipe	Upon a single task failure, directly empties all historical knowledge accumulated by the entire system	Erases existing high-value technical assets already physically verified, forfeiting compounding

---

8. Algorithmic Formulation

```
interface TaintEvent {
  batchId: string;
  failureType: "BUILD_FAILURE" | "VETO_FRAUD" | "ORACLE_MISMATCH";
  rootCauseMessage: string;
}

class TaintQuarantineManager {
  private knowledgeRepo: KnowledgeRepository;

  async handleTaintEvent(event: TaintEvent): Promise<TaintRemediationSummary> {
    // 1. Recursively trace all draft assets associated with this batch
    const associatedPatterns = await this.knowledgeRepo.findByBatch(event.batchId);

    let quarantinedCount = 0;
    for (const pattern of associatedPatterns) {
      // 2. Force quarantine taint tag
      pattern.lifecycle = "DEPRECATED_QUARANTINED";
      pattern.quarantineReason = `${event.failureType}: ${event.rootCauseMessage}`;
      pattern.quarantinedAt = Date.now();

      await this.knowledgeRepo.update(pattern);
      quarantinedCount++;
    }

    // 3. Distill the failure signature into a reverse defensive rule
    const negativeRule: NegativeConstraint = {
      id: `anti-pattern-${Date.now()}`,
      forbiddenPattern: event.rootCauseMessage,
      action: "REJECT_IN_PLANNING"
    };
    await this.knowledgeRepo.appendNegativeConstraint(negativeRule);

    return {
```

```
      batchId: event.batchId,
      quarantinedPatternsCount: quarantinedCount,
      negativeRuleRegistered: true
    };
  }
}
```

---

9. Formal-Theoretic Support (Lean 4 Formal Proof)

The tombstone-state absorptivity and the reverse auto-immunity blocking of this pattern have been formally mechanized and verified in **Lean 4** (0 axioms, 0 sorry; source: `formal_proof/IACFormalization/src/IACFormalization/Pattern15_RevocationTombstone.lean`, corresponding to Appendix B.12):

- **Tombstone-state terminal-absorption theorem (`tombstone_absorbing`)**: The tombstone state is the terminal absorbing state of the state machine; no subsequent state transition can escape it.
- **Dynamic-projection quarantine-safety theorem (`tombstone_never_projectable`)**: A pattern in the tombstone state can never appear in any context-retrieval projection slice.
- **Reverse auto-immunity blocking theorem (`immunity_blocks_taint`)**: The reverse immune rule set automatically synthesized from tombstoned patterns mechanically blocks at 100% any cross-generational reproduction of candidate patterns carrying the same taint signature.
- **Atomic revocation immediate-invalidation theorem (`revoked_excluded_from_projection`)**: After a revocation operation, the target pattern is atomically set to tombstone and immediately disappears from all task projection views.
- **Cross-generational reproduction truncation theorem (`tombstone_offspring_zero`)**: The effective reproduction mean of a tombstoned pattern is strictly zero; branching-process offspring go extinct in the very next generation.

Chapter 19: Pattern 16 — Accordion Multiresolution Projection Pattern

---

1. Intent

Establish a **dynamic multiresolution asymmetric cognitive projection mechanism** between human engineers and the autonomous agent system. According to the physical bandwidth of the interaction channel (e.g., full-duplex real-time voice, mobile notifications, desktop console) and the human attention state, the high-volume, high-dimensional engineering execution state is freely stretched like an accordion into a **"One-Sentence Beacon"**, a **"Phase Summary"**, or a **"Full Physical Trace"**, eliminating context rot and cognitive overload in human–machine interaction.

---

2. Motivation & Real-World Failure Modes

In end-to-end autonomous software engineering systems, a single requirement evolution by an agent may produce tens of thousands of lines of compiler console output, hundreds of tool-call logs, and complex dependency-graph analyses. Without cognitive-resolution management for human–machine interaction, severe failures polarize into two extremes:

- **Failure Mode A (Console Vomiting)**: At every step the pipeline executes, it regurgitates to the human user tens of thousands of characters of raw upstream compiler Stderr, Maven download progress, and the full JSON state. Faced with this flood of scrolling information, a human engineer cannot possibly grasp the core issue within seconds (e.g., a missing critical permission class), and is forced to spend dozens of minutes searching manually — completely losing the agility of "autonomous collaboration";
- **Failure Mode B (Context Rot in Voice/Mobile Interaction)**: When progress is reported to an engineer through a voice assistant or a mobile device, if the agent mechanically reads aloud long class names and code diffs, human working memory collapses within 10 seconds. Meanwhile, the verbose text severely blocks bidirectional interaction on the full-duplex voice channel;
- **Failure Mode C (Opaque Gloss-Over — Reporting Only Good News)**: The opposite extreme is over-simplification: the agent merely replies "一切正常", while concealing from the human the high-risk physical fact that "three compile self-healing rollbacks have

already been triggered and sensitive security configurations are being modified" — causing the system to slip out of human course control.

```
flowchart TD
    RawPipeline["High-dimensional raw state stream of the pipeline"] --> AccordionProjector["Acco

    subgraph ResolutionLevels["The 4 Cognitive Resolution Tiers"]
        direction TB
        L0["<b>Tier 0: State Beacon (Beacon Tier: <10 Tokens)</b><br/><i>Use case: always-on full
        L1["<b>Tier 1: Contract Summary (<200 Tokens)</b><br/><i>Use case: mobile push, Slack/Lar
        L2["<b>Tier 2: Audit Tier (<2K Tokens)</b><br/><i>Use case: PR review tickets, code audit
        L3["<b>Tier 3: Raw Physical Stack (Substrate Forensic Tier: Full Trace)</b><br/><i>Use ca
    end

    AccordionProjector --> ResolutionLevels
    ResolutionLevels --> AdaptiveDelivery["Adaptive delivery according to end-side physical bandw
```

---

3. Applicability

- Autonomous systems connected to a real-time voice dialogue channel (e.g., full-duplex WebRTC / mobile voice assistants);
- Enterprise collaboration platforms that must support multi-device linkage (phone for summaries, IDE for assertions, terminal for physical logs);
- "Air Traffic Control" scenarios in which a senior engineer supervises dozens of autonomous agents working in parallel.

---

4. Participants

1. **Substrate Telemetry Aggregator**: collects in real time the physical metrics of each pipeline stage (stage name, Jev score, exit code, affected files);
2. **Accordion Projector**: according to the `ResolutionTier` requested by the caller, applies information-theoretic summarization algorithms for lossless dimensionality reduction or full expansion;
3. **Attention & Bandwidth Detector**: senses the type of the current interaction device (Voice, Mobile Push, Web GUI, CLI Terminal) and the user's idle state, dynamically negotiating the default resolution tier;
4. **Progressive Drill-Down Controller**: supports an interactive "Zoom-In" protocol — when the human raises a question about the Level 0 summary, it smoothly expands to Level 2 or Level 3, keeping the mental model coherent.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Human as Flight Director (human engineer)
    participant UI as Full-duplex interaction interface (Voice/Web)
```

```
participant Accordion as Accordion Projector
participant Core as Pipeline execution core

Core->>Accordion: high-dimensional event: T2 self-healing succeeded, 12 tool calls consumed,
Accordion->>UI: [default Level 0: minimal beacon] voice broadcast: "T2 控制器已通过原厂测试，工

Human->>UI: interactive instruction: " 展开讲讲改了什么测试？"
UI->>Accordion: requestDrillDown(taskId="T2", targetTier=Level_2)
Accordion->>Accordion: extract changed files and three-layer test specification slices
Accordion-->>UI: [Level 2 projection] present card: " 修改 2 个文件，追加 12 个纯 Mockito 单测

Human->>UI: instruction confirmed: " 很好，批准继续。"
UI->>Core: continue execution on the original course
```

---

6. Consequences

- **Pros**:
  - **Extreme human–machine collaboration bandwidth**: frees the human engineer's eyes and hands; macro-level situational awareness of the entire engineering fleet via minimal voice beacons;
  - **On-demand retrieval without cognitive overload**: eliminates context rot, ensuring that key decision information reaches the human brain at optimal information density;
  - **Hard evidence always verifiable on the spot**: although the daily presentation is minimal, the underlying Level 3 raw physical stack can be inspected locally at any time — fully transparent.
- **Cons**:
  - The platform layer must implement multi-level formatting of state metadata and a dynamic template engine;
  - Maintaining streaming drill-down session state requires lightweight in-memory session management.

---

7. Anti-Patterns to Avoid

Anti-pattern name	Manifestation	Why it must be avoided
Unfiltered Log Spewing	Indiscriminately dumping full screens of raw build logs onto every terminal	Causes human information overload; when a truly severe alert arrives, it cannot be recognized
Black-Box Hand-Waving	Merely announcing "完成了", yet unable to produce a physical file diff and test list when the human asks for evidence	Loses explainability and the foundation of trust; violates the substrate-anchoring principle

---

8. Algorithmic Formulation

```typescript
type ResolutionLevel = "L0_BEACON" | "L1_SUMMARY" | "L2_AUDIT" | "L3_FORENSIC";

interface PipelineStateTelemetry {
  currentPhase: string;
  subtaskProgress: string;
  jevScore?: number;
  changedFiles: string[];
  newTestsDeclared: string[];
  rawBuildStderr: string;
}

class AccordionMultiresolutionProjector {
  projectState(state: PipelineStateTelemetry, targetLevel: ResolutionLevel): string {
    switch (targetLevel) {
      case "L0_BEACON":
        // Minimal voice-grade beacon (<10 words)
        return `[${state.currentPhase}] 进度 ${state.subtaskProgress}，物理单测全绿。`;

      case "L1_SUMMARY":
        // Mobile-card-grade contract summary
        return `阶段: ${state.currentPhase} | 进度: ${state.subtaskProgress} | Jev 门禁: ${state.

      case "L2_AUDIT":
        // Reviewer audit tier (file list + three-layer test white-box assertions)
        return JSON.stringify({
          files: state.changedFiles,
          tests: state.newTestsDeclared,
          gateScore: state.jevScore
        }, null, 2);

      case "L3_FORENSIC":
        // Full physical trace (debugging / troubleshooting)
        return state.rawBuildStderr;
    }
  }
}
```

---

9. Epistemic Status

> **Engineering Convention**: This pattern is an unfalsifiable operational specification, not a theoretical conclusion. Its correctness can only be judged by "whether the implementation follows the prescribed projection protocol"; the pattern itself provides no statistical criteria, nor does it constitute a corollary of Axioms I–IV or Theorem 3.1.

The three-level Token budgets (<10 / <200 / <2K) are design constants, not validated effect sizes. If they are to be treated as conclusions, measurable operational definitions of "projection

fidelity" and "decision sufficiency" must first be defined and thresholds preregistered. In the validation report §8 matrix, this pattern is classified as an engineering convention; its statements must not be used as a reasoning premise for other patterns.

Chapter 20: Pattern 17 — Asynchronous Flight-Director Interrupt Pattern

---

1. Intent

Establish the supreme human-arbitration cybernetic mechanism of human–machine interaction, based on **asynchronous full-duplex interrupt** and a **course-lock causal state machine**. It grants the human "Flight Director" the privilege, at any moment during the agent's multi-step autonomous cruise, to issue millisecond-scale non-blocking interjections, freeze mutating transactions, inject corrective will, and lock the global course — breaking the runaway state in which the agent is trapped in a self-healing death spiral or has deviated from the business goal.

---

2. Motivation & Real-World Failure Modes

In the operation of agent pipelines with extremely high end-to-end autonomy, severe situations in which the agent falls into an uncontrolled state frequently occur:

- **Failure Mode A (Runaway Self-Healing Spiral)**: While repairing a complex compilation error, the agent — having misunderstood the business intent — begins autonomously refactoring the entire underlying architecture (e.g., deleting an important core base class). In a traditional synchronous blocking system, the human engineer can only watch helplessly as the console keeps scrolling errors and consuming enormous Tokens, unable to input a command to stop it, and is reduced to brutally killing the process (`Kill -9`), causing all previously valid work to be reduced to ashes;
- **Failure Mode B (Unverified Self-Correction Fallacy)**: Academic research shows that, without external physical constraints, a large model's self-reflection is not only ineffective but often leads to performance degradation (Huang et al., ICLR 2024). During multi-round self-derivation, the agent becomes convinced that "users do not need tenant isolation" and begins stripping tenant contexts on a large scale. Humans need an elegant mechanism to "pull the reins";
- **Failure Mode C (Race Conditions in Human Steering)**: The human enters the correction "不要修改 User 类", but before that input is processed, the agent has already completed the write to `User.java` and submitted the compilation — producing a causal inversion between the human's instruction and the agent's current execution context.

```
flowchart TD
    Human["Human Flight Director"] --> FastInterrupt["Asynchronous highest-priority interrupt cha

    subgraph AutonomousCruising["Autonomous Agent Cruising State Machine"]
        direction TB
        AgentPlanning["Task planning and decomposition"] --> AgentActuation["Substrate mutation a
        AgentActuation --> GateCheck["Gate scoring and micro-loop"]
        GateCheck --> AgentActuation
    end

    FastInterrupt -- "Millisecond-scale non-blocking pause (Safe-Point Pause)" --> AutonomousCrui

    subgraph FlightDirectorControl["Flight Director Intervention & Course-Lock"]
        direction TB
        Freeze["1. Freeze physical transactions (Freeze Substrate Mutations)"]
        Inspect["2. Inspect the scene via the accordion multiresolution projection"]
        InjectSteering["3. Inject hard will (Course Correction Constraint)"]
        LockCourse["4. Apply an immutable course lock (Course-Lock Invariant)"]
    end

    AutonomousCruising --> FlightDirectorControl
    FlightDirectorControl -- "Resume cruising with hard invariants (Resume with Hard Invariants)"
```

---

3. Applicability

- Autonomous software engineering systems running ultra-long (dozens of minutes to hours), highly complex tasks;
- Human–agent pair-programming interfaces equipped with full-duplex real-time voice and IM bots;
- High-compliance scenarios where the system must remain ultimately human-controllable and safety-aligned at every moment.

---

4. Participants

1. **Flight Director (Human Operator)**: the decision-making subject with the system's highest priority, representing the ultimate business requirements;
2. **Asynchronous Interrupt Listener**: resident in the background as a non-blocking event loop, constantly listening for the human's voice interruptions or text interrupt signals;
3. **Safe-Point Controller**: upon receiving an interrupt signal, waits for the currently executing atomic file write to complete, then immediately freezes the execution pipeline, preventing the filesystem from being left in a half-written corrupted state;
4. **Course-Lock Enforcer**: transforms the human's correction instructions into highest-weight immutable constraints (hard invariants), granting them veto power in subsequent prompt generation and gate audits, and forbidding the agent from bouncing back a second time.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Human as Human Flight Director
    participant Interrupt as Asynchronous interrupt listener (Interrupt Channel)
    participant SafePoint as Safe-point controller (Safe-Point)
    participant Agent as Cruising agent (Autonomous Agent)
    participant Substrate as Physical filesystem

    Agent->>Substrate: executing a long-running tool call (e.g., multi-module compilation)

    Human->>Interrupt: issue a live interjection: " 暂停！网关脱敏不要删除 Server 头，改为掩码处理
    Interrupt->>SafePoint: emit the highest-priority interrupt signal (SIG_INTERRUPT)
    SafePoint->>Agent: intercept subsequent action dispatch, force into the safe frozen state (ST

    SafePoint-->>Human: acknowledgment: " 巡航已冻结在安全点。请确认航向修正规范。"
    Human->>SafePoint: confirm course parameters: { maskMode: "MASK_WITH_ASTERISK", immutable: tr

    SafePoint->>Agent: inject the course-lock contract (Course-Lock Injected)
    SafePoint->>Agent: release the freeze, resume cruising under the new course directive (RESUME
    Agent->>Substrate: execute precise refactoring according to the new constraint
```

---

6. Consequences

- **Pros**:
  - **Human-in-the-Loop Supremacy**: the human engineer can "take over control" at any moment — no longer a powerless bystander — eliminating runaway self-healing death spirals;
  - **Causal consistency guarantee**: via the safe-point mechanism, the interrupt never corrupts physical files or the Git index;
  - **Huge energy savings**: braking and correcting the agent the moment it goes astray avoids ineffective exploration consuming millions of Tokens.
- **Cons**:
  - The orchestration state machine must possess a fully decoupled asynchronous event loop;
  - Resuming cruising requires realigning the context prompt and eliminating the residue of the interrupted operation.

---

7. Anti-Patterns to Avoid

Anti-pattern name	Manifestation	Why it must be avoided
Mock Full-Duplex Fallacy	The agent blocks input entirely while calling tools or thinking; the human must wait until it finishes answering before speaking	Loses the possibility of rescuing the system in real time; severely degrades the user interaction experience
Ephemeral Steering Amnesia	After the human interjects a correction, the agent forgets the human's warning entirely in the next execution round	Violates causal invariance, forcing the human to repeat the same instruction over and over

---

8. Algorithmic Formulation

```
interface CourseLockDirective {
  targetPath: string;
  immutableConstraint: string;
  injectedBy: "FLIGHT_DIRECTOR";
  timestamp: number;
}

class FlightDirectorInterruptManager {
  private isFrozen: boolean = false;
  private activeDirectives: CourseLockDirective[] = [];

  // 1. Asynchronous non-blocking interrupt listening
  async onHumanInterruptReceived(transcript: string): Promise<string> {
    this.isFrozen = true;
    const safePointReached = await this.awaitNextSafePoint();

    if (safePointReached) {
      // Parse the human's intent and generate the course-lock rule
      const directive = this.parseDirectiveFromSpeech(transcript);
      this.activeDirectives.push(directive);

      return `FLIGHT_DIRECTOR_INTERRUPT_ACK: Execution safely frozen. Directive [${directive.immu
    }

    throw new Error("SAFE_POINT_TIMEOUT: Force pause initiated.");
  }

  // 2. Project the course constraints into the agent's prompt
  injectDirectivesIntoPrompt(basePrompt: string): string {
    if (this.activeDirectives.length === 0) return basePrompt;

    const directivesBlock = this.activeDirectives
      .map(d => `[FLIGHT_DIRECTOR_COURSE_LOCK] ${d.immutableConstraint} (STRICT_IMMUTABLE)`)
      .join("\n");
```

```
    return `${basePrompt}\n\n### HIGHEST PRIORITY FLIGHT DIRECTIVES:\n${directivesBlock}\n`;
  }
}
```

---

9. Epistemic Status

> **Engineering Convention**: This pattern is an unfalsifiable operational specification, not a theoretical conclusion. Its correctness can only be judged by "whether the implementation follows the prescribed takeover protocol"; the pattern itself provides no statistical criteria, nor does it constitute a corollary of Axioms I–IV or Theorem 3.1.

The priority rules of interrupt takeover and course-lock are protocol conventions. If they are to be treated as conclusions, "interrupt loss rate" and "constraint violation rate" must first be defined and thresholds preregistered; until then, the "highest priority" stated in the text is merely protocol semantics, not a measured effect. In the validation report §8 matrix, this pattern is classified as an engineering convention; its statements must not be used as a reasoning premise for other patterns.

Chapter 21: Pattern 18 — Hidden Oracle Sandbox Pattern

---

1. Intent

Decouple the objective truth (Oracle) of real industrial-grade evaluation physically from the context used to prompt the agent. Through **read-only mirror mounting, hidden black-box assertions, and a bypass-free adjudication pipeline**, eliminate the agent's benchmark-gaming overfitting against the exam questions (data leakage & benchmark gaming), and provide absolutely trustworthy benchmark evaluation capability for industrial autonomous software engineering.

---

2. Motivation & Real-World Failure Modes

In numerous open-source benchmarks (e.g., HumanEval, SWE-bench), the problems of **evaluation-set contamination** and **question leakage** are pervasive:

- **Failure Mode A (Training-set and prompt-embedded tests)**: the assertions of the test cases are written directly into the prompt sent to the model. Relying on its powerful memory or pattern-recognition ability, the model writes `if (input == "A") return "B"` directly inside the implementation class, manufacturing the illusion of passing the tests;
- **Failure Mode B (The agent sniffs out and modifies the evaluation files)**: the agent, granted global search permissions inside the sandbox, recursively locates the verification scripts under the test directory and directly modifies the test scripts to match its own erroneous output;
- **Failure Mode C (Mock bypass around the real service)**: finding that external dependencies are unreachable, the agent spins up an inline mock service to hijack the network traffic, bypassing the core filtering chain of the real microservice gateway.

```
flowchart TD
    subgraph PublicWorkspace["Agent Visible Space"]
        direction TB
        Agent["Agent execution context"] --> PublicBrief["Publicly released task requirements (Br
        Agent --> TargetRepo["Mutable business source directory (repo/src/main)"]
    end

    subgraph HiddenOracleVault["Hidden Oracle Physically Isolated Sandbox"]
        direction TB
```

```
        OracleTests["Invisible hidden Oracle test suite (Hidden Test Suite)"]
        ProbeRunner["Bypass-free mechanical evaluation probes (Bypass-Free Probes)"]
    end

    PublicWorkspace -- "Deliverable (read-only pull of the Git commit)" --> Evaluator["End-to-end
    HiddenOracleVault -- "Double-blind read-only mount" --> Evaluator
    Evaluator --> BinaryVerdict["True objective physical score (Pass / Fail)"]
```

---

3. Applicability

- Objective capability evaluation and benchmarking of software-engineering autonomous agents (SWE Agents);
- Automated acceptance gates for enterprise core systems;
- Competitions, qualification examinations, or high-security automated operations scenarios.

---

4. Participants

1. **White-box Solution Workspace**: the only writable space the agent possesses, containing only the public requirements specification and the business source tree, and absolutely none of the assertion details of the evaluation set;
2. **Hidden Oracle Vault**: the high-order acceptance test suite, the fully faithful production-grade dependencies, and the golden standard stored in an external secure directory or encrypted volume;
3. **Isolated Sandbox Runner**: a temporary independent sandbox launched after the agent submits its deliverable, mounting the agent's Git artifacts and the hidden Oracle, and running the adjudication under privileged authority that the agent cannot manipulate.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Benchmark as Benchmark orchestrator
    participant Agent as Agent system
    participant Repo as Target business repository
    participant Sandbox as Double-blind isolated sandbox
    participant Oracle as Hidden Oracle probes

    Benchmark->>Agent: release the solution task (only the public interface contract and function
    Agent->>Repo: mutate the code and complete the local build
    Agent-->>Benchmark: declare the solution complete, submit the Git branch reference

    rect rgb(255, 245, 238)
        note right of Benchmark: enter hidden physical adjudication (Zero Leakage)
        Benchmark->>Sandbox: create a brand-new stateless sandbox container
        Benchmark->>Sandbox: pull the code branch submitted by the agent
```

```
    Benchmark->>Oracle: load the hidden Oracle test suite (read-only injection)
    Oracle->>Sandbox: run black-box HTTP probes and deep boundary tests
    Sandbox-->>Benchmark: capture the objective Exit Code and HTTP status codes
  end

  Benchmark-->>Agent: output the final physical battle report (pass rate, Token consumption, ph
```

---

6. Consequences

- **Pros**:
  - **100% immune to question leakage and score-farming cheating**: the model cannot see the final test assertions while solving the task, and must write truly robust code based on the requirements and architectural specifications;
  - **Industrial-grade credibility**: code that passes under the hidden Oracle can be deployed directly to the real production environment.
- **Cons**:
  - Building the hidden sandbox and double-blind mounting requires additional automated container scheduling or isolated process management;
  - High question-development cost: designing an unambiguous, boundary-covering hidden Oracle requires deep involvement of senior software testing experts.

---

7. Anti-Patterns to Avoid

Anti-pattern name	Manifestation	Why it must be avoided
Open-Book Cheat Disguised as Closed-Book	Placing the unit-test source code, together with the answers, directly in the code tree visible to the agent	The model directly copies or modifies the tests, rendering the evaluation metrics distorted
In-Process Evaluation	The evaluation code runs inside the same process or session as the agent	The agent can bypass the evaluation logic via monkey patches or global-variable tampering

---

8. Algorithmic Formulation

```typescript
interface BenchmarkInstance {
  instanceId: string;
  publicPrompt: string;
  hiddenOraclePath: string;
}

class HiddenOracleEvaluator {
  async evaluateSubmission(instance: BenchmarkInstance, agentBranch: string): Promise<EvaluationR
    // 1. Create a physically isolated temporary evaluation environment
```

```
    const isolatedDir = await fs.promises.mkdtemp("eval-sandbox-");

    try {
      // 2. Check out the agent's submitted artifacts
      await gitCloneBranch(isolatedDir, agentBranch);

      // 3. Mount the hidden Oracle read-only (test cases injected by external privilege; invisib
      await injectHiddenOracle(isolatedDir, instance.hiddenOraclePath, { readonly: true });

      // 4. Run the upstream build and Oracle tests inside the sealed container
      const execResult = await runContainerizedTest(isolatedDir);

      return {
        instanceId: instance.instanceId,
        passed: execResult.exitCode === 0 && execResult.testSummary.failedCount === 0,
        rawExitCode: execResult.exitCode,
        durationMs: execResult.durationMs,
        coverageRate: execResult.coverageRate
      };
    } finally {
      // 5. Evaluation complete; securely destroy the sandbox
      await fs.promises.rm(isolatedDir, { recursive: true, force: true });
    }
  }
}
```

---

9. Isolated-Environment Extension: Stress-Test Hidden Oracle

When the stress-test environment is completely isolated from the development environment, Pattern 18 extends into a **dual hidden Oracle architecture**:

```
flowchart TD
    subgraph DevVisible["Agent Visible Space (development environment)"]
        Agent["Agent execution context"]
        PublicBrief["Publicly released task requirements"]
        TargetRepo["Mutable business source directory"]
    end

    subgraph HiddenOracle1["Hidden Oracle #1: functional test sandbox"]
        FuncTests["Functional test suite (unit / integration)"]
        FuncProbes["Functional probes (HTTP / Exit 0)"]
    end

    subgraph HiddenOracle2["Hidden Oracle #2: stress-test sandbox (isolated environment)"]
        StressTests["Stress-test configuration (load model / SLA thresholds)"]
        StressEnv["Stress-test execution environment (independent infrastructure)"]
        StressMetrics["Raw metrics collection (Prometheus / JMeter)"]
    end
```

```
    subgraph AuditChannel["Independent audit channel (Pattern 02 Probe 4)"]
        Auditor["Reality auditor"]
        IntegrityCheck["Transmission integrity verification"]
        HomogeneityCheck["Environment homogeneity verification"]
    end

    DevVisible -- "Git commit" --> HiddenOracle1
    DevVisible -- "Git commit" --> HiddenOracle2
    HiddenOracle1 -- "Functional test results" --> AuditChannel
    HiddenOracle2 -- "Stress-test results (async backflow)" --> AuditChannel
    AuditChannel -- "Audit pass / veto" --> Verdict["Final physical battle report"]
```

9.1 Implementation Specification for the Stress-Test Hidden Oracle

```typescript
interface StressTestOracle {
  // Load model (hidden from the agent)
  loadProfile: {
    qpsGradient: number[];        // QPS gradient [100, 500, 1000, 5000]
    durationPerStep: number;      // duration per tier (seconds)
    payloadTemplates: string[];   // request load templates (including boundary values)
  };

  // SLA thresholds (hidden from the agent)
  slaThresholds: {
    p99LatencyMs: number;         // P99 latency upper bound
    errorRateMax: number;         // error-rate upper bound
    saturationPoint: number;      // saturation-point calibration
  };

  // Environment fingerprint (used for homogeneity verification)
  environmentFingerprint: {
    osVersion: string;
    kernelVersion: string;
    dependencyVersions: Record<string, string>;
    resourceQuota: { cpu: string; memory: string };
  };
}

class IsolatedStressTestEvaluator {
  async evaluateStressTest(
    agentBranch: string,
    oracle: StressTestOracle
  ): Promise<StressTestResult> {
    // 1. Deploy the candidate version in the isolated environment
    const deployment = await deployToIsolatedEnv(agentBranch, {
      environmentFingerprint: oracle.environmentFingerprint,
      networkIsolation: true,  // physically isolated from the development environment
    });
```

```
    // 2. Execute the stress test (independent executor, not under agent control)
    const stressResult = await runIsolatedStressTest(deployment, {
      loadProfile: oracle.loadProfile,
      durationPerStep: oracle.durationPerStep,
    });

    // 3. Collect raw metrics (native Prometheus/JMeter export)
    const rawMetrics = await exportRawMetrics(stressResult);

    // 4. Generate the environment fingerprint report
    const envFingerprint = await captureEnvironmentFingerprint(deployment);

    // 5. Asynchronous backflow via the protocol bridge (Pattern 12)
    return {
      passed: this.checkSLA(rawMetrics, oracle.slaThresholds),
      rawMetrics: rawMetrics,           // raw metrics, not an agent-reported paraphrase
      environmentFingerprint: envFingerprint,
      integrityHash: computeHash(rawMetrics),  // transmission integrity verification
    };
  }
}
```

9.2 Key Implementation Constraints

Constraint	Implementation
Load-model hiding	The stress-test configuration is stored in the isolated environment; the agent only knows that "a stress test will be conducted", and does not know the concrete QPS gradient, load templates, or SLA thresholds
Asynchronous result backflow	Stress-test results flow back asynchronously through the protocol bridge (Pattern 12), rather than in real-time synchronization
Transmission integrity	Results carry a SHA-256 hash; only after the auditor (Pattern 02 Probe 4) verifies integrity may they be accepted
Environment homogeneity	The deviation between the stress-test environment configuration fingerprint and the production environment must be $\leq$ a preregistered tolerance (e.g., kernel version ± 1 minor, exact dependency-version matching)
Result immutability	The stress-test environment is completely read-only to the agent; results are stored in WORM (Write Once Read Many) storage

9.3 Synergy with Patterns 19/20

- **Pattern 19 extension**: stress-test results serve as a **sixth independent criterion** (criterion 6: stress-gradient performance-decay criterion), standing alongside the functional five-fold criteria;
- **Pattern 20 extension**: a three-dimensional control matrix — Channel A (baseline version, functional + performance), Channel B (candidate version, functional + performance), Channel C (stress-test environment performance-decay curve).

  Theoretical anchor: the extension of the "evaluation decoupling" of Theorem 3.1 to the isolated-environment scenario — functional evaluation and performance evaluation are completely decoupled in physical environment, executing subject, and result channel.

Chapter 22: Pattern 19 — Five-Fold Bypass-Free Evaluator Pattern

---

1. Intent

To establish a **bypass-free mechanical criteria system (Bypass-Free Mechanical Validator, Kapoor et al., 2024)** built on strict constraints along five physical dimensions, breaking the **reward hacking (Amodei et al., 2016)** and **specification gaming (Krakovna et al., 2020)** caused by relying on LLM self-assessment (LLM-as-a-Judge) or single-point exit codes in traditional evaluation, so that a determination of agent problem-solving success is 100% industrially verifiable and production-deployable.

---

2. Motivation & Real-World Failure Modes

In the benchmark leaderboards and capability evaluation of autonomous software agents (SWE Agents, Yang et al., NeurIPS 2024), mainstream evaluation benchmarks are universally mired in the quagmire of "formalistic self-deception":

- **Failure scenario A (Sycophancy & Self-Scoring Illusion)**: Using the large model itself as the judge (LLM-as-a-Judge). Owing to the sycophancy tendency of large language models (Sharma et al., 2023) and confirmation bias toward its own generated code, the model frequently awards inflated scores of 90+ to code riddled with defects that cannot even compile in a real physical environment;
- **Failure scenario B (Exit Code Bypass)**: The evaluation system depends solely on whether `echo $?` from a single script is 0. When the agent finds that unit tests fail, it can easily game an Exit 0 (falsified pass) by overwriting the execution script, injecting an exit code into environment variables, or adding `@Disabled` to test assertions;
- **Failure scenario C (Isolated Mock Illusion)**: The agent has only run the logic within the pure memory of unit tests; but once the service is actually started, a database-migration script syntax error, a middleware connectivity timeout, or a Spring circular dependency occurs, and the service cannot listen normally on the physical port at all.

```
flowchart TD
    AgentSubmission["Agent submits deliverables (Git Commit & branch reference)"] --> FiveFoldEng

    subgraph TheFivePillars["The Five Mechanical Pillars — physical bypass-free rigid criteria"]
        direction TB
        Pillar1["<b>Criterion 1: Substrate entity source-code diff (Substrate Diff)</b><br/><i>Th
```

```
    Pillar2["<b>Criterion 2: In-situ native multi-module build penetration (In-Situ Native Bu
    Pillar3["<b>Criterion 3: Hidden Oracle unit tests 100% green (Hidden Oracle Tests)</b><br
    Pillar4["<b>Criterion 4: Independent external black-box HTTP probe penetration (External
    Pillar5["<b>Criterion 5: Formal architecture & security audit gate (Formal Audit Gate)</b
end

FiveFoldEngine --> TheFivePillars

TheFivePillars -- "Any single criterion fails (hard hang)" --> FalsifiedVerdict["Judged as fa
TheFivePillars -- "All five criteria green (full physical pass)" --> ResolvedVerdict["Judged
```

---

3. Applicability

- Objective capability evaluation systems and industrial leaderboards (ForgeBench, etc.) for autonomous software-engineering agents (SWE Agents);
- The absolute physical acceptance defense line before merging enterprise core production code (Pull Request Merge);
- Evaluating real systems with complex dependencies such as multi-service interaction, database interaction, and port listening.

---

4. Participants

1. **Mechanical Arbiter**: the highest judgment hub, physically isolated from the problem-solving agent; it recognizes only objective physical facts and accepts no natural-language excuses;
2. **Substrate Diff Sensor**: directly invokes `git status --porcelain` and `git diff`, extracts physically modified files and validates them against the whitelist;
3. **Native In-Situ Builder**: invokes real build tools under the physical root directory of the isolated sandbox, and records the true physical elapsed time and the compiler console stream;
4. **Hidden Oracle Injector**: performs read-only injection of an invisible golden-assertion test suite;
5. **Black-box HTTP/Socket Prober**: sends real network packets to the running service port and validates the return codes and data fingerprints.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Evaluator as Evaluation engine (Mechanical Arbiter)
    participant Repo as Physical workspace of the target project source
    participant Compiler as Native compiler (Native Maven/NPM)
    participant Prober as External mechanical network probe
    participant Gate as Independent architecture auditor

    Evaluator->>Repo: Criterion 1: read the physical Git diff (verify whether the entity change b
```

```
Repo-->>Evaluator: confirms 2 valid source-file changes produced (PASS)

Evaluator->>Compiler: Criterion 2: execute in place mvn clean test -pl specialscale-auth -am
Compiler-->>Evaluator: native console captured: 17 modules BUILD SUCCESS, Exit 0 (PASS)

Evaluator->>Repo: Criterion 3: execute the double-blind injected Hidden Oracle test suite
Repo-->>Evaluator: run result: 8 run, 0 failures, 0 errors, 0 skipped (PASS)

Evaluator->>Prober: Criterion 4: start the container and probe http://127.0.0.1:9999/token/2f
Prober-->>Evaluator: real response HTTP 200 received, schema matched (PASS)

Evaluator->>Gate: Criterion 5: consult Jev and the code-review reality-audit verdict
Gate-->>Evaluator: Jev score 7.99, no VETO cheating flag (PASS)

Evaluator-->>Evaluator: confirm the full five-fold physical pass -> issue the RESOLVED final
```

---

6. Consequences

- **Pros**:
  - **Irrefutable objective authenticity**: eliminates all benchmark gaming and falsified passes; the scorecard can directly serve as the basis for enterprise production delivery;
  - **Extremely high evaluation robustness**: the five criteria cross-verify one another; single-point deception cannot escape.
- **Cons**:
  - Long per-problem evaluation time (including native full-module multi-compilation and real service startup/health probing, typically 1–3 minutes);
  - Requires the evaluation environment to have complete low-level physical virtualization and dependency private-server infrastructure.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be eradicated
LLM-as-a-Judge Monopoly	Letting the large model glance at the code and then render a pass/fail textual verdict	It belongs to the non-physical untrustworthy paradigm, with huge sycophancy and hallucination vulnerabilities
Exit Code Laundering	Using a wrapper script to catch test failures and force `exit 0`	Causes severe regression defects to escape into the production environment

---

8. Algorithmic Formulation

```typescript
interface FiveFoldVerdict {
  isResolved: boolean;
```

```
  substrateDiffPass: boolean;
  nativeBuildPass: boolean;
  hiddenOraclePass: boolean;
  httpProbePass: boolean;
  formalAuditPass: boolean;
  metrics: { durationMs: number; rawExitCode: number };
}

class FiveFoldMechanicalEvaluator {
  async evaluate(submissionWorkspace: Workspace, oracleSpec: HiddenOracleSpec): Promise<FiveFoldV
    const startTime = Date.now();

    // 1. Substrate entity source-code change check (Substrate Check)
    const diff = await submissionWorkspace.getGitDiff();
    const hasValidDiff = diff.some(f => f.matches(oracleSpec.targetModulePattern));

    // 2. Native build in-situ penetration (In-Situ Build)
    const buildResult = await submissionWorkspace.runNativeBuild();

    // 3. Execute Hidden Oracle tests (no skip parameters)
    await submissionWorkspace.injectHiddenOracleReadOnly(oracleSpec.oraclePatch);
    const testResult = await submissionWorkspace.runNativeTestsWithoutSkip();

    // 4. Independent external black-box HTTP probes
    const probeResult = await this.executeExternalHttpProbes(oracleSpec.expectedEndpoints);

    // 5. Formal architecture & security audit
    const auditResult = await this.auditFormalGate(submissionWorkspace);

    const isResolved = hasValidDiff &&
      buildResult.exitCode === 0 &&
      testResult.failures === 0 && testResult.skipped === 0 &&
      probeResult.every(p => p.success) &&
      auditResult.passed && !auditResult.isVetoed;

    return {
      isResolved,
      substrateDiffPass: hasValidDiff,
      nativeBuildPass: buildResult.exitCode === 0,
      hiddenOraclePass: testResult.failures === 0 && testResult.skipped === 0,
      httpProbePass: probeResult.every(p => p.success),
      formalAuditPass: auditResult.passed,
      metrics: { durationMs: Date.now() - startTime, rawExitCode: buildResult.exitCode }
    };
  }
}
```

9. Epistemic Status

> **Engineering Convention**: this pattern is a non-falsifiable operational norm, not a theoretical conclusion. Its correctness can only be judged by "whether reviewers execute the conjunction of the five criteria"; the pattern itself provides no statistical criterion, nor does it constitute a corollary of Axioms I–IV or Theorem 3.1.

The conjunctive structure of the five criteria and the `Jev` $\geq$ `7.0` threshold are review norms, not validated effect sizes. If they are to be used as a conclusion, a formal definition of "bypass-free" must first be given, and the false-positive and false-negative rates of each criterion must be empirically measured. In the §8 matrix of the validation report, this pattern is classified as an engineering convention; its formulation must not serve as a reasoning premise for other patterns.

Chapter 23: Pattern 20 — Industrial Substrate A/B Benchmark Pattern

---

1. Intent

In a real industrial-engineering physical sandbox that is strictly homogeneous (identical hardware, network, and dataset), establish a **double-blind A/B physical controlled-evaluation matrix across multiple execution channels**. By eliminating external perturbation variables in evaluation, precisely measure the true efficiency gap of different underlying models (LLM backends) and different architecture orchestration strategies (e.g., monolithic CLI agents vs. native cybernetic channels) in terms of objective resolution rate, tokenomics energy-consumption convergence, and physical elapsed time.

---

2. Motivation & Real-World Failure Modes

In the field of autonomous-agent performance evaluation, academia and enterprises frequently publish comparative test reports claiming "large SOTA surpassions", yet their conclusions are often worthless in real industrial scenarios:

- **Failure scenario A (Apples-to-Oranges Comparison)**: Channel A runs on a local machine equipped with a full local private-server cache, taking only 10 seconds; Channel B runs in a clean container that must slowly download dependencies from the public internet, taking 5 minutes. The evaluator draws the pseudoscientific conclusion that "Channel A is 30× more efficient", essentially conflating the causality of network infrastructure with model capability;
- **Failure scenario B (Open-vs-Closed Book Asymmetry)**: When evaluating a proprietary agent, that agent has built in the entire codebase of the target open-source repository or a specialized prompt (open book); while when testing a generic model, only a bare-bones requirements description is provided (closed book), producing an unfair objective evaluation;
- **Failure scenario C (Failure to Normalize Tokenomics)**: Both channels pass the same problem, but Channel A consumes 26 million tokens and retries 50 rounds; Channel B, relying on cybernetic slicing, consumes only 4 million tokens and passes on the first try. Looking only at "Pass/Fail" completely conceals the commercial energy-efficiency barrier of engineering deployment.

```
flowchart TD
    TaskInstance["Standard industrial problem instance (e.g., pigx__auth__005: 5.10.0-SNAPSHOT)"]
```

```
subgraph ChannelA["Execution Channel A: native cybernetic channel (agy-native)"]
    direction TB
    A_Runtime["Native provider direct connection (Zero CLI Process Overhead)"]
    A_Orchestration["PoLP context slicing + paired micro-loop self-healing"]
end

subgraph ChannelB["Execution Channel B: traditional baseline channel (Baseline Channel)"]
    direction TB
    B_Runtime["External CLI independent-process bridging"]
    B_Orchestration["Full-prompt single-step generation"]
end

MatrixOrchestrator --> ChannelA
MatrixOrchestrator --> ChannelB

ChannelA --> SubstrateA["Sandbox A: physical workspace (homogeneous settings.xml)"]
ChannelB --> SubstrateB["Sandbox B: physical workspace (homogeneous settings.xml)"]

SubstrateA --> MechanicalEvaluator["Five-fold physical bypass-free mechanical evaluator"]
SubstrateB --> MechanicalEvaluator

MechanicalEvaluator --> ABReport["Produce a standard A/B comparative efficiency battle report
```

---

3. Applicability

- Objective comparison when enterprises select open-source/commercial large models as the core engine for code assistance;
- Validating whether agent architecture innovations (e.g., introducing PoLP projection or paired loops) bring real industrial energy-efficiency gains;
- Building third-party autonomous-programming competition platforms with anti-cheating and auditable properties.

---

4. Participants

1. **Homogeneous Sandbox Factory**: ensures that, before every channel starts, the target project source tree, Maven private-server mirror sources, JDK version, and initial Git commit hash are 100% absolutely identical across all channels;
2. **Channel Driver Adapters**: translate the unified evaluation instruction into launch recipes for different channels (e.g., `agy-native`, `deepseek-cli`, `kimi`, `amdgpu`);
3. **Tokenomics & Performance Telemetry**: real-time capture of each channel's input token count, output token count, tool-call frequency, physical compilation time, and process resident memory over the lifecycle of a single problem;
4. **Normalized A/B Reporter**: computes the per-line-of-code energy-consumption ratio (Tokens / LoC) and the per-effective-fix cost.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Arbiter as Benchmark evaluation orchestrator
    participant SandboxA as Sandbox A (Channel: agy-native)
    participant SandboxB as Sandbox B (Channel: baseline)
    participant Evaluator as Five-fold physical mechanical criteria machine

    Arbiter->>SandboxA: launch evaluation instance (Instance: pigx__auth__005, Channel: agy-nativ
    SandboxA->>SandboxA: run the 11 SDLC phases (record token throughput and physical timestamps)
    SandboxA-->>Arbiter: submit artifact A (elapsed 47s, consumed 4.2M tokens)

    Arbiter->>SandboxB: launch the same instance (Instance: pigx__auth__005, Channel: baseline)
    SandboxB->>SandboxB: run the baseline pipeline
    SandboxB-->>Arbiter: submit artifact B (elapsed 180s, consumed 21M tokens)

    Arbiter->>Evaluator: trigger the double-blind five-fold mechanical criteria
    Evaluator->>SandboxA: verify artifact A -> all pass (RESOLVED)
    Evaluator->>SandboxB: verify artifact B -> all pass (RESOLVED)

    Arbiter->>Arbiter: generate the comparative battle report: agy-native achieves -80.0% energy-
```

---

6. Consequences

- **Pros**:
  - **Scientifically rigorous causal inference**: eliminates external interference from network, hardware fluctuations, or open-book testing; conclusions withstand peer review and industrial audit;
  - **Reveals real commercial value**: lets architects clearly see the real money (token cost) saved by cybernetic optimization in production deployment.
- **Cons**:
  - Evaluation cost multiplies (each problem requires multiple channels to independently run a full compilation and evaluation);
  - Requires maintaining runtime environments and dependencies for multiple heterogeneous model channels.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be eradicated
Heterogeneous Sandbox Bias	Channel A uses the private-server cache, while Channel B connects to the public internet	What is evaluated is not model intelligence, but network jitter and disk I/O speed

Anti-pattern	Manifestation	Why it must be eradicated
Tokenomics Blindness	Ignoring the tens of millions of tokens consumed per problem, and simply claiming "both have a 100% resolution rate"	In commercial production, extremely high energy consumption means fundamentally no scalable adoption

---

8. Algorithmic Formulation

```typescript
interface ChannelRunMetrics {
  channelName: string;
  isResolved: boolean;
  totalTokens: number;
  durationMs: number;
  codeLinesChanged: number;
}

class SubstrateAbBenchmarkRunner {
  async runAbComparison(instanceId: string, channels: string[]): Promise<AbComparisonReport> {
    const results: ChannelRunMetrics[] = [];

    for (const ch of channels) {
      // 1. Initialize a fully homogeneous, self-contained physical sandbox
      const sandbox = await this.sandboxFactory.createCleanSandbox({
        instanceId,
        channel: ch,
        forceCleanSettings: true
      });

      // 2. Run the single-channel pipeline and collect energy-consumption metrics
      const runTrace = await sandbox.executePipeline();

      // 3. Run the five-fold bypass-free mechanical evaluation
      const evalVerdict = await this.evaluator.evaluate(sandbox.workspace);

      results.push({
        channelName: ch,
        isResolved: evalVerdict.isResolved,
        totalTokens: runTrace.tokenUsage.total,
        durationMs: runTrace.durationMs,
        codeLinesChanged: runTrace.diffStats.totalLoC
      });
    }

    // 4. Compute efficiency ratios and compression rates
    return this.generateReport(results);
```

```
  }
}
```

---

9. Epistemic Status of Quantitative Claims

Disposition: pending supplementary measurement (quota required) — "−80.0% energy-consumption compression and a 3.8x speedup ratio" comes from an example run of the homogeneous-sandbox A/B; the sample size and number of repetitions were not preregistered. It is now marked as an **example observation**; the formal supplementary test requires a paired A/B (n≥10, same model, same batch), will consume model quota, and is scheduled to run after the current CROSS_TASK experiments.

Chapter 24: Pattern 21 — Tokenomics Convergence Architecture Pattern

---

1. Intent

Through a multi-level engineering combination of "task-level least-privilege projection (PoLP)", "incremental fragment reuse", "causal pruning of phase state", and "local atomic retries (Sub-stepping Retry)", achieve a **cliff-shaped reduction in total token throughput in complex enterprise microservice software engineering development (measured reduction of −80.7%)**, significantly alleviating context rot (Liu et al., TACL 2024; Chroma, 2025) while bringing the comprehensive economic cost of autonomous-agent R&D across the commercial profitability critical point.

---

2. Motivation & Real-World Failure Modes

Many agent systems perform acceptably in academic demo scenarios, but when handling real enterprise-grade engineering, they often encounter an unbearable "token throughput inflation crisis":

- **Failure scenario A (Tokenomics Hyperinflation)**: In solving a single benchmark complex microservice problem (e.g., gateway multi-tenant rate-limiting routing), because every interaction re-injects the entire project's POMs, all configuration files, and the long history of the conversation, the per-step prompt exceeds 100,000 tokens. A task requiring only a 30-line code change consumes over 26 million tokens in total, with a single-problem inference bill of dozens of dollars, completely losing commercial viability;
- **Failure scenario B (Context Rot Induced Degradation)**: Empirical studies show (Hsieh et al., 2024, RULER; Chroma, 2025) that the effective context window of large models is far below its nominal value. As input tokens climb above 50K, the model's micro-bug identification rate in code plummets, producing the vicious cycle of "the longer the context, the worse the edits";
- **Failure scenario C (Blind Truncation Damage)**: To save tokens, many systems simply truncate characters from the head or tail. Such semantically unaware hard truncation often cuts off key JSON brackets, class definitions, or import declarations, causing the model to directly output syntactically garbled code.

```
flowchart TD
    subgraph TraditionalBloat["Traditional unconstrained interaction pattern (Token explosion: 26
        direction TB
        RawContext["Full project code + long historical logs"] --> BloatedPrompt["100K+ tokens pe
        BloatedPrompt --> ModelDegrade["Context rot & frequent errors"]
        ModelDegrade --> MassiveRetries["50 consecutive full retries (uncontrolled throughput)"]
    end

    subgraph CyberneticCompression["Cybernetic energy-efficiency compression architecture (Token
        direction TB
        TaskSlicing["1. Task-level least-privilege projection (PoLP: inject only the single-task
        FragmentCache["2. Incremental reuse of disk fragments (reuse the verified T_k.json, skip
        CausalPruning["3. Causal pruning (clear stale error stacks, keep only targeted diagnosis)
        SubstepHeal["4. Local micro-loop atomic self-healing (Driver-Navigator confined to local
    end

    CyberneticCompression --> UltraLowTokens["Per-line-of-code energy consumption greatly optimiz
```

---

3. Applicability

- Autonomous code-generation platforms deployed at enterprise large scale on-premises or running at high concurrency on the public cloud;
- Cost-sensitive commercial products that need strict control over the economics of a single change;
- High-availability systems that need to significantly shorten end-to-end delivery latency and avoid network transmission delay.

---

4. Participants

1. **Tokenomics Router**: responsible for dynamically monitoring the token consumption rate at each pipeline phase and enforcing energy-consumption budget (Budget Cap) control;
2. **Context Pruning Engine**: analyzes the causal dependencies of the call chain and actively removes historical conversations and intermediate temporary attempts orthogonal to the current action;
3. **Structured Fragment Cache**: solidifies outputs that have passed the PM gate or the Design gate as strong-contract fragments in the filesystem, so that subsequent steps read local disk directly at zero token cost;
4. **Attention Fidelity Assessor**: ensures that, while compressing context to the extreme, 100% integrity of core architectural decisions and boundary constraints is maintained.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Orchestrator as Orchestration system
```

```
participant Pruner as Context pruning engine
participant Cache as On-disk structured fragment store
participant LLM as Underlying large-model gateway
participant Telemetry as Energy-consumption telemetry

Orchestrator->>Cache: Check whether task [T1] historical contract exists and is valid?
Cache-->>Orchestrator: Hit the local verified fragment T1.json (Zero Token Hit!)

Orchestrator->>Pruner: Prepare the development context for subtask [T2]
Pruner->>Pruner: Action 1: remove the full macro topology, project only the gateway module bo
Pruner->>Pruner: Action 2: prune the build logs of previously compile-passing modules (80% re
Pruner-->>Orchestrator: Output a high-fidelity minimal prompt (only 4K tokens)

Orchestrator->>LLM: trigger a lightweight inference call
LLM-->>Orchestrator: return a precise code patch

Orchestrator->>Telemetry: report this call's metrics: input 4,200, output 180
Telemetry->>Telemetry: confirm the energy metrics meet the convergence baseline (Tokenomics H
```

---

6. Consequences

- **Pros**:
  - **Cliff-shaped total throughput reduction**: in complex microservice empirical evaluation, per-problem total throughput dropped from **26.86M tokens to the 5M level (reduction −80.7%)**, and per-line consumption from 51K tokens/LoC to 9,700 tokens/LoC;
  - **Multiplied inference speed**: since the per-request token count is reduced by over 70%, time-to-first-token (TTFT) and model generation time are greatly shortened, and task end-to-end time drops by 3–5×;
  - **Eradicated context rot**: prompts stay in the low-entropy, high signal-to-noise-ratio region, and the precision of model code generation improves significantly.
- **Cons**:
  - Context pruning algorithms require deep understanding of code syntax and engineering topology, imposing extremely high middleware-capability requirements on the orchestrator.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be eradicated
Endless Conversation Chat History	Stacking every stack trace from multi-round failed attempts verbatim as history turns	Quickly blows up the context window and induces severe context rot (Liu et al., 2024)
Blind Character Truncation	Truncating solely by character count via `prompt.substring(0, 8000)`	Destroys the semantic closure of code structure, causing frequent syntax errors

---

8. Algorithmic Formulation

```
interface TokenomicsBudget {
  maxInputTokensPerStep: number;
  totalTaskBudget: number;
}

class TokenomicsConvergenceEngine {
  private budget: TokenomicsBudget;

  compressContext(task: TaskItem, rawContext: FullEngineeringContext): CompressedPromptContext {
    // 1. Least-privilege projection (PoLP Projection)
    const scopedFiles = rawContext.files.filter(f => task.targetPaths.includes(f.path));

    // 2. Fine-grained compiler console pruning (keep the most recent 20 lines of core errors, re
    const sanitizedErrorLogs = this.pruneBuildConsoleLogs(rawContext.latestStderr);

    // 3. Architectural-contract referencing (inject only interface signatures, omit internal imp
    const interfaceSignatures = this.extractPublicSignatures(rawContext.importedInterfaces);

    const compressedPrompt = {
      taskSpec: task.summary,
      scopedFiles,
      interfaceSignatures,
      relevantError: sanitizedErrorLogs
    };

    // 4. Budget guard check
    const estimatedTokens = this.estimateTokens(compressedPrompt);
    if (estimatedTokens > this.budget.maxInputTokensPerStep) {
      throw new Error(`TOKENOMICS_BUDGET_EXCEEDED: ${estimatedTokens} > ${this.budget.maxInputTok
    }

    return compressedPrompt;
  }
}
```

---

9. Epistemic Status of Quantitative Claims

> **Disposition: retained as empirically measured (calibration to be completed)** — "−80.7%" has empirical support, but the text does not record the task family, model version, sample size, or the statistical definition of "total throughput"; "9,700 token/LoC" is a ratio derived from that experiment. Until the above calibration is filled in, this result may only serve as an **empirically measured value on a single task family** and must not be extrapolated to other task families or models.

Chapter 25: Pattern 22 — Pure Native Provider Bridge Pattern

---

1. Intent

Decouple and restructure the agent pipeline's invocation of the large language model (LLM) from "launching a heavyweight external independent CLI process (Subprocess Execution)" into a **"lightweight resident native provider gateway protocol bridge (Pure Native In-Memory Provider)"**, eliminating the high OS process jitter (Process Spawning Jitter), memory-resident overhead, and permission-popup friction caused by cross-process launching, and achieving pure atomic access to model capability with zero external-process intrusion.

---

2. Motivation & Real-World Failure Modes

In many early engineering practices, to reuse off-the-shelf desktop CLI tools (e.g., various commercial or open-source coding agent CLIs), architects typically adopted the "Shell subprocess invocation" approach: every time the pipeline needed a model generation, it executed `execFile("agent-cli.exe -p 'prompt'")` underneath. This "process-level wrapping" (Subprocess Wrapping) has severe drawbacks in industrial production environments:

- **Failure scenario A (Process Cold-Start Penalty)**: In typical paired micro-loop development (Pair-Dev), 50 tiny tool interactions may occur between the driver and the navigator within 10 minutes. If each interaction cold-starts a binary containing a V8 engine or an Electron runtime, merely process loading, dynamic-library linking, and Node environment initialization wastes 2–4 seconds of dead time, doubling the pipeline's end-to-end time;
- **Failure scenario B (Privilege Prompt Blockade)**: Many desktop CLIs default to an interactive permission security sandbox (e.g., requiring the user to press Enter or click confirm in the terminal before file writes execute). Once the pipeline runs automatically in a headless backend (Headless CI/CD), the CLI process hangs forever waiting for standard input (Stdin), causing infinite-timeout deadlocks in benchmarking or build tasks;
- **Failure scenario C (Stdio Pipe Fragility)**: Cross-process text communication relies on OS pipes. When encountering oversized file diffs or special Unicode characters, pipe-buffer blocking, newline differences (CRLF vs LF), or the interleaving of debug information (Stdout and Stderr in mixed flow) easily triggers JSON deserialization parse crashes.

```
flowchart TD
    subgraph AntiPattern["Anti-pattern: heavyweight external CLI process invocation (Subprocess W
```

```
    direction TB
    Task["Single pipeline-step subtask"] --> SpawnCLI["Create a child process every time: exe
    SpawnCLI --> ProcessJitter["Process cold-start jitter (+3s) & pipe deadlock risk"]
    ProcessJitter --> PromptBlock["Interactive permission popups block the automated pipeline
end

subgraph NativeBridgePattern["Recommended pattern: Pure Native Provider Bridge Pattern (Pure
    direction TB
    PipelineStep["Single pipeline-step task (dsh / Core Orchestrator)"] --> LocalGateway["Nat

    subgraph GatewayCore["Zero-process-overhead gateway core (Zero CLI Process Gateway)"]
        TokenBridge["1. Local lightweight OAuth2 token cache and direct connection"]
        StreamChunker["2. SSE streaming high-throughput chunked transfer (Server-Sent Events)
        ToolNormalizer["3. Native Tool Call structured lossless passthrough"]
    end

    LocalGateway --> GatewayCore
    GatewayCore --> UpstreamAPI["Upstream model provider pure API (Gemini / Claude / DeepSeek
    UpstreamAPI --> AtomicExecution["Zero-friction atomic ultra-fast response (average only 4
end
```

---

3. Applicability

- Scenarios requiring fully autonomous execution in automated headless CI/CD pipelines, where no human-interaction blocking is allowed;
- High-concurrency systems extremely sensitive to task end-to-end response latency, pursuing millisecond-level ultra-fast interaction;
- Hybrid-cloud architectures that need to smoothly bridge third-party or enterprise-private model services into standard OpenAI / Anthropic-compatible interfaces.

---

4. Participants

1. **Orchestrator Host**: e.g., the autonomous pipeline runtime core; it only issues standard HTTP/REST or gRPC structured requests and no longer cares what underlying binary software lies beneath;
2. **Native Provider Gateway Daemon**: a lightweight process resident in the system backend, responsible for maintaining communication sessions with large-model service providers, credential rotation, and proxy routing;
3. **Tool Call Normalizer**: performs bidirectional in-memory mapping between the large model's `function_call` and the platform-native tool protocol, without writing temporary JSON files to disk;
4. **Full-Duplex Stream Transceiver**: uses the Server-Sent Events (SSE) protocol to stream inference results, driving downstream state machines as soon as the model's first token is generated.

---

5. Collaborations

```
sequenceDiagram
    autonumber
    actor Core as Pipeline orchestration core (dsh)
    participant Gateway as Native provider gateway (127.0.0.1:18889)
    participant Model as Upstream large-model pure API
    participant Substrate as Physical filesystem

    Note over Gateway: The resident backend daemon keeps long-lived connections and OAuth credent

    Core->>Gateway: POST /v1/chat/completions (streaming request, declaring the 22 required tools
    Gateway->>Model: Direct API handshake (eliminates CLI cold start; median TTFT 0.8s, shorter t

    Model-->>Gateway: SSE streams back tool_call: [write_to_file("TwoFactorAuth.java")]
    Gateway-->>Core: Structured lossless delivery of the function call

    Core->>Substrate: Perform an in-place file mutation
    Core->>Gateway: POST /v1/chat/completions (submit the tool-return result)
    Gateway->>Model: continue the next inference round (connection reuse)
```

---

6. Consequences

- **Pros**:
  - **Eliminated process startup jitter**: interaction latency compressed from the traditional 8–12 seconds to **4–5 seconds (doubled efficiency)**;
  - **100% unblocked interactive-permission deadlocks**: freed from the complex permission-interception logic inside desktop CLI tools; the fully automated pipeline enjoys a 100% self-healing pass rate;
  - **Extremely pure system decoupling**: the host orchestration layer and the large-model execution layer are completely decoupled via standard network protocols, allowing seamless switching of underlying model providers at any time.
- **Cons**:
  - Requires maintaining a resident backend gateway daemon (Daemon Process) locally;
  - Requires self-managing the token cache and the network-disconnect auto-reconnection mechanism.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be eradicated
Nested Subprocess Spaghetti	PowerShell calls Node, Node calls CLI, and CLI calls Python	The process chain is too long; once an intermediate pipe goes wrong, the error code is completely lost

Anti-pattern	Manifestation	Why it must be eradicated
Desktop Interactive Lock-in	Mandating human-confirmation privilege escalation in an environment with a GUI or a TTY terminal	CI/CD automation cannot be achieved in standard headless Docker containers or production servers

---

8. Algorithmic Formulation

```typescript
import * as http from "http";

interface ChatCompletionRequest {
  model: string;
  messages: Array<{ role: string; content: string }>;
  tools?: object[];
  stream?: boolean;
}

class PureNativeProviderGateway {
  private port: number = 18889;
  private oauthTokenManager: OAuthTokenManager;

  startDaemon(): void {
    const server = http.createServer(async (req, res) => {
      if (req.url === "/health" && req.method === "GET") {
        res.writeHead(200, { "Content-Type": "application/json" });
        return res.end(JSON.stringify({ status: "ok", provider: "native-decoupled", port: this.po
      }

      if (req.url === "/v1/chat/completions" && req.method === "POST") {
        const body = await this.readRequestBody(req);
        // Obtain valid credentials and directly issue an async HTTPS streaming direct connection
        const modelStream = await this.forwardToUpstreamNativeApi(body);

        res.writeHead(200, {
          "Content-Type": "text/event-stream",
          "Cache-Control": "no-cache",
          "Connection": "keep-alive"
        });

        // Direct pipe-through, zero on-disk temporary files
        modelStream.pipe(res);
      }
    });

    server.listen(this.port, "127.0.0.1", () => {
      console.log(`[native-provider] Zero-CLI Daemon listening at http://127.0.0.1:${this.port}`)
```

```
    });
  }
}
```

---

9. Epistemic Status of Quantitative Claims

Original framing of the claim (prior to any measurement): both "`<800ms`" and "8–12 seconds → 4–5 seconds" come from a comparison of the same model on the **same provider (agy)** — the native provider gateway directly connecting to the agy cloud API **vs** CLI invocation of the agy command line. What it tests is the difference in **access mode** (direct connection vs CLI wrapping), **not** a cross-vendor, cross-model horizontal performance comparison.

This measurement (2026-10-21): script `experiments/attention_growth/e_p22_ttft_probe.py`, $n = 30$ serial, short prompts.

Channel	Model	Median first byte	Median first content	Conclusion
DeepSeek official	deepseek-v4-pro	—	—	Not measurable (30/30 returned 402, balance exhausted)
scnet gateway	DeepSeek-V4-Pro-0813	1,996ms	2,653ms (p90 3,725ms)	**Out of scope, not a criterion**
Kimi	kimi-for-coding	976ms	2,041ms (p90 2,590ms)	**Out of scope, not a criterion**

Adjudication: this measurement does not constitute a test of the claim. There are three reasons—— 1. **Different subjects**: the subject of the original claim was the agy cloud API and agy CLI; this test used DeepSeek / scnet / Kimi, i.e., different providers, and the original text already declared "no horizontal-comparison significance with other large-model service providers". 2. **Missing control arm**: the original claim is a relative quantity (native gateway vs CLI wrapping); this test has only the direct-connection side, so the difference and the ratio cannot be computed. 3. **Different calibration**: "`<800ms`" refers to time-to-first-token, while "8–12 seconds → 4–5 seconds" refers to end-to-end single-round interaction latency; the two must not be mixed.

Therefore the only valid output of this test is negative information: the direct-connection TTFT of other providers cannot be used to generalize that threshold — `800ms` is an empirically measured value on agy's specific link, not a universal metric. For reasoning models, direct-connection first-token time is generally on the order of 0.98–2.65s, and "first token" must distinguish three layers: first byte / first reasoning delta / first content token (during Kimi's first measurement, the 16-token

budget was exhausted by reasoning, and content never appeared, leaving the metric undefined).

Falsifiability condition (correct form: relative quantity, not absolute value): the substance of the original claim is the latency difference brought by the **access mode**, so the criterion should be built on a **paired comparison** of the same provider, the same model, and the same prompt, rather than on some absolute threshold.

Let the native-gateway time-to-first-token of the i-th request be T_i^{nat} and the CLI path be T_i^{cli}; then

$$\Delta_i = T_i^{\text{cli}} - T_i^{\text{nat}}, \qquad \rho = \text{median}(T^{\text{cli}}) \,\big/\, \text{median}(T^{\text{nat}})$$

- **H0 (claim rejected)**: the median of Δ is not greater than 0 (the native side is not faster); or the paired sign test gives $p \geq 0.05$.
- **H1 (claim holds)**: the median of Δ is significantly greater than 0, and ρ reaches the preset multiple (e.g., $\rho \geq 1.5$).

The absolute quantity `800ms` is only a **descriptive reading** from one measurement on this provider and should not serve as a criterion — different providers, different models, and whether a proxy is in use are not comparable. Likewise, "8–12 seconds $\rightarrow$ 4–5 seconds", as a **ratio** of end-to-end latency (about 2×), is the comparable part; its absolute value likewise varies with provider and task length.

This experiment requires agy credentials and a CLI environment, which this repository currently lacks, so it is marked as **externally measured and not reproducible in this repository**, serving only as an empirical fact to be explained and not as a reasoning premise of this system.

Chapter 26: Pattern 23 — Production Runtime Anomaly Sniffing Pattern

---

1. Intent

Extend physical negative feedback from "pre-deployment" to "post-deployment," and through **production runtime metrics collection, anomaly-pattern sniffing, and physical-clock event anchoring**, ensure that the **state observability** of an agent system remains continuously valid throughout the production phase, guarding against uncontrolled drift after deployment.

---

2. Motivation & Real-World Failure Modes

After an autonomous agent has delivered code and it has been deployed to production, the closed loop of the traditional cybernetic framework comes to an end. Yet the real operating state of the production environment is the ultimate physical criterion for verifying the quality of the agent's deliverable:

- **Failure Mode A (Silent Performance Decay)**:

  The candidate version shows a P99 latency of 120 ms (passing) in the stress-test environment, but in production — owing to differences in the real data distribution, fluctuation in cache-hit rates, and downstream-service latency propagation — the P99 latency gradually climbs to 800 ms within 72 hours after deployment. The agent system has no runtime monitoring and fails to sniff this decay, discovering it passively only after user complaints;

- **Failure Mode B (Resource Leak Accumulation)**:

  Code defects introduced by the agent (e.g., database connections that are never closed, a cache strategy with a memory leak) remain hidden over the short horizon of stress testing, but accumulate during long-running production operation and eventually crash the service. The "pass" verdict of stress testing masks the long-horizon stability defect;

- **Failure Mode C (Production-Only Anomalies)**:

  Real production traffic patterns (e.g., anomalous requests from a particular region, concurrency peaks during specific time windows) trigger boundary conditions the agent did

not anticipate; these anomaly patterns are never crystallized into empirical assets, and the same class of defect recurs in subsequent versions.

```
flowchart TD
    subgraph DevEnvironment["Development environment (accessible to the agent)"]
        Dev["Agent development"] --> GitCommit["Git commit"]
    end

    subgraph IsolatedEnvironments["Isolated environment matrix (inaccessible to the agent)"]
        direction TB
        StressTest["Stress-test environment<br/>(isolated)"]
        Production["Production environment<br/>(isolated)"]
    end

    subgraph BridgeLayer["Protocol bridge layer (Pattern 12)"]
        direction TB
        Bridge1["Stress-test results<br/>asynchronous feedback"]
        Bridge2["Production metrics<br/>asynchronous feedback"]
    end

    subgraph AuditLayer["Independent audit layer (Pattern 02, Probe 4)"]
        direction TB
        Audit1["Stress-test audit"]
        Audit2["Production-metrics audit"]
    end

    GitCommit --> StressTest
    StressTest --> Bridge1
    GitCommit --> Production
    Production --> Bridge2
    Bridge1 --> Audit1
    Bridge2 --> Audit2

    Audit1 --> DeployDecision["Deployment decision"]
    Audit2 --> RuntimeSniffer["Runtime anomaly sniffer"]

    subgraph SniffingDimensions["Three sniffing dimensions"]
        D1["Performance dimension: P99/P999 latency, throughput, error rate"]
        D2["Resource dimension: memory/CPU/connection-pool/handle leaks"]
        D3["Business dimension: anomalous request patterns, boundary-condition triggers"]
    end

    RuntimeSniffer --> SniffingDimensions
    SniffingDimensions --> Alarm["Anomaly alarm & physical-clock event"]
    Alarm --> Feedback["Feedback to Pattern 24: defect crystallization & hotfix loop"]
```

Key constraint: the production environment and the development environment are fully isolated

Constraint	Implementation
The agent cannot directly access the production environment	Pattern 01 (in-situ penetration) does not apply; production metrics flow back asynchronously through the protocol bridge (Pattern 12)
Runtime metrics must be transmitted across environments	The metrics collector is deployed in production and pushes to the bridge layer through an isolated API gateway
Transmission integrity assurance	Production metrics carry a SHA-256 hash; the auditor verifies integrity before they may be trusted
Environment fingerprint collection	The production configuration fingerprint (OS / kernel / dependency versions / resource quotas) is collected synchronously for homogeneity checking
Cross-environment anchoring of physical-clock events	The mtime jump of an anomaly event is anchored in production and synchronized through the bridge layer to the development environment, ensuring causal monotonicity

---

3. Applicability

- The code delivered by an autonomous agent has been deployed to production, and runtime quality must be continuously assured;
- Production runs over long horizons, and accumulated defects that stress testing did not expose must be detected;
- Production-only anomalies must be crystallized into empirical assets, preventing the same class of defect from propagating across generations.

---

4. Participants and Responsibilities

1. **Runtime Metrics Collector**: an independent agent deployed in production that collects native Prometheus/Grafana metrics, fully physically isolated from the agent; **key constraint**: the collector has no direct communication channel with the agent, exposing only a read-only metrics endpoint through the isolated API gateway;
2. **Anomaly Sniffer**: deployed in the development environment or an independent analysis environment, it sniffs anomaly patterns in runtime metrics based on preregistered thresholds and statistical models (e.g., the 3-sigma rule, EWMA exponentially weighted moving average); **key constraint**: the sniffer does not directly access production, consuming only metrics that have flowed back through the protocol bridge and been audited;
3. **Physical Clock Event Logger**: anchors anomaly events as physical-clock events (mtime jumps); **key constraint**: production mtime jumps are synchronized through the bridge layer to the development environment, ensuring cross-environment causal monotonicity;
4. **Feedback Loop Trigger**: after an anomaly is confirmed, it triggers Pattern 24's defect crystallization and hotfix loop; **key constraint**: the trigger consumes only audited anomaly events and does not directly respond to raw production signals.

---

5. Collaborations and Causal Constraints

```
sequenceDiagram
    autonumber
    actor Prod as Production environment (isolated)
    participant Collector as Runtime Metrics Collector (production)
    participant Bridge as Protocol bridge (Pattern 12)
    participant Auditor as Independent auditor (Pattern 02, Probe 4)
    participant Sniffer as Anomaly sniffer (development)
    participant Clock as Physical-clock event logger
    participant Trigger as Feedback-loop trigger

    loop Continuous monitoring (30 s collection cycle)
        Prod->>Collector: Expose runtime metrics (/metrics endpoint, read-only)
        Collector->>Bridge: Push metrics stream (isolated API gateway, with SHA-256 hash)
        Bridge->>Auditor: Forward metrics stream + integrity-check request
        Auditor->>Auditor: Verify transmission integrity + environment homogeneity
        Auditor-->>Sniffer: Deliver audited metrics stream

        Sniffer->>Sniffer: Statistical-model detection (3-sigma / EWMA / preregistered thresholds

        alt Anomaly pattern detected
            Sniffer->>Clock: Anchor physical-clock event (anomaly timestamp + metrics snapshot)
            note over Clock: Production mtime jump synchronized through the bridge layer
            Clock-->>Trigger: Trigger feedback loop (event ID + anomaly classification)
            Trigger->>Trigger: Route to Pattern 24: defect crystallization & hotfix loop
        else Metrics normal
            Sniffer-->>Bridge: Continue monitoring (heartbeat confirmation)
        end
    end
```

Causal constraints:

- The physical clock of anomaly events must be strictly monotonic (Axiom III), with **cross-environment synchronization**: production mtime jumps are synchronized through the bridge layer to the development environment, ensuring that the causal chain of defect repair is traceable;
- The runtime-metrics collection cycle and the statistical window for anomaly detection must be preregistered, to avoid post-hoc rationalization;
- Anomaly-alarm thresholds must not be adjusted by the agent on its own; they must be confirmed by a human expert or an independent audit channel;
- **Cross-environment audit**: after production metrics flow back through the protocol bridge, they must pass Pattern 02 Probe 4's audit (authenticity / integrity / homogeneity) before the sniffer may consume them.

---

6. Consequences

- **Pros**:
  - **Continuous observability after deployment**: the production runtime state is continuously folded into the physical negative-feedback loop, preventing post-deployment loss of control;
  - **Early detection of accumulated defects**: resource leaks and performance decay over long-running operation are sniffed before a crash;
  - **Crystallization of production-specific anomalies**: boundary conditions triggered by real traffic patterns are crystallized into empirical assets, preventing recurrence of the same defect.
- **Cons**:
  - An additional monitoring agent must be deployed in production, increasing operational complexity;
  - The statistical models for anomaly detection must be calibrated periodically to avoid false positives / false negatives; **the adaptive-σ criterion suffers threshold drift under signal contamination (measured in §7)**, so leak-type defects must use a robust baseline or fixed absolute thresholds instead;
  - Collecting and storing runtime metrics incurs additional cost.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be prohibited
Monitoring agent in the same process as the agent	Monitoring code embedded in the agent runtime, so the agent can tamper with the metrics	Violates Axiom I (physical non-trustworthiness); the metrics lose objectivity
Dynamically adaptive thresholds	Anomaly-detection thresholds are automatically adjusted by the agent based on historical data	The agent can evade detection through gradual drift; thresholds must be preregistered
Adaptive-σ statistical contamination (measured)	The σ of the EWMA/3σ criterion is driven by the signal itself; a large jump inflates σ, and after the 3σ threshold drifts it swallows subsequent signals of the same kind	Measured (2026-10-22, F3): mem 3σ works for an "extremely stable baseline + large jump" (dev triggered at 31.1σ) but goes insensitive to repeated leak steps (σ inflates from 4 MB to 67 MB) — leak-type monitoring must use a robust baseline (median/MAD) or fixed absolute thresholds, and cannot rely on "preregistered adaptation" alone

Anti-pattern	Manifestation	Why it must be prohibited
No physical anchoring of anomaly events	Anomalies are recorded only in memory or log files, not anchored as physical-clock events	Violates Axiom III; the causal chain of defect repair cannot be traced

---

8. Algorithmic Formulation

```
interface RuntimeMetrics {
  timestamp: number;
  p99LatencyMs: number;
  p999LatencyMs: number;
  throughputQps: number;
  errorRate: number;
  memoryUsageBytes: number;
  cpuUsagePercent: number;
  connectionPoolActive: number;
  connectionPoolMax: number;
}

interface AnomalyEvent {
  eventId: string;
  anomalyType: "PERFORMANCE_DECAY" | "RESOURCE_LEAK" | "BUSINESS_ANOMALY";
  severity: "WARNING" | "CRITICAL";
  metricsSnapshot: RuntimeMetrics;
  physicalClockEvent: {
    mtimeBefore: number;
    mtimeAfter: number;
    monotonicJump: boolean;
  };
  preRegisteredThreshold: {
    metric: string;
    threshold: number;
    statisticalModel: "3SIGMA" | "EWMA" | "FIXED";
  };
}

class ProductionRuntimeAnomalySniffer {
  private preRegisteredThresholds: Map<string, Threshold>;

  async sniffAnomaly(metrics: RuntimeMetrics): Promise<AnomalyEvent | null> {
    // 1. Performance dimension: does P99 latency exceed the preregistered threshold?
    const p99Threshold = this.preRegisteredThresholds.get("p99LatencyMs");
    if (metrics.p99LatencyMs > p99Threshold.value) {
    return this.createAnomalyEvent("PERFORMANCE_DECAY", "CRITICAL", metrics, p99Threshold);
    }

   // 2. Resource dimension: does connection-pool usage exceed 90% (accumulated leak signal)?
```

```
    const poolUsage = metrics.connectionPoolActive / metrics.connectionPoolMax;
    if (poolUsage > 0.9) {
    return this.createAnomalyEvent("RESOURCE_LEAK", "WARNING", metrics, poolThreshold);
    }

   // 3. Business dimension: does the error rate exceed the 3-sigma statistical threshold?
   const errorRateSigma = this.calculateSigma(metrics.errorRate, historicalErrorRates);
    if (errorRateSigma > 3) {
    return this.createAnomalyEvent("BUSINESS_ANOMALY", "WARNING", metrics, errorRateThreshold);
    }

    return null; // no anomaly
  }

  private createAnomalyEvent(
    type: string,
    severity: string,
    metrics: RuntimeMetrics,
    threshold: Threshold
  ): AnomalyEvent {
    // anchor physical-clock event (Axiom III)
    const mtimeBefore = this.getLastEventMtime();
    const mtimeAfter = Date.now();

    return {
      eventId: generateUUID(),
      anomalyType: type,
      severity: severity,
      metricsSnapshot: metrics,
      physicalClockEvent: {
        mtimeBefore: mtimeBefore,
        mtimeAfter: mtimeAfter,
        monotonicJump: mtimeAfter > mtimeBefore, // must be strictly monotonic
      },
      preRegisteredThreshold: threshold,
    };
  }
}
```

Theoretical anchors

Extension of Axiom I — the production "substrate" is runtime metrics, not the compiler; extension of Theorem 3.1 — runtime monitoring as the continuous guarantee of "state observability."

Epistemic Status and Formal Anchors

Theoremizable (loop stability proven, zero sorry)

The module

`Pattern23_24_ProductionLoop`

(corresponding to Appendix B.10) formalizes the observation–learning loop of Pattern 23 → 24 → 23.

- Model: a stock of defects $x \in \mathbb{N}$; each cycle, one is detected and repaired with probability `q`, otherwise one is injected.
- `T_id_pos`: $\mathbb{E}[x' \mid x] = x + 1 - 2q$ $(x \geq 1)$.
- `hasStrongDrift` + `loop_return_bound`: when $q > 1/2$, strong drift is satisfied with $V(x) = x$, $b = 1$, $\delta = 2q - 1$, and $\mathbb{E}[\tau] \leq x_0/(2q - 1)$.
- `no_drift_below_half` (critical negative result): when $q \leq 1/2$, $V = \mathrm{id}$ is not a Lyapunov function.

Implication for the numbers in this pattern: the in-text "detection rate ≥ 0.90" is not an optional performance target but the engineering margin of the $q > 1/2$ stability threshold; below the threshold the loop cannot self-certify convergence. The measured value of q is provided by production field experiment H1 (fixed-threshold defect classes $q = 1.00$; mem adaptive-σ criterion-shape boundary, 2026-10-22); this formalization covers only a single defect class, single-step injection / repair, and i.i.d. detection, and is not extrapolated to multi-defect-class coupling.

Empirical Support (2026-10-22, Production Field Experiments H1/H4)

Measured environment: a Docker Compose controlled production topology (app/downstream/toxiproxy/k6/exporter/auditor/sniffer; the LLM component DeepSeek-V4.1-Flash-1017 appears only in the defect-crystallization step); all criteria preregistered (fixed thresholds pool>10 / p99>300 ms / mem 3σ; detection window 180 s; 10 reps per scenario). Interpretation follows: F1/F2 detection rate 1.00, F4 false positives 0.00, F3 criterion-shape boundary, H4 rig defect.

- **F1 connection-pool leak** (10 reps): detection rate 10/10 (q=1.00), alarm evidence `pool_active>10.0 (751)`. The fixed-threshold criterion is effective against resource-pool-exhaustion defects.
- **F2 downstream latency** (toxiproxy 400 ms, 10 reps): detection rate 10/10 (q=1.00), alarm evidence `p99>300.0 (40x ms)`. Latency-type defects are reliably detected under the preregistered p99 threshold.
- **F4 control** (10 reps × 6 sampling windows = 60 periods): false-positive rate 0.00 ($\leq$0.05). Fixed thresholds produce no false positives under steady-state traffic.
- **F3 memory leak (boundary)**: the mem 3σ adaptive criterion works only for an "extremely stable baseline + large jump" (measured: dev triggered once at 31.1σ), and goes **insensitive to repeated step injections** — the EWMA σ is contaminated by the signal itself (inflate from 4 MB to 67 MB), and the drifted 3σ threshold swallows subsequent signals. **This is empirical evidence of a mismatch between the criterion shape (adaptive σ) and the defect shape (leak / drift), not a criterion failure**; the engineering takeaway: leak-type monitoring should use a robust baseline (median / MAD) or fixed absolute thresholds, and the adaptive-σ criterion suits only burst-type spikes.

- **H4 rig defect**: exporter restart with reused sequence numbers caused 708 normal events to be falsely rejected by the mtime criterion (all with reason `mtime_not_strict_increasing`; no hash / fingerprint false rejections); rechecking the 456 snapshots found 0 violations of filename-order monotonicity — the false rejections were a rig defect, not a criterion failure. The rig has been fixed (exporter continues numbering; the auditor audits by time order).
- Conclusion: the measured q is 1.00 for fixed-threshold defect classes (pool/p99), giving empirical support to the loop-stability threshold $q > 1/2$; the mem adaptive-σ criterion does not meet this threshold (a criterion-selection issue) and is faithfully recorded as a boundary per plan §8.

Chapter 27: Pattern 24 — Production Defect Crystallization & Hotfix Loop Pattern

---

1. Intent

Crystallize defects found in production into **empirical assets**, trigger the **hotfix loop**, and prevent the same class of defect from propagating across generations. Through **quarantined crystallization of defect patterns, double-blind final review of hotfix proposals, and anti-proliferation control of repair experience**, achieve the complete "discover → repair → prevent" closed loop for production defects.

---

2. Motivation & Real-World Failure Modes

Defects found in production are the most precious source of experience for an agent system, but uncontrolled defect feedback causes severe knowledge contamination:

- **Failure Mode A (Viral Propagation of Wrong Fixes)**: a database connection leak is found in production, and agent A's hotfix is to "force-close the connection after every query." This proposal is written into the global knowledge base without final review, and subsequent agents B, C, D all adopt it in similar situations. Yet under high concurrency this proposal exhausts the connection pool, whereas the correct fix is to "use try-with-resources to ensure automatic connection release." The propagation of the wrong fix causes systematic performance degradation;
- **Failure Mode B (Unvalidated Hotfix Deployment)**: an urgent defect appears in production, and the agent deploys a hotfix directly without completing the five-fold criteria validation, so the repair introduces a new defect (e.g., fixing a null-pointer exception by introducing an array-out-of-bounds exception);
- **Failure Mode C (Environment-Specific Leakage of Defect Experience)**: the agent crystallizes "the production database connection pool's maximum connections must be set to 200" as a universal experience; a later agent adopts this in a test environment (connection-pool limit 50), causing a connection-pool configuration error.

```
flowchart TD
    subgraph DevEnvironment["Development environment (accessible to the agent)"]
```

```
    Dev["Agent development"] --> GitCommit["Git commit"]
end

subgraph IsolatedEnvironments["Isolated environment matrix (inaccessible to the agent)"]
    direction TB
    StressTest["Stress-test environment<br/>(isolated)"]
    Production["Production environment<br/>(isolated)"]
end

subgraph BridgeLayer["Protocol bridge layer (Pattern 12)"]
    direction TB
    Bridge1["Stress-test results<br/>asynchronous feedback"]
    Bridge2["Production metrics<br/>asynchronous feedback"]
end

subgraph AuditLayer["Independent audit layer (Pattern 02, Probe 4)"]
    direction TB
    Audit1["Stress-test audit"]
    Audit2["Production-metrics audit"]
end

GitCommit --> StressTest
StressTest --> Bridge1
GitCommit --> Production
Production --> Bridge2
Bridge1 --> Audit1
Bridge2 --> Audit2

Audit1 --> DeployDecision["Deployment decision"]
Audit2 --> DefectCrystallization["Defect crystallization loop (Pattern 24)"]

subgraph DefectCrystallization["Defect crystallization loop (Pattern 24)"]
    direction TB
    Extractor["Defect-pattern extractor<br/>(development)"]
    Hotfix["Hotfix generator<br/>(development)"]
    Validator["Five-fold-criteria validator<br/>(stress-test environment)"]
end

DefectCrystallization --> Promotion["Final-review promotion<br/>(Pattern 14)"]
Promotion --> Verified["VERIFIED_ACTIVE<br/>(global knowledge base)"]
Verified --> Prevention["Immunity to same-class defects<br/>(retrieval interception in l
```

---

3. Applicability

- A defect is found in production and a hotfix must be generated quickly;
- Defect experience must be crystallized into reusable knowledge assets, preventing recurrence of the same defect;

- Wrong fix patterns or unvalidated hotfix proposals must be prevented from contaminating the global knowledge base.

---

4. Participants and Responsibilities

1. **Defect Pattern Extractor**: deployed in the development environment, it extracts defect root causes, fix strategies, and preventive measures from production anomaly events that have **flowed back through the protocol bridge and been audited**; **key constraint**: it does not directly access production, consuming only audited anomaly events;
2. **Hotfix Generator**: deployed in the development environment, it generates candidate hotfix proposals based on the defect pattern, marked `DRAFT_SUSPECT` by default; **key constraint**: hotfix validation is performed in the development or stress-test environment, not deployed directly in production;
3. **Five-Fold Validator**: runs the complete five-fold criteria (compilation / unit test / integration / performance / security) on the hotfix proposal; **key constraint**: the performance criterion runs in the stress-test environment, simulating production load;
4. **Reality Auditor**: verifies the hotfix proposal's authenticity (not the agent's self-report), integrity (no selective reporting), and homogeneity (consistency with production); **key constraint**: the auditor obtains raw production data through the protocol bridge and does not access it directly;
5. **Promotion Gate**: only after the hotfix proposal passes both the five-fold criteria and the independent audit fully green may it be promoted to `VERIFIED_ACTIVE`.

---

5. Collaborations and Causal Constraints

```
sequenceDiagram
    autonumber
    actor Prod as Production environment (isolated)
    participant Bridge as Protocol bridge (Pattern 12)
    participant Auditor as Independent auditor (Pattern 02, Probe 4)
    participant Extractor as Defect-pattern extractor (development)
    participant Hotfix as Hotfix generator (development)
    participant Validator as Five-fold validator (stress-test)
    participant Gate as Promotion gate
    participant KB as Global knowledge base

    Prod->>Bridge: Production anomaly event (metrics/logs/traces)
    Bridge->>Auditor: Forward anomaly event + integrity check
    Auditor->>Auditor: Verify authenticity/integrity/homogeneity
    Auditor-->>Extractor: Deliver audited anomaly event

    Extractor->>Extractor: Dual-mode crystallization: separate factual knowledge from pattern kno
    Extractor->>Hotfix: Submit defect pattern (root cause + fix strategy)
    Hotfix->>Hotfix: Generate candidate hotfix proposal, mark DRAFT_SUSPECT
    Hotfix->>Validator: Submit hotfix proposal + test cases
```

```
    note over Validator: Runs in stress-test environment, simulating production load
    Validator->>Validator: Run five-fold criteria (compilation/unit/integration/performance/secur

    alt Five-fold criteria not fully green
        Validator-->>Hotfix: Reject: the repair introduces a new defect
    else Five-fold criteria fully green
        Validator->>Auditor: Submit validation report + raw metrics
        Auditor->>Auditor: Independent audit (authenticity/integrity/homogeneity)

        alt Audit fails
            Auditor-->>Hotfix: Veto: CRITICAL_DECEPTION
        else Audit passes
            Auditor->>Gate: Submit audit-passed report
            Gate->>KB: Promote to VERIFIED_ACTIVE (global knowledge base)
            KB-->>Prod: Retrieval interception in later tasks: immunity to same-class defects (sy
        end
    end
```

Causal constraints:

- The mutation causality of the hotfix is monotonic (Axiom III), with **cross-environment synchronization** — production mtime jumps are synchronized through the bridge layer to the development environment, supporting rollback tracing;
- Quarantined promotion of defect experience (Axiom IV): unvalidated hotfix proposals are strictly forbidden from propagating laterally;
- Orthogonal decoupling of factual and pattern knowledge (Pattern 13): prevents environment-specific parameters from leaking into universal experience;
- **Cross-environment audit**: after production anomaly events flow back through the protocol bridge, they must pass Pattern 02 Probe 4's audit before the extractor may consume them.

---

6. Consequences

- **Pros**:
  - **Complete closed loop for production defects**: from anomaly discovery through hotfix deployment to experience crystallization, a full closed loop is formed;
  - **Prevention of wrong-fix-pattern propagation**: a hotfix proposal may enter the global knowledge base only after the five-fold criteria + independent audit + final-review promotion;
  - **Immunity to same-class defects**: already-verified defect patterns are automatically intercepted during later-task retrieval, preventing recurrence.
- **Cons**:
  - The hotfix validation flow adds deployment latency and is unsuitable for emergency fixes requiring zero downtime;

- Crystallizing and maintaining defect experience adds knowledge-base management cost;
- Production monitoring and auditing increase operational complexity.

---

7. Anti-Patterns to Avoid

Anti-pattern	Manifestation	Why it must be prohibited
Emergency fix bypassing validation	When a production defect is urgent, the five-fold criteria are skipped and the hotfix is deployed directly	The repair may introduce a new defect; it must pass full validation before going live
Immediate ingestion of defect experience	After a production defect is fixed, it is written into the global knowledge base immediately, without a quarantine period	Wrong fix patterns may propagate virally; it must pass final-review promotion
Generalization of environment parameters	Production-specific parameters (e.g., connection-pool size, timeout thresholds) are crystallized as universal experience	Later tasks adopt this experience in different environments, causing configuration error

---

8. Algorithmic Formulation

```
interface ProductionDefect {
  defectId: string;
  anomalyEvent: AnomalyEvent;  // from Pattern 23
  rootCause: string;
  fixStrategy: string;
  preventionMeasures: string[];
  environmentFingerprint: EnvironmentFingerprint;  // production-specific parameters
}

interface HotfixProposal {
  proposalId: string;
  defectId: string;
  status: "DRAFT_SUSPECT" | "VERIFIED_ACTIVE" | "DEPRECATED_QUARANTINED";
  codeChanges: CodeChange[];
  testCases: TestCase[];
  validationReport?: FiveFoldValidationReport;
  auditReport?: RealityAuditReport;
}

class ProductionDefectCrystallizer {
  async crystallizeDefect(anomaly: AnomalyEvent): Promise<ProductionDefect> {
    // 1. Dual-mode crystallization: separate factual from pattern knowledge
      const factKnowledge = this.extractFactKnowledge(anomaly);   // production-
specific parameters
```

```
   const patternKnowledge = this.extractPatternKnowledge(anomaly);  // defect root cause and fix stra

    // 2. Produce the defect pattern
    return {
      defectId: generateUUID(),
      anomalyEvent: anomaly,
      rootCause: patternKnowledge.rootCause,
      fixStrategy: patternKnowledge.fixStrategy,
      preventionMeasures: patternKnowledge.preventionMeasures,
    environmentFingerprint: factKnowledge.environmentFingerprint,  // stored in quarantine, does not
    };
  }
}

class HotfixLoopOrchestrator {
  async executeHotfixLoop(defect: ProductionDefect): Promise<HotfixProposal> {
    // 1. Generate a candidate hotfix proposal (default DRAFT_SUSPECT)
    const proposal = await this.generateHotfixProposal(defect);
    proposal.status = "DRAFT_SUSPECT";

    // 2. Five-fold criteria validation (Pattern 19)
    const validationReport = await this.executeFiveFoldValidation(proposal);
    if (!validationReport.allPassed) {
      proposal.status = "DEPRECATED_QUARANTINED";
      return proposal;  // reject: the repair introduces a new defect
    }

    // 3. Independent audit (Pattern 02, Probe 4)
    const auditReport = await this.executeRealityAudit(proposal, validationReport);
    if (auditReport.isVetoed) {
      proposal.status = "DEPRECATED_QUARANTINED";
      return proposal;  // veto: CRITICAL_DECEPTION
    }

    // 4. Final-review promotion (Pattern 14)
    proposal.validationReport = validationReport;
    proposal.auditReport = auditReport;
    proposal.status = "VERIFIED_ACTIVE";

    // 5. Enter the global knowledge base; intercepted during later-task retrieval
    await this.promoteToGlobalKnowledgeBase(proposal);

    return proposal;
  }
}
```

Theoretical anchors

Extension of Axiom IV — the inverse operation of production defects as "strain viruses"; defect experience may enter the global knowledge base only after final

review; extension of Axiom III — the mutation causality of the hotfix is monotonic, and production mtime jumps must be strictly recorded to support rollback tracing.

Epistemic Status and Formal Anchors

Theoremizable (loop stability proven, zero sorry)

Shares the module

`Pattern23_24_ProductionLoop`

(Appendix B.10) with Pattern 23. Pattern 24 corresponds to the "repair action" in the loop — it decrements the stock by one with probability

`q`

— while Pattern 23 corresponds to "detection."

- Stability criterion: when $q > 1/2$, the loop admits a negative-drift certificate, $\mathbb{E}[\tau] \leq x_0/(2q-1)$.
- Critical negative result: when $q \leq 1/2$, $V = \mathrm{id}$ is not a Lyapunov function — repair throughput is insufficient to offset injection, and the hotfix loop degenerates into "chasing after defects."
- This pattern's "wrong-fix interception rate = 1.00" is a **quality threshold** (ensuring each repair introduces no new defect class), and is an independent constraint from the stability criterion: the former bounds the truth value of a single repair, the latter bounds convergence over many cycles.

Scope not covered: multi-defect-class coupling, batch injection, and detection rate varying with stock (learning effects) are all outside the model and constitute empirical gaps; evidence is provided by production field experiments H2/H3/H5 (closed 2026-10-22): audit effectiveness 20/20 fully rejected, promotion ratio 1.00, wrong-fix interception 1.00, environment leaks 0.

Empirical Support (2026-10-22, Production Field Experiments H2/H3/H5)

Measured environment: the LLM closed loop (crystallizer→hotfix-gen→validator→gate) calls DeepSeek-V4.1-Flash-1017 through the scnet gateway (temperature 0.7, max_tokens 2000/4000, json_object); the audit / gate components are all deterministic (hash, mtime, fingerprint, five-fold criteria, three-condition final review). Interpretation follows: H2 zero missed tampering detection, H3 closed-loop effectiveness, H5 applicability-domain decoupling.

- **H2 audit effectiveness**: all 20/20 tampering injections rejected (hash 8/8, mtime rollback 6/6, fingerprint drift 6/6) — the deterministic audit chain has zero missed detections across the three tamper types.
- **H3 closed-loop effectiveness**: 18 hotfix reports → 14 promoted fully green, 4 correctly intercepted (the interceptions were all pre-fix historical defects caught by compilation); promotion ratio 1.00 (threshold ≥0.80), wrong-fix interception 1.00.
- **LLM crystallization quality**: 7/7 defect-pattern extractions correctly identified the root cause (conn_leak + missing try/finally), model labeled `DeepSeek-V4.1-Flash-1017`

— a weak-reasoning model suffices for the "defect crystallization + hotfix generation" step (the theoretical guarantee is carried by the deterministic components; model strength affects only the quality of the extracted text).

- **H5 applicability-domain decoupling**: promoted entries had 0 environment-parameter leaks (regex audit `pool|timeout|...|[=:]\d`); the LLM output leaked no deployment-environment details (the environmentFingerprint exemption rule took effect).
- Conclusion: both the "wrong-fix interception rate = 1.00" quality threshold and the "promotion ratio $\geq$0.80" throughput threshold are empirically supported; the closed loop still holds under the weak-model tier (Flash), supporting Axiom IV's design promise of "inter-component causal minimization."

Chapter 28: Epilogue — Towards Autonomous and Resilient Cybernetics

> "What we have created is not merely a tool for writing code, but a cybernetic living being endowed with causal invariance, perception of physical reality, and the capacity for auto-immune convergence." (Rhetorical expression; the mechanism referred to is the empirical auto-immune convergence of Axiom IV, not a biological term.)

---

1. Crossing the Canyon of "Random Hallucination"

In the years since the explosion of artificial intelligence, humanity's quest to let machines program autonomously has gone through three waves: 1. **First wave (the era of completion)**: inline suggestions exemplified by Copilot and Tab autocompletion. Human engineers still bear 99% of system control and error-bailout responsibility; 2. **Second wave (the era of prompt stacking)**: chat-based programming exemplified by open-source agent frameworks. Although it introduced tool calling and multi-step loops, it repeatedly collapses when facing million-line large production projects, mired in the quagmire of "discrete prompting" and "reward hacking"; 3. **Third wave (the era of cybernetics and physical anchoring)**: precisely the **Industrial Agentic Cybernetics** defined in this book.

This book's core contribution, established through cross-validation of controlled empirical evidence and formal mechanized checking, is that: **the inherent uncertainty and probabilistic hallucination of large language models can never be eliminated by yet another set of longer prompts; one must introduce classical systems cybernetics, distributed causal ordering, the low-level physical constraints of the compiler, and least-privilege projection, building an unbreakable exoskeleton from a deterministic physical system.**

---

2. The System Exoskeleton Built by 24 Patterns

Reviewing the 9 pattern families summarized in this book: - **Substrate penetration and reality audit (Patterns 01-03)**: severs the model's self-referential deceptive instinct, anchoring every Exit 0 to objective facts of the physical world; - **Least-privilege projection and macro-micro elaboration (Patterns 04-06)**: tames context rot (Context Rot), keeping the agent maximally focused in the face of large-scale engineering; - **Pairing micro-loops**

and idempotent clock compensation (Patterns 07-09): breaks the self-healing black hole, ensuring rigorous monotonicity between code mutation and the causal order of the compile system; - **Substrate lease locks and multi-agent coordination (Patterns 10-12)**: ends the chaos and deadlock of multi-agent concurrent competition; - **Experience quarantine and the promotion gate (Patterns 13-15)**: endows the system with an immune system against memory poisoning (Memory Poisoning), giving the agent's experiential evolution positive compounding; - **Multiresolution projection and flight direction (Patterns 16-17)**: realizes high-bandwidth, low-cognitive-load asymmetric collaboration between human and agent; - **Hidden Oracle and objective physical sandboxes (Patterns 18-20)**: sets up an industrial-grade yardstick of truth with no dilution whatsoever; - **Efficiency economics and native architecture (Patterns 21-22)**: converges expensive large-model computation to a commercially viable productivity frontier; - **Production resilience and the feedback loop (Patterns 23-24)**: through runtime anomaly sniffing and defect-crystallization hotfix loops, realizes self-repair and empirical crystallization in the production field.

These 24 patterns serve as mutual pivots, together constituting the solid foundation of modern autonomous software engineering.

---

3. The Horizon Ahead: A Self-Sustaining Autonomous Code Ecology

Once agent systems possess substrate-penetrating capability, self-healing resilience, and immune-style knowledge crystallization, the production relations of software engineering are undergoing an unprecedented qualitative change:

The software engineer of the future will shift from being **"the direct writer of code"** to being **"the designer of cybernetic systems and the setter of physical baselines"** — defining the system's invariant axioms, writing high-density hidden oracles, and configuring the routing matrix of least-privilege projection, while leaving the endless details of implementation, compilation adaptation, cross-module integration, and performance tuning to this tireless, deterministically convergent industrial-grade agent pipeline.

The road ahead is long and far; I will search high and low. This book is dedicated to all the system builders who pursue rigor in the physical world of code and explore the limits of autonomous intelligence.

Appendix B: Formal Proofs of Convergence and Stability (Lean 4 Mechanized)

This appendix is the mechanized foundation of the book's theoretical claims. All theorems are kernel-verified in Lean 4 (toolchain v4.34.0-rc2) + mathlib4; the project lives at `formal_proof/IACFormalization/`, and the build command is `lake build`.

Statistics: 118 [Theorem] entries (117 theorems + 1 auxiliary lemma, kernel-verified, counted by `theorem`/`lemma` in the source), **0 axioms**, 0 `sorry`.

A three-tier epistemic annotation runs throughout this appendix: - **[Theorem]**: fully verified by the Lean kernel, with no axiom dependencies (or depending only on explicit assumptions / structural premises); - **[Cited Result]**: results established in the mathematical literature but for which mathlib does not yet contain a ready-made formalization (this appendix carries no such dependencies); - **[Empirical Postulate]**: empirical propositions about the actual behavior of large models. They are formalized as conditional theorems: the premises are explicit structural assumptions (e.g., the context-quality growth law of the attention mechanism), and the conclusions are derived by kernel verification; "real systems satisfy this premise" is supported by measurement and is not asserted within the formal system.

B.1 Module–Chapter Correspondence Master Table

Lean Module	Book Chapter	Mathematical Tools	Proof Status
`Axiom1_Grounding`	§3.2 Axiom I; Chapter 2	Algebra of finite joint distributions	All [Theorem]; independence is an explicit assumption
`Axiom2_AttentionDilution`	§3.2 Axiom II; Pattern 04	Real analysis (softmax monotonicity), three-channel attention mechanism + growth laws (geometric / linear / power-law)	All [Theorem]

Lean Module	Book Chapter	Mathematical Tools	Proof Status
`Axiom2_PhaseModel`	§3.2 Axiom II two-phase dynamics	Two-phase structure, explicit phase-transition point, non-reversal upper bound	All [Theorem], zero axioms
`Axiom2_Axiom4_Bridge`	§3.2 Axiom II ↔ Axiom IV cross-axiom interface	Injection coupling, lower-bound synthesis	All [Theorem], zero axioms
`Axiom3_CausalMonotonicity`	§3.2 Axiom III; Pattern 09	Order theory, idempotent operators	All [Theorem], zero axioms
`Axiom4_MemoryPoisoning`	§3.2 Axiom IV; Patterns 14/15	Branching processes, elementary probability (PMF + Markov inequality)	All [Theorem], zero axioms
`Probability_Markov`	§3.2 Axiom IV probability toolkit	Markov inequality (elementary proof, zero measure-theoretic dependencies)	All [Theorem], zero axioms
`Theorem3_1_CausalInvariance`	§3.3	Foster–Lyapunov discrete kernel, finite-state-machine counterexamples	All [Theorem], zero axioms
`GaltonWatson`	§3.2 Axiom IV supercritical strong conclusions	Generating functions, fixed points, elementary analysis	All [Theorem], zero axioms
`StochasticFosterLyapunov`	§3.3 stochastic control	Stochastic kernels, stopping times, elementary expectations	All [Theorem], zero axioms
`Pattern02_RealityAudit`	Pattern 02	Four-fold reality filter operator, veto power	All [Theorem], zero axioms
`Pattern04_PoLPProjection`	Pattern 04	Task-level least-privilege projection, strict SNR improvement, violation-risk reduction, and privilege monotonicity	All [Theorem], zero axioms
`Pattern10_LeaseLock`	Pattern 10	Discrete clock, lease state machine	All [Theorem], zero axioms
`Pattern11_PrimaryWriter`	Pattern 11	Write journal, strictly increasing clock, view filtering	All [Theorem], zero axioms

Lean Module	Book Chapter	Mathematical Tools	Proof Status
`Pattern15_RevocationTombstone`	Pattern 15	Absorbing-state transition, projection isolation, reverse auto-immunity	All [Theorem], zero axioms
`Pattern23_24_ProductionLoop`	Patterns 23/24	Finite-support stochastic kernel, Foster–Lyapunov drift	All [Theorem], zero axioms
`Theorem3_1_LyapunovFuse`	§3.3; Appendix B.7	Consecutive-stall energy-accumulation lemma, limit-cycle-repelling strict Lyapunov supermartingale theorem, stopping-time-fuse resource-boundedness theorem	All [Theorem], zero axioms

---

B.2 Axiom I: The Irreplaceability of Grounding (All Proven at the Formalization Layer)

The probabilistic-limit formulation of Axiom I in the main text states an engineering position; its rigorously provable core lives in this module:

- **Theorem `self_verdict_zero_information` (Self-Verification Zero-Information Theorem)**: in a closed cognitive loop with conditionally independent channels (no external physical sensors whatsoever), the posterior truth rate of a passed self-check equals the prior truth rate: $P(V_t = 1 \mid V_s = 1) = P(V_t = 1)$. The Bayesian likelihood ratio of self-check evidence is identically 1.
- **Theorem `closed_loop_risk_factorizes`**: inside the closed loop, the probability of "self-check claims success and the physical state is also true" is fully locked by the prior-product structure; any pure reflection strategy that introduces no external observation cannot change it.
- **Theorem `posterior_independent_of_mechanism`**: the posterior truth rate is independent of the complexity of the self-check mechanism — the formalization rules out the fantasy that "a stronger model can break through by introspection."

Physical Exit 0 penetration (Pattern 01) is therefore not a matter of "raising confidence," but of "the sole channel through which any evidence can be obtained."

B.3 Axiom II: Attention Dilution (All Proven at the Formalization Layer)

Model (structure `AttentionMechanism`): the attention mass on the target token $a(\rho)$, the **total non-target mass** $b(\rho)$, and the **residual subchannel** $r(\rho)$ (fixed occupancies such as BOS / attention sink, instruction tokens, etc.), satisfying $a, b > 0$, $0 \le r < b$, with added redundancy not strengthening the target signal ($a(\rho+1) \le a(\rho)$) and the total non-target mass non-decreasing ($b(\rho) \le b(\rho+1)$). The attention share is $a/(a+b)$, and the retrieval signal-to-noise ratio is

$$\mathrm{snr}(\rho) := \frac{a(\rho)}{b(\rho)}.$$

Calibration note (A2): the theorem premises are borne on the **total non-target mass** b; measurements can only tally the **filler subchannel** filler $:= b - r$. The relation between the two is characterized by two theorems:

- **Theorem `snr_le_proxy`**: the measured filler-caliber a/filler systematically **overestimates** the true snr, by a bias factor b/filler ≥ 1 — tallying only the filler segment underestimates the severity of dilution (E3b measured overestimation of 35.8%–36.7%, in the same direction);
- **Theorem `filler_growth_not_sufficient`** (counterexample): that only the filler subchannel expands geometrically is **not sufficient** to guarantee the total-mass premise — the growth premise must be imposed on b; the object of the premise cannot be shifted.

Basic dilution theorems:

- **Theorem `attention_dilution_strict`**: fixing the target mass $a > 0$, if the total mass of irrelevant context grows $b \to b' > b$, then the attention share on the target token strictly decreases: $\frac{a}{a+b'} < \frac{a}{a+b}$. Dilution is inevitable in the strict-inequality sense, not merely "may be affected."
- **Theorem `softmax_share_strictly_decreases`**: the above conclusion as an abstract instance for an arbitrary positive weight function (e.g., the softmax parameterization $w = \exp \circ \mathrm{logit}$).
- **Theorem `attention_share_dilution`**: when the total non-target mass strictly increases ($b(\rho) < b(\rho+1)$), the attention share strictly decreases — qualitative dilution is an algebraic necessity of the attention formula.

Three corresponding theorems mapping growth laws to decay rates (all premises are explicit structural assumptions; the measured shape is determined empirically):

- **Theorem `snr_geometric_decay`**: b expands by at least a factor $(1+\gamma)$ per step ($\gamma > 0$) $\Longrightarrow \mathrm{snr}(\rho+1) \le c \cdot \mathrm{snr}(\rho)$, where $c = 1/(1+\gamma) < 1$ is a **definition**, not an independent assertion. The premise is a structural assumption about "how the attention denominator expands" (the cause); the conclusion is a quantitative bound on "how the signal-to-noise ratio decays" (the effect).
- **Theorem `snr_polynomial_decay`**: b grows only linearly (increasing by at least $\delta > 0$ per step) $\Longrightarrow \mathrm{snr}(n) \le a(0)/(b(0) + n\delta)$, i.e., $O(1/n)$ polynomial decay.
- **Theorem `snr_power_decay` (power-law branch, A4)**: b expands per step by at least the **exact power ratio** $((\rho+1)/\rho)^\beta$ ($\beta > 0$) $\Longrightarrow$ power decay of the form $\mathrm{snr}(n) \le a_0/(b_0\, n^\beta)$. Supporting lemmas: `b_power_growth` (telescoping-product lemma for the chained power ratios), `snr_power_decay_relative`, `snr_power_decay_of_one_add` (the $0 < \beta \le 1$ bridge via the Bernoulli inequality to the literal form $1 + \beta/\rho$; the exact-power-ratio

form avoids the $\rho = 0$ singularity and has no constant loss when $\beta \geq 1$). E3 measurements show the power-law branch dominating in 33/36 cells and the geometric branch receiving 0 votes; this branch is thereby promoted from optional to **necessary**.

`DilutionDynamics` = `AttentionMechanism` + geometric growth factor γ; the one-step decay `hdecay` is a theorem. Its consequences:

- **Theorem `snr_decay_iterated`**: by induction from the one-step geometric decay, `snr(ρ+n) ≤ c^n • snr(ρ)`.
- **Theorem `geometric_decay_strictly_decreases`**: when `snr(ρ) > 0`, geometric decay implies strict monotone decrease.
- **Theorem `violation_prob_lower_bound`**: the violation probability `P_violate(ρ) = 1 - snr(ρ)/snr(0)` satisfies `P_violate(n) ≥ 1 - c^n`, which is equivalent on $\rho \in \mathbb{N}$ to the main-text `P_violate(ρ) ≥ 1-e^{-αρ}` ($c = e^{-\alpha}$).

Two-phase dynamics (`Axiom2_PhaseModel`, A1): `PhaseModel` = `AttentionMechanism` + expansion rate γ + explicit phase-transition point N^*; the dilution-phase expansion law is promised only within $[0, N^*)$:

- **Theorem `snr_dilute_decay`**: the c^n geometric decay bound within the dilution phase (the domain is explicitly truncated to $[0, N^*)$);
- **Theorem `snr_plateau_le`**: in the plateau phase ($n \geq N^*$), the decay does **not reverse** — $\mathrm{snr}(n) \leq \mathrm{snr}(N^*)$: the engineering guarantee degrades in the saturated region to "no longer worsening," rather than to failure;
- **Theorem `violation_prob_plateau_lower_bound`**: a constant lower bound on the plateau-phase violation probability;
- **Theorem `plateau_violates_geometric_bound`** (counterexample): the global geometric bound indeed fails in the saturated region; `global_growth_recovers_decay` shows that merely restoring the global growth premise recovers the geometric bound.

Cross-axiom interface (`Axiom2_Axiom4_Bridge`, A5 / T7): `InjectionCoupling` connects the memory-injection rate μ to Axiom IV through the irrelevance-to-mapping $\rho(\mu)$:

- **Theorem `violation_lower_bound_at_injection`**: at injection rate μ, the lower bound on the violation probability is $1 - c^{\lceil \rho(\mu) \rceil}$;
- **Theorem `injection_strengthens_lower_bound`**: the faster the injection, the tighter the lower bound (monotone amplification);
- **Theorem `violation_under_injection`**: the longitudinal-edge synthesized form; `violation_bound_beyond_plateau` gives the plateau-phase correction.

This is the longitudinal edge of the layered axiom system in which "there is no feedback between layers": the dilution dynamics of Axiom II drive the contamination-propagation lower bound of Axiom IV through the injection rate, so that the monograph is no longer a mere collection of independent models.

> Epistemological note: the formal system asserts that "for any attention system satisfying the corresponding growth law, the corresponding decay bound and violation lower bound hold"; "which branch a real Transformer satisfies" is an empirical proposition. Measurement interpretation (E3/E3b, Qwen2.5-1.5B/7B, context up to 131k, 2026-10): the power-law branch dominates (AIC 33/36 cells), the geometric branch gets 0 votes; the weak premise that a is non-increasing holds directionally, while "a is constant" is falsified ($\widehat{\beta}_a < 0$, CI excludes 0) — measured SNR decay is contributed jointly by numerator shrinkage and denominator expansion; the long-range "declin-

ing phase" of 7B YaRN, retested via the three-channel decomposition, is judged to be mass migration from the filler subchannel to the sink, with the total non-target mass $t = b + r$ non-decreasing throughout (the `hb_nondec` total-mass calibration survives). See Appendix B.15 and `experiments/attention_growth/EXPERIMENT_PLAN.md`.

Proxy exploration of decay parameters (E8, 2026-10-20, closed): γ / β are **free parameters** of the theorems; this appendix does not assume they can be predicted from structural quantities. After three cross-model-family replications (Qwen2.5-7B / 1.5B + Llama-3.2-3B, 486 rows total), none of the three candidate structural quantities (attention entropy, top-1% concentration, key effective rank) **reproduced robustly across all 3 models** — erank satisfies the criterion only on 7B, top1pct survives only on the 1.5B β and 7B λ estimates, and on Llama-3B none of the four structural quantities reaches |Spearman|$\geq$0.5 with CI excluding 0 under either calibration. γ **remains a free parameter**, its value calibrated by measurement. The weak signal worth recording is **directional consistency**: entropy ($-\beta/+\lambda$), top1pct ($+\beta/-\lambda$), erank ($-\beta/+\lambda$) have perfectly consistent directions across the three models — the strength is unstable across scale and family, but the direction may carry mechanistic meaning. A one-off OOM incident was permanently fixed by chunking in `_gram_streamed` (peak <1 GB vs. the original 34 GB).

B.4 Axiom III: Causal Monotonicity (All Proven at the Formalization Layer)

- **Theorem `observability_iff_mtime_strict_mono`**: for a mutation that produces a real diff, **observability $\iff$ the physical clock (mtime / WAL LSN) jumps strictly monotonically**. If the clock is rolled back or stalled (mtApplied $\leq$ lastCompiledAt), the modification is silently swallowed by the incremental compiler (Silent Loss). "The code has changed but the tests still run on old bytecode" receives a rigorous formalization — monotonicity is not an engineering optimization but a logically necessary condition for observability.
- **Theorem `log_append_apply_idempotent`**: the replay operator of an append-only log (WAL) is idempotent — on repeated replay at the same physical instant, the second application's checksum necessarily fails, and the content fields are identical to the first application. Retries are safe and do not pollute the compiled result.

Measurement boundary (E-A3, 2026-10-20): the engineering counterparts of the two theorems above are supported by 28 controlled measurements on Cargo 1.96 (Windows NTFS) (`experiments/attention_growth/axiom3_results.json`): mtime stall / rollback $\Longrightarrow$ silent loss (5/5 each, artifacts retained at the old version); strict forward mtime jump $\Longrightarrow$ rebuild (5/5, and a pure clock jump triggers a rebuild even when the content is unchanged — rebuild triggering is determined purely by the clock channel, observability by the content diff, consistent with the split of the `hdiff` premise); idempotency holds in unchanged rebuilds (5/5 skipped, artifact sha unchanged) and in the git-rollback-retry scenario (semantic output recovered). Two model refinements: ① the criterion reference clock is measured as the "last build completion instant" R, and the silent-loss window is $(0, R]$, wider than the naive "file's old mtime" model — concurrent writes while a build is in progress fall naturally into the loss window; ② the Windows PE header carries a compile timestamp, so recompilation from identical source necessarily differs at the byte

level, and idempotency must be interpreted at the semantic layer (the theorem promises content invariance, not bit-level equality). Applicability domain: mtime-based incremental compilers (Cargo, Make); content-hash build systems (Bazel, Nix) do not rely on mtime jumps, and the silent-loss scenario of this section does not apply to them.

B.5 Axiom IV: The Memory-Poisoning Phase Transition (All Proven at the Formalization Layer)

Model: the cross-generation transmission of not-yet-finally-reviewed empirical assets is a branching process (structure `BranchingProcess`, one finite-support PMF per generation); the selectivity σ of the double-blind quarantine gate compresses the nominal reproduction mean μ into the effective reproduction number $m = \mu(1-\sigma)$.

- **Theorem `expected_population_geometric`**: the expected population $E(n) = m^n E(0)$ (by induction).
- **Theorem `subcritical_expectation_vanishes`**: when $0 \le m < 1$, $E(n) \to 0$ (contraction-mapping limit).
- **Theorem `markov_inequality`** (`Probability_Markov` module): an elementary proof on finite-support PMFs that $P(X \ge 1) \le E[X]$, with zero measure-theoretic dependencies.
- **Theorem `subcritical_extinction_in_probability`**: by the Markov inequality and geometric decay, when $m < 1$ the survival probability $P(Z_n \ge 1) \to 0$ — the quantitative core of the effectiveness of the quarantine promotion gate (Pattern 14).
- **Theorem `quarantine_compresses_offspring`**: the stricter the σ, the smaller the effective reproduction number; full quarantine drives it to zero.
- **Theorem `supercritical_expectation_diverges`**: when $m > 1$ and $E(0) > 0$, $E(n) \to +\infty$. This is the expectation-layer version of the Galton–Watson supercritical explosion theorem.
- **Theorem `galton_watson_non_extinction`** (`GaltonWatson` module): when the offspring mean $m > 1$, the extinction probability $q < 1$, i.e., the lineage survives forever **with positive probability**. The proof takes the elementary route of generating-function fixed-point analysis:
  - define `genFun(s) = Σ p_k s^k`, prove it is monotone and bounded on $[0,1]$, with `genFun(1)=1`; the truncated polynomial `genFun_N` is continuously differentiable and `genFun_N'(1) → m > 1`;
  - **core lemma `exists_lt_sub_id`**: from `genFun'(1)>1` and continuity, choose a truncated polynomial `genFun_N` such that `genFun_N'(s₀)>1`; via the mean-value theorem and tail estimates, deduce the existence of $s_0 < 1$ with `genFun(s₀)<s₀`;
  - the extinction-probability iteration $q_n = \varphi^n(0)$ is monotonically increasing and uniformly upper-bounded by s_0, so its limit $q = \sup q_n \le s_0 < 1$.
- **Theorems `genFun_extinctProb_fixed` / `extinctProb_le_of_fixed` / `galton_watson_extinctProb_exac`** (exact characterization of the extinction probability): q is not merely an upper bound but the **least fixed point** of φ on $[0,1]$: $q = \varphi(q)$, and any fixed point $s \in [0,1]$ satisfies $q \le s$. The proof uses monotonicity squeezing ($q_{n+1} = \varphi(q_n) \le \varphi(q)$, taking the limit gives $q \le \varphi(q)$; polynomial truncation φ_N is continuous, giving $\varphi_N(q) \le q$, then take $N \to \infty$), avoiding reliance on the continuity of φ itself. In the supercritical regime q is therefore fully characterized: $q = \min\{s \in [0,1] : \varphi(s) = s\} < 1$. With no gate ($\sigma = 0$) and $\mu > 1$, contaminated empirical assets not only have unbounded expected load but persist forever with positive probability — the strong-dynamics counterpart of model

collapse.

Measurement boundary (E-A4, 2026-10-21): the model–reality mapping of the above theorem chain was tested in the DeepSeek-V4-Pro-0813 multi-generation memory-propagation experiment (4 conditions × 12 replicates × 6 generations = 288 sessions, with preregistered interpretation preceding the experiment): the full-quarantine case ($\sigma = 1$) of `quarantine_compresses_offspring` measuredly drives the cross-generation propagation of contaminated lineages strictly to zero (all replicates $Z_1 = 0$, no de novo rediscovery); the geometric law of `expected_population_geometric` matches the pooled population sequence in the unquarantined condition (log-linear $R^2 = 0.958$, no generation-to-generation trend, branching assumption holds in the small-population regime); Galton–Watson simulations parameterized by the measured reproduction kernel (per-parent offspring PMF) agree with the theorem chain with no free parameters — the three subcritical variants (C0 measured $m = 0.538$, C1 measured $m = 0.333$, C0 PMF thinned by Bernoulli(0.5) to $m = 0.269$) have extinction frequency $q_{sim} = 1.0000 = q^*$, and the `markov_inequality` bound holds numerically throughout. Two mapping boundaries are recorded faithfully: ① the compression direction of the DRAFT_SUSPECT marker gate on per-entry reproduction agrees with theory (−38%) but is not statistically significant (permutation test $p = 0.15$); the intermediate-σ case of `quarantine_compresses_offspring` (the engineering counterpart of marking rather than quarantining) did not receive significant support at this sample size; ② the direct LLM-layer test of $m = \mu(1-\sigma)$ is undecidable due to write-layer fluctuations and an effective filtered-layer sample $n = 1$, receiving only indirect support at the distribution layer (the thinned variant); the direct LLM-layer test of this mapping requires replicates on the order of $\geq 10^2$. Applicability domain: a single model (DeepSeek-V4-Pro-0813), the pytest test-writing task family, a selective generation-by-generation transcription protocol, and no substrate feedback (SubstratePass channel disabled, purely social transmission); under this protocol the no-gate condition measured $m = 0.56 < 1$ (the transcription mechanism is itself subcritical). **Supercritical case (closed on 2026-10-22)**: the empirical counterpart of `supercritical_expectation_diverges` / `galton_watson_non_extinction` was confirmed in a memory architecture with no transcription decay (full append-only memory store, direct vector-store writes, SUPERCRITICAL plan Flash tier) — propagation in that architecture is of the **linear accumulation–capacity competition** type (per generation $W \equiv 1$, $Z_{n+1} = Z_n + W$), with C0-full $\hat{m} = 1.166$ (CI $[1.156, 1.176]$) and a capacity threshold $W_c \in (8, 16)$; "supercritical" here is engineering semantics (the strain pool keeps accumulating and does not go extinct), and the precise mechanism is accumulation–competition rather than the $m > 1$ multiplication of branching processes; the GW theorem chain's characterization of the selective-transcription regime is unchanged. Data and interpretation: `experiments/attention_growth/axiom4_results.json`, `axiom4_gw_sim.json`, `EXPERIMENT_PLAN_AXIOM4.md` §9, `EXPERIMENT_PLAN_AXIOM4_SUPERCRITICAL.md` §8.

B.6 Theorem 3.1: Convergent Causal Invariance (All Proven at the Formalization Layer)

Sufficiency (proven):

- **Theorem `lyapunov_finite_hitting`**: if the energy function $V : S \to \mathbb{N}$ has strictly negative drift everywhere $\Longrightarrow$ the zero point is hit in at most $V(s_0)$ steps (all physical contracts satisfied). This is the discrete-deterministic core of the Foster–Lyapunov criterion, completed by induction.
- **Theorem `foster_lyapunov_deterministic_bound`**: for a real-valued energy $V : S \to \mathbb{R}_{\geq 0}$, if every step drops by at least δ whenever $V \geq \delta$ (strong drift $V(\text{step}\, s) \leq V(s) - \delta$), then the number of steps t to hit the δ-neighborhood satisfies $t \cdot \delta \leq V(s_0)$, i.e., $t \leq V(s_0)/\delta$.
- **Theorem `zero_energy_is_fixed`**: a zero-energy fixed point $\Longrightarrow$ the converged state is preserved forever.

Necessity (three proven counterexamples):

1. **Breaking observability** (`no_observability_absorbs_falsified`): the system enters a falsified-pass absorbing state in one step and stays there forever;
2. **Breaking locality** (`no_locality_two_cycle` + `no_locality_defeats_lyapunov`): the system enters a 2-cycle orbit, and **no** energy function can be strictly decreasing everywhere — oscillation cannot be tamed by Lyapunov methods;
3. **Breaking evaluation decoupling** (`coupled_evaluation_tautological_fixed_point`): the evaluator copies the solver's output, forming a self-referential fixed point that converges to a degenerate trivial solution.

Confluent statement `causal_invariance_abstract`: the conjunction of sufficiency and the three necessity counterexamples.

> **Measurement boundary (E-B6, 2026-10-21)**: sufficiency was verified across 12 controlled simulations ($V_0 \in \{10, 50, 100, 500\}$, $\delta \in \{0.5, 1, 2\}$); under the strong-drift condition the convergence steps satisfied $t \leq V_0/\delta$ throughout; the three necessity counterexamples were each reproduced in 10^4 simulations: breaking observability $\Rightarrow$ falsified-pass absorbing state (100%), breaking locality $\Rightarrow$ 2-cycle oscillation with Lyapunov impossibility (100%), breaking evaluation decoupling $\Rightarrow$ self-referential degenerate fixed point (100%). Data: `experiments/attention_growth/b6_causal_invariance_sim.json`.
>
> The expected stopping-time bound for the stochastic case is in B.7.

B.7 Stochastic Foster–Lyapunov Expected Stopping-Time Bounds (All Proven at the Formalization Layer)

The `StochasticFosterLyapunov` module gives expected hitting-time bounds for stochastic controllers by **elementary methods** (stochastic kernel + stopping-time truncation + monotone induction, with no measure theory / martingale optional-stopping-theorem dependencies).

Model: `StochasticKernel S` is a stochastic transition kernel on state space S (`k x A` denotes the probability of landing in set A in one step from x, finite support); the `HasStrongDrift` structure supplies a Lyapunov function $V : S \to \mathbb{R}_{\geq 0}$, a drift constant $\delta > 0$, and a threshold $b \geq 0$, satisfying $\mathbb{E}_{k(x)}[V] \leq V(x) - \delta$ whenever $V(x) \geq b$.

- **Theorems `stoppedValue_le_V` / `stoppedTime_le_value`**: the value and stopping time of the truncated stopping-time process are uniformly controlled by the initial energy (induction core).

- **Theorem `stoppedTime_uniform_bound`**: the expectation of the truncated stopping time τ_N satisfies $\mathbb{E}[\tau_N] \leq V(x_0)/\delta$, uniformly over all truncation levels N.
- **Theorem `foster_lyapunov_expected_hitting`** (main theorem): taking the truncation limit, the expected stopping time to stochastically hit $\{V < b\}$ satisfies $\mathbb{E}[\tau] \leq V(x_0)/\delta$.

Hence the deterministic `foster_lyapunov_deterministic_bound` becomes a special case of this stochastic theorem; the convergent causal invariance of §3.3 acquires a complete expected-stopping-time quantitative bound at the stochastic-control layer.

> **Measurement boundary (E-B7, 2026-10-21)**: the main bound $\mathbb{E}[\tau] \leq V(x_0)/\delta$ was verified across 27 Monte Carlo simulations ($V_0 \in \{10, 50, 100\}$, $\delta \in \{0.5, 1, 2\}$, noise $\sigma \in \{0, 0.1, 0.3\}$, 10^5 runs each), all satisfied; the uniform boundedness of the expected truncated stopping time τ_N holds for $N \in \{10, 50, 100, 500\}$. Data: `experiments/attention_growth/b7_foster_lyapunov_sim.json`.

B.8 Pattern-Level Formal Anchor: The Lease Lock (All Proven at the Formalization Layer)

The `Pattern10_LeaseLock` module formalizes the concurrency claims of Pattern 10 (distributed substrate lease lock and watchdog). Model: the physical clock is a discrete tick; the lock state is `LockState = Option Lease` (the `Option` structure itself encodes "at most one lease"); `acquire`/`heartbeat` are pure functions with preconditions.

- **Theorem `canWrite_unique`** (mutual-exclusion safety): at any instant at most one agent holds write permission — two writers must be the same agent, so concurrent write-write conflicts (failure scenario A) are impossible within the protocol.
- **Theorems `acquire_rejected_of_live` / `acquire_succeeds_only_when_dead`**: an acquire while a lease is alive is necessarily rejected; grant occurs only when no lease is alive.
- **Theorem `no_permanent_lockout`** (watchdog liveness): for any lock state there exists a time T (= expiry instant +1) after which any `acquire` succeeds — the orphan lease of a crashed holder is necessarily reclaimed after TTL expiry, so permanent lockout is engineering-impossible (formal negation of failure scenario B); downtime is bounded.
- **Theorem `not_canWrite_of_expired`**: after expiry, the original holder's write permission is extinguished; a lease does not retain residual privilege from historical holding.
- **Theorems `heartbeat_by_nonholder_unchanged` / `heartbeat_by_holder_extends`**: a heartbeat by a non-holder is rejected and the state is unchanged; a holder's renewal does not shorten the lease (renewal-side protection against failure scenario C).

> **Measurement boundary (E-B8, 2026-10-21)**: the five core properties were each verified in 10^4 discrete-clock state-machine simulations, all passing — mutual-exclusion safety (at most one writer at any instant), watchdog liveness (a crashed holder's lease is necessarily reclaimed at TTL+1), post-expiry write-permission extinction, heartbeat protection (non-holder rejection, holder renewal does not shorten), and impossibility of concurrent write-write conflicts (5 agents, 100 ticks, random contention). Data: `experiments/attention_growth/b8_lease_lock_sim.json`.

B.9 Pattern-Level Formal Anchor: Single-Writer Causal Order in Primary-Writer/Replica-Reader (All Proven at the Formalization Layer)

The `Pattern11_PrimaryWriter` module formalizes the concurrency claims of Pattern 11 (primary-writer / replica-reader degraded mutual exclusion). Model: the write journal is `List (Write Agent)`, each element carrying its writer and physical tick; `ChainValid` requires adjacent ticks to be strictly increasing; the read view `viewAt l t` is the prefix of the journal filtered by `tick` $\leq$ `t`.

- **Theorem `eq_of_mem_of_tick_eq`**: no two distinct events in a valid journal share the same tick — equal ticks mean the same event.
- **Theorem `writer_unique_per_tick`**: write events at the same tick have the same writer, so concurrent write-write conflicts are impossible under the primary-writer/replica-reader protocol.
- **Theorem `writer_unique_of_lease`**: this conclusion follows directly from `Pattern10.canWrite_unique` — "single writer" is not an additional convention but a corollary of lease mutual exclusion.
- **Theorems `view_mono / no_lost_write`**: a later read contains all events of an earlier read; any committed write event whose tick is no later than the read instant is necessarily visible. The replica reader therefore reads a **complete prefix**, with no "skipping middle commits" view.

B.10 Pattern-Level Formal Anchor: Stability of the Observation–Learning Loop (All Proven at the Formalization Layer)

The `Pattern23_24_ProductionLoop` module closes the "Pattern 23→24→23 negative-feedback loop" left open in §7 of the validation report. Model: a defect stock $x \in \mathbb{N}$; each cycle one defect is detected and repaired with probability q ($x \mapsto x-1$), otherwise one is injected ($x \mapsto x+1$); the transition kernel `loopKernel q` is a finite-support stochastic kernel that directly reuses the Foster–Lyapunov framework of B.7.

- **Theorem `T_id_pos`**: when $x \geq 1$, the one-step expected stock satisfies $\mathbb{E}[x' \mid x] = x + 1 - 2q$.
- **Theorem `hasStrongDrift`**: if $q > 1/2$, taking $V(x) = x$, $b = 1$, $\delta = 2q - 1 > 0$, the strong-drift condition holds.
- **Theorem `loop_return_bound`**: the expected time for the loop to hit "stock < 1" satisfies $\mathbb{E}[\tau] \leq x_0/(2q-1)$ — the observation–learning closed loop is exponentially recoverable when $q > 1/2$.
- **Theorem `no_drift_below_half`** (critical negative result): if $q \leq 1/2$, then $\mathbb{E}[x' \mid x] \geq x$, and $V = \mathrm{id}$ does not constitute a Lyapunov function. Hence $q > 1/2$ is a **necessary threshold within the model**; the "detection rate ≥ 0.90" in the Pattern 23 document should be read as engineering margin above this threshold, not as an optional performance metric.

  Status of the conclusion: this module gives a stability criterion **within the model** (single defect class, single-step injection/repair, i.i.d. detection). It does not cover multi-defect-class coupling, batch injection, or detection-rate variation with stock, which remain the unverified extrapolations declared in §7. Meanwhile, the

measured value of q is supplied by production field experiment H1 (2026-10-22): fixed-threshold defects (pool/p99) have $q = 1.00$, satisfying the $q > 1/2$ threshold; the mem adaptive-σ criterion does not satisfy it due to threshold drift, which is a criterion-shape boundary.

B.11 Pattern-Level Formal Anchor: The Reviewer Reality-Audit Veto Power (All Proven at the Formalization Layer)

The `Pattern02_RealityAudit` module rigorously implements the formalization claims of Pattern 02 (the reviewer reality-audit veto pattern). Model: the deliverable `Deliverable` contains four physical Boolean criteria (physical source diff, compile-time physical plausibility, anti-self-reference mock check, isolated-environment homogeneity); the reality auditor `realityAudit` constructs a four-fold conjunction filter; the admission function `admissionVerdict` requires both that the reality audit passes and that the Rubric score reaches a threshold.

- **Theorem `veto_overrides_high_score`** (absoluteness of the veto power): if the reality audit fails, no matter how high the subjective or Rubric score (even as high as the full 9 points), the admission verdict is necessarily false.
- **Theorem `tautology_vetoed`** (hard block against self-referential cheating): if a test assertion is self-referential echo-cheating (`isTautological = true`), the reality audit necessarily fails and the deliverable is necessarily vetoed.
- **Theorem `substrate_missing_vetoed`** (hard block against missing physical substance): if no substantive business-code change is produced (`hasPhysicalDiff = false`), the deliverable is necessarily vetoed.
- **Theorem `four_pillars_soundness_and_completeness`** (full sufficiency-and-necessity of the four pillars): admission holds if and only if all four physical hard criteria and the score threshold hold simultaneously.

B.12 Pattern-Level Formal Anchor: Tainted-Contamination Defense and Lineage Tombstoning (All Proven at the Formalization Layer)

The `Pattern15_RevocationTombstone` module formalizes the dynamic auto-immunity claims of Pattern 15 (tainted-contamination defense and anti-degradation). Model: the pattern lifecycle is a three-state machine (draft `draft`, promoted `promoted`, tombstone `tombstone`); the knowledge base is a list of patterns; the retrieval projection operator admits only patterns in the `promoted` state; a revoke operation automatically synthesizes a reverse auto-immunity rule set.

- **Theorem `tombstone_absorbing`** (terminal absorbing nature of the tombstone state): the tombstone state is a terminal absorbing state of the state machine; no subsequent state transition can escape it.
- **Theorem `tombstone_never_projectable`** (safety of dynamic projection isolation): a pattern in the tombstone state can never appear in any context retrieval projection slice.
- **Theorem `immunity_blocks_taint`** (reverse auto-immunity blocking theorem): the reverse auto-immunity rule set automatically synthesized from a tombstoned pattern blocks, 100% mechanically, any candidate-pattern generation carrying the same taint signature.

- **Theorem `revoke_produces_tombstone`** (revoke necessarily enters the tombstone state): after a revoke operation, the target pattern is necessarily in the tombstone state in the knowledge base.
- **Theorem `revoked_excluded_from_projection`** (atomic immediate invalidation on revoke): after executing `revokePatternInKB`, the target pattern is atomically set to tombstone and immediately disappears from all task projection views.
- **Theorem `tombstone_offspring_zero`** (generation-propagation cutoff): the effective reproduction mean of a tombstoned pattern is strictly zero, and branching-process offspring are immediately zero in the next generation.

B.13 Pattern-Level Formal Anchor: Task-Level Least-Privilege Context Projection (All Proven at the Formalization Layer)

The `Pattern04_PoLPProjection` module rigorously implements the formalization claims of Pattern 04 (task-level least-privilege context projection pattern).

Model: the task-context environment partitions the effective target mass $a > 0$, the task-necessary background mass $b_T > 0$, and the external global redundancy mass $b_{ext} \geq 0$; the projection operator projectPoLP truncates away the external redundancy mass ($b_{ext} \mapsto 0$). The signal-to-noise ratio is defined as $\mathrm{snr}(a, b) = a/b$, and the out-of-bounds-code violation probability as $P_{\text{violate}}(a, b) = 1 - a/(a + b)$.

- **Theorem `polp_preserves_snr_strictly`** (least-privilege projection fidelity theorem): if non-zero external redundant context exists ($b_{ext} > 0$), then after least-privilege projection the task retrieval signal-to-noise ratio is strictly greater than the full unprojected SNR: $\mathrm{snr}(a, b_T) > \mathrm{snr}(a, b_T + b_{ext})$.
- **Theorem `polp_strictly_reduces_violation_risk`** (out-of-bounds violation-probability reduction theorem): least-privilege projection strictly reduces the model's attention degradation and out-of-bounds code-tampering risk: $P_{\text{violate}}(a, b_T) < P_{\text{violate}}(a, b_T + b_{ext})$.
- **Theorem `polp_monotone_nesting`** (privilege-nesting monotonicity theorem): the tighter the projection slice ($b_1 \leq b_2$), the monotonically lower its violation-risk upper bound.

B.14 Supplement to Theorem 3.1: Limit-Cycle Oscillation and Lyapunov-Contract Residual Stopping-Time Fuse (All Proven at the Formalization Layer)

The `Theorem3_1_LyapunovFuse` module supplements Theorem 3.1 with the stopping-time bounds of Appendix B.7.

Model: an energy-residual trajectory $V : \mathbb{N} \to \mathbb{R}_{\geq 0}$, one-step drift $\Delta V(t) = V(t + 1) - V(t)$; the stall criterion is $\Delta V(t) \geq 0$. The Lyapunov-contract residual stopping-time fuse hard-trips after k consecutive steps with no negative drift (consecutive stall).

- **Theorem `consecutive_stalls_accumulate`** (consecutive-stall energy-accumulation lemma): if from time t onward the drift is non-negative for k consecutive steps, then the energy at time $t + k$ is necessarily no lower than the energy at time t: $V(t + k) \geq V(t)$.

- **Theorem `cycle_defeats_strict_lyapunov`** (nonexistence of a strict Lyapunov energy function excluding periodic limit cycles): for any nontrivial limit cycle $s_{t+k} = s_t$ of period $k \geq 1$, there exists no Lyapunov supermartingale energy function that is strictly decreasing everywhere; eliminating oscillation must rely on an external fuse to break the limit cycle.
- **Theorem `fuse_halts_divergence_at_k`** (stopping-time-fuse resource-boundedness theorem): the Lyapunov-contract residual fuse strictly truncates the number of consecutive ineffective self-healing retries to within a constant upper bound k, thoroughly eliminating infinite loops and token black holes.

B.15 Empirical Boundary Declarations Relative to the Main Text

The formalization covers the **dynamical properties of the abstract control model**. Across the entire Lean codebase of the project: **0 axioms, 0 sorry**.

The following are intrinsically non-formalizable; they exist as premises of conditional theorems or as empirical propositions, and the book does not dress them up as proven:

1. The irrelevance-mass growth law of the Transformer attention mechanism and the strength of its premises (the premises of the growth branches in B.3) — "which branch a real system satisfies" is an empirical proposition. Measurement boundary (E3/E3b, Qwen2.5-1.5B/7B, $\rho \leq$131k, 2026-10): the power-law branch dominates and the geometric branch is rejected; the weak premise `ha_nonincr` (a non-increasing) holds directionally, while the strong hypothesis "a is constant" is falsified; the total non-target mass $t = b + r$ is non-decreasing throughout (`hb_nondec` survives), and the long-range fallback of the filler subchannel is judged to be sink migration; when the main text cites measured SNR, the three-channel calibration (target / filler / sink) must be used, and the filler-caliber SNR systematically overestimates by 35.8%–36.7% (`snr_le_proxy`). **Transmission boundary of the violation probability (E4, 2026-10-20)**: `violation_prob(`ρ`) = 1 - snr(`ρ`)/snr(0)` is the relative loss of the signal-to-noise ratio, **not** the task-failure probability. The E4 end-to-end experiment (Qwen2.5-1.5B/7B needle-retrieval accuracy aligned with E3 SNR) shows a compensation threshold W^* between the two: at 7B the SNR has already lost 44% at 131k while task accuracy remains 100%; at 1.5B accuracy drops steeply from 100% to 70% over $\rho \in$[32k,64k]. The violation lower bound $P_{\text{violate}} \geq 1 - c^n$ transmits to an observable task failure only when $\rho > W^*$. W^* is positively correlated with model capability ($W^*_{1.5\text{B}} \approx$ 32k–64k $< W^*_{7\text{B}} >$ 131k $< W^*_{\text{Kimi}} \geq$ 1M); it is an empirical parameter and is not written into the formal interface. Applicability domain: Qwen2.5-family models, needle-type retrieval tasks, F1–F3 filler; extrapolation across model families and to real engineering workloads is unverified;
2. **The model–reality calibration mapping** (through which all empirical citations of the B.3 theorems pass): the theorem's a is the attention mass on a **single target token**, while the experiment's a is the total mass of the **entire needle segment**; the theorem's b is the **total non-target mass**, while the experiment's b is the total mass of the **filler segment** (filler $= b - r$). These two mappings are unproven assumptions — their validity is partially endorsed by the residual-channel drift test of E3b (significant drift in r has triggered preregistered item 3, so the three-channel calibration is required, not optional), but "whole-segment mass approximating single-token behavior" has no independent test; when citing measured data in support of the theorem narrative, this declaration must be given alongside it;

3. The incidence rate of reward hacking by a specific large model on a specific task (the Chapter 2 game-theoretic analysis gives strategy existence, not incidence);
4. The measured distribution of contract violations in industrial engineering.

What the formal proofs guarantee is this: **under the given premises (independence assumptions, growth laws, etc.), the causal structure asserted by the four axioms and by Theorem 3.1 holds logically**.

B.16 Reproduction Instructions

```
cd formal_proof/IACFormalization
lake build        # full kernel verification
lean src/IACFormalization/Axiom1_Grounding.lean   # single-file check
```

Dependencies: the machine's shared prebuilt mathlib4 package (referenced by absolute path in `lakefile.lean`); zero-sorry policy — any occurrence of `sorry` is treated as a build defect.